



Computer Programming I (Java I)

COS221

Covenant Centre for Open
and Distance eLearning

ccodel.covenantuniversity.edu.ng

Table of Contents

Module One: Introduction to Computer Programming	4
Unit 1 — Overview of Computer Programming	5
Module Two: Java Setup	14
Unit 1 — Introduction to Java and Java Development Environment	15
Module Three: Java Fundamentals	22
Unit 1 — Java Basics-Data Types, Variables and Operations In Java Part 1	23
Unit 2 — Java Basics-Data Types, Variables and Operations in Java Part II	41
Module Four: Java Control Structures	54
Unit 1 — Conditional Statements and Decision Making	55
Unit 2 — Loops and Branches	Error! Bookmark not defined.
Module Five Methods	84
Unit 1 — Methods and Parameter Testing	85
Module Six: Object-Oriented Programming (OOP)	104
Unit 1 — Object-Oriented Programming (OOP) Basics	105
Unit 2 — Object-Oriented Programming (OOP) Advanced	127
Module Seven: Strings and String Processing	146
Unit 1 — Data Structures and String Handling	147
Module Eight: Arrays and Arrays Manipulation	159
Unit 1 — Arrays and Arrays Manipulation	160
Module Nine: Advanced Programming Techniques	175
Unit 1 — Introduction to Recursion	176
Module Ten: Exception handling and File I/O	189
Unit 1 — Exception Handling and File I/O	190
Answers to Self-Assessment Questions	207
Module One	208
Module Two	208

Module Three	208
Module Four	209
Module Five	210
Module Six	211
Module Seven	212
Module Eight	212
Module Nine	213
Module Ten	213
GLOSSARY	215

Module One

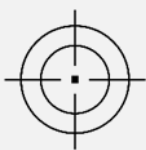
Introduction to Computer Programming

Unit 1 — Overview of Computer Programming

INTRODUCTION

Computer programming is the process of writing instructions that enable a computer to perform specific tasks. It is the backbone of modern software development, driving innovation in fields such as artificial intelligence, web development, data analysis, and automation. Through programming, individuals can build applications, process information efficiently, and solve complex problems.

This unit provides an introduction to the fundamental principles of programming, including different programming paradigms, their key distinctions, and their practical applications. By the end of this unit, you will have a solid grasp of programming basics and will be able to write a simple Java program.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Explain computer programming and its significance.
- Identify various programming paradigms and their associated programming languages.
- Identify basic concepts like attributes, methods, class and objects in a written Java program

1.1.1 What is Computer Programming?

Computer programming, or simply programming, is the process of instructing a computer to perform specific tasks through code. The person who writes these instructions is known as a computer programmer, and they use specialised languages like Java, JavaScript, Python, C, C++, C#, etc to write the instructions for the computer.

Similar to how an author uses a natural language (e.g., English or French) to craft a book, a programmer uses programming languages to create software. Before writing in any language, one must first read and understand existing works in that language. This same principle applies to programming; before you can write a program, you must study and analyse already written code. This helps the learner to understand the structure and logic of the language.

Why is Programming Important?

The following are the importance of programming:

- **Task Automation:** Programming allows repetitive tasks to be automated, saving time and reducing human errors.
- **Software Development:** It enables the creation of websites, apps, and enterprise solutions that drive businesses.
- **Problem-Solving:** Programming teaches logical thinking and problem-solving, which are crucial skills in various fields.
- **Career Growth:** Programming offers diverse high-paying opportunities in software engineering, data science, etc.

1.1.2 Programming Paradigms

A programming paradigm is a style of writing programs. Different programming styles are suited for different types of problems. The following are some programming paradigms:

1. Imperative Programming

This is a step-by-step approach where the program tells the computer how to achieve a task. Key concepts here include the use of variables, loops and conditionals. Code sample of imperative programming paradigm in C:

```
int main () {  
    int x = 5;  
    int y = 10;
```

```
int sum = x + y;
Printf("Sum : %d", sum);
return 0;
}
```

2. Functional Programming

Functional programming focuses on what should be computed rather than how. Functions are treated as first-class citizens. Key concepts include using pure functions and avoiding changing variables (immutability). Examples include Haskell, Lisp, JavaScript (functional style), etc. Code sample of a functional programming paradigm in JavaScript:

```
function add(x, y) {
    return x + y;
}

Console.log(add(5, 10))
```

3. Declarative Programming

This focuses on what the result should be, not how to get there. Examples are SQL, HTML, etc. Code sample of Declarative Programming in SQL:

```
SELECT name, age FROM students WHERE age > 18;
```

4. Logic-Based Programming

This uses rules and logic to derive conclusions. Examples include Prolog, etc. Code sample of Logic-based programming paradigm in Prolog:

```
male (john);
female (susan);
likes (susan, pasta);
parent (susan, alice);
parent (john, bob);
```

5. Object-Oriented Programming (OOP)

This organizes codes using objects (Figure 1.2), which are instances of classes (Figure 1.1). Examples include Java, C++, Python (OOP style), etc. Code sample of OOP paradigm in Java:

```
class Car {
    String model;

    Car (String model) {
        this.model = model;
    }

    void display() {
        System.out.println("Car Model: " + model);
    }
}
```

Figure 1.1 Car Class in Object-Oriented Programming

```
public class Main {  
    public static void main (String[] args) {  
        Car myCar = new Car("Toyota");  
        myCar.display();  
    }  
}
```

Figure 1.2

The code above demonstrates OOP using a simple **Car** class. A class is created **Car**, which serves as a blueprint for creating the car object, **myCar**. The output of the above code is

Car model: Toyota

In-Text Questions (ITQs)

1. What is computer programming?
2. Why is programming considered the backbone of software development?
3. Who writes the instructions that a computer follows?
4. How is programming similar to writing in natural languages like English?
5. List three major benefits of learning programming.
6. What is a programming paradigm?
7. What is computer programming?

In-Text Answers (ITAs)

1. Programming is the process of writing instructions that enable a computer to perform specific tasks.
2. Because it drives innovation in areas like artificial intelligence, web development, data analysis, and automation.
3. A computer programmer writes these instructions using programming languages like Java, Python, or C++.
4. Just as authors read and understand existing works before writing, programmers must read and understand existing code to learn a language's structure and logic.

5. The answer:
 - a) Task automation to save time and reduce errors.
 - b) Creation of software like websites and mobile apps.
 - c) Development of logical thinking and problem-solving skills.
6. A programming paradigm is a style or approach to writing computer programs.
7. A programming paradigm is a style or approach to writing computer programs.

1.1.3 A Quick Dive into the Concept of OOP

Object-Oriented Programming (OOP) is a programming paradigm that organizes codes into objects, making it more modular and reusable. An object is an instance of a class with specific attributes (data) and behaviour (methods). A class defines a blueprint for creating an object. Consider the code below:

```
// Defining a class
class Car {
    String brand = "Toyota";    // Attribute
    int speed = 120;            // Attribute

    // Method to display car details
    void displayInfo() {
        System.out.println("Car brand: " + brand);
        System.out.println("Car speed: " + speed);
    }
}

public class Main {
    public static void main(String[] args) {
        Car carObject = new Car (); // Creating an object
        carObject.displayInfo();    // Calling method
    }
}
```

Figure 1.3 Car Class Showing Attributes and Method

The code snippet below is a simple Java program that demonstrates the basics of classes, objects, attributes and methods. A class named **Car** was defined at the beginning. Two attributes, **brand** and **speed**, were created to store the values of the brand and the speed of the car. A method (which is simply a function inside a class) displays information about the car. The Main class contains the main method, where execution **starts**. While a **new Car()** is used to create a new car object, a reference

variable, carObject, stores the created Car object. The **displayInfo()** method will print the output below:

Car brand: Toyota

Car speed: 120

Also, consider the code snippet below

```
class Dog {
    // Attributes
    String name;
    int age;

    // Method to make the dog bark
    void bark() {
        System.out.println(name + " says Woof! Woof!");
    }
}

public class Main {
    public static void main(String[] args) {
        // Creating an object of Dog class
        Dog myDog = new Dog();

        // Assigning values to attributes
        myDog.name = "Buddy";
        myDog.age = 3;

        // Calling method
        myDog.bark();
    }
}
```

Figure 1.4 Code Snippet Showing the Concept of OOP

The code snippet below defines a Dog class, which acts as a blueprint for creating dog objects. Each dog object will have specific attributes (such as name and age) that store information about the dog. Additionally, the class includes a method (bark()) that defines a behavior all dog objects can perform. This allows us to create multiple dog instances, each with unique characteristics, while still sharing common functionality.

1.1.4 Conclusion

Computer programming is a vital skill in today's digital world, enabling innovation, automation, and effective problem-solving. This unit has laid a strong foundation in programming concepts and paradigms, with a focus on understanding and applying basic Java principles. With this knowledge, learners are now equipped to write simple programs and explore more advanced programming techniques in future modules.

Summary

This unit introduces the foundational concepts of computer programming, emphasising its significance in modern technology and problem-solving. It explores various programming paradigms: imperative, functional, declarative, logic-based, and object-oriented, highlighting their unique approaches and use cases.

Self-Assessment Questions

1. What is computer programming?
 - A. The process of writing books using natural languages
 - B. The process of instructing a computer to perform specific tasks through code
 - C. The process of designing hardware components
 - D. The process of manually operating a computer
2. Who is responsible for writing instructions that a computer can execute?
 - A. Computer scientist
 - B. Computer engineer
 - C. Computer programmer
 - D. Data analyst
3. Which of the following is NOT a programming language?
 - A. Java
 - B. Python
 - C. C++
 - D. RUST
4. Why is programming important?
 - A. It allows repetitive tasks to be automated
 - B. It is used to create software applications
 - C. It enhances problem-solving skills
 - D. All of the above
5. Which programming paradigm follows a step-by-step approach to solving a problem?

- A. Imperative Programming B. Functional Programming
 - C. Declarative Programming D. Logic-Based Programming
6. Which programming paradigm focuses on "what should be computed" rather than "how"?
- A. Imperative Programming
 - B. Functional Programming
 - C. Object-Oriented Programming
 - D. Logic-Based Programming
7. Which of the following is an example of declarative programming?
- A. Python B. JavaC. SQL D. C++
8. What does Object-Oriented Programming (OOP) focus on?
- A. Writing code in a step-by-step manner
 - B. Organizing code using objects and classes
 - C. Writing programs that do not change variables
 - D. Using only mathematical functions in coding
9. Which of the following is NOT an OOP concept?
- A. Encapsulation B. Inheritance
 - C. Polymorphism D. Automation
10. In OOP, an object is an instance of a ____.
- A. Function B. Class C. Method D. Variable

Unit Activities

Activity 1: Define and Discuss

Objective: Understand the meaning and importance of programming.

Instructions:

1. In groups, students define computer programming in their own words.
2. Each group discusses and lists three reasons why programming is important in modern society.

Task:

Present your definitions and reasons to the class in 2 to 3 minutes.

Activity 2: Pair Programming (Write a Simple OOP Class)

Objective: Practice writing basic OOP code in Java.

Instructions:

1. Students work in pairs to write a simple class (e.g., Student, Phone, or Book) with 2 attributes and 1 method.
2. Use main() to create an object and call the method.

Task:

Submit your .java file and run your code to print object data. Present your definitions and reasons. Tutor-Marked Assignment.

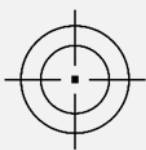
Module Two

Java Setup

Unit 1 — Introduction to Java and Java Development Environment

INTRODUCTION

Java is a versatile, object-oriented language used to build scalable applications across multiple platforms. Since its launch in 1995, it has remained a top choice for web, desktop, enterprise, and mobile development. Its "Write Once, Run Anywhere" (WORA) capability, powered by the Java Virtual Machine (JVM), ensures cross-platform compatibility. This unit introduces Java programming, covering key concepts, syntax, and development tools. Learners will explore the installation and configuration of the Java Development Kit (JDK).



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Understand the fundamental concept of Java programming.
- Install and configure the Java Development Kit (JDK) on different operating systems.
- Set up the environment variable necessary for running Java applications.

2.1.1 Getting Started

To get started in Java programming, you have to download the Java Development Kit (JDK). A JDK is a software development environment used for developing Java applications. In order to run Java, you need to install a JDK

Java Development Kit (JDK) Installation

For Windows users, to check if your PC has Java installed, open the Command Prompt and type the command below:

```
> java --version
```

If Java is installed, you get something similar to what is shown below:

```
java 23 2024-09-17
```

```
Java(TM) SE Runtime Environment (build 23+37-2369)
```

```
Java HotSpot(TM) 64-Bit Server VM (build 23+37-2369, mixed mode,  
sharing)
```

If Java is not installed in your PC, visit the URL <https://www.oracle.com/ng/java/technologies/downloads/> to download the JDK. Scroll down and choose your Operating System (OS). Figure 2.1 shows the download page interface for Windows OS.

JDK 23	JDK 21	GraalVM for JDK 23	GraalVM for JDK 21
Java SE Development Kit 23.0.2 downloads			
JDK 23 binaries are free to use in production and free to redistribute, at no cost, under the Oracle No-Fee Terms and Conditions (NFTC).			
JDK 23 will receive updates under these terms, until March 2025, when it will be superseded by JDK 24.			
Linux	macOS	Windows	
Product/file description		File size	Download
x64 Compressed Archive		228.77 MB	https://download.oracle.com/java/23/latest/jdk-23_windows-x64_bin.zip (sha256)
x64 Installer		205.27 MB	https://download.oracle.com/java/23/latest/jdk-23_windows-x64_bin.exe (sha256)
x64 MSI Installer		204.02 MB	https://download.oracle.com/java/23/latest/jdk-23_windows-x64_bin.msi (sha256)

Figure 2.1 JDK Download for Windows OS

Configuration of Environment Variables

After installing the JDK, you need to configure environment variables to run Java commands like `javac` (for compiling the java code) and `java` (for running the compiled code) from any directory. Follow these steps:

- I. Locate the JDK Installation Path: Open your file explorer and navigate to the directory where the JDK is installed. By default, it is installed at the location:

`C:\Program Files\Java\jdk-<version>\bin`

- II. Set the Environment Variable: Right-click on "This PC" > click on "Properties" > click on "Advanced system settings" > "Environment Variables" under System variables, click "New". Set the details (Figure xxxxxx):

Variable name: `JAVA_HOME`

Variable value: `C:\Program Files\Java\jdk-<version>`

- III. Click on the OK button (Figure 2.2).

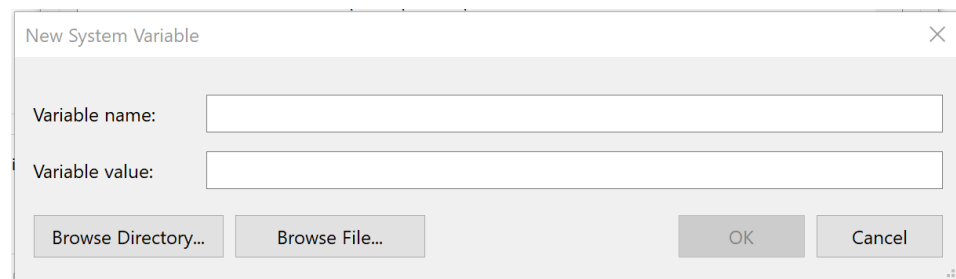


Figure 2.2 New System Variable Dialog Box

- IV. Verify your installation: After the installation, verify your installation on the Command Prompt using the command below:

`java --version`

In-Text Questions (ITQs)

1. What does "WORA" stand for in Java development?
2. When was Java first launched?
3. What are some common areas where Java is used?
4. What is the role of the Java Virtual Machine (JVM)?
5. What is the Java Development Kit (JDK)?
6. Why do you need to install the JDK before starting Java programming?

7. Why do we need to set environment variables after installing the JDK?
8. What is the significance of setting JAVA_HOME?

In-Text Answers (ITAs)

1. "WORA" stands for "Write Once, Run Anywhere", referring to Java's ability to run the same code on multiple platforms using the Java Virtual Machine (JVM).
2. Java was first launched in 1995.
3. Java is widely used in web, desktop, enterprise, and mobile application development.
4. The JVM enables Java programs to run on any platform, ensuring cross-platform compatibility.
5. The JDK is a software development environment used to develop Java applications.
6. Because the JDK provides the necessary tools and libraries to compile and run Java programs.
7. To enable Java commands like javac and java to run from any directory in the Command Prompt.
8. It allows system tools and applications to find the JDK location for compiling and running Java programs.

2.1.2 Conclusion

Proper installation and configuration of the JDK and environment variables are crucial for Java development. By following these setup steps, learners ensure that their systems are ready to compile and run Java applications efficiently, laying a strong foundation for further learning in Java programming.

Summary

This unit introduces Java as a versatile, platform-independent programming language widely used in various application domains. It guides learners through the

essential first steps of setting up the Java Development Kit (JDK), including downloading and installing it on different operating systems. The unit also explains how to configure environment variables, such as JAVA_HOME, to enable easy compilation and execution of Java programs from the command line.

Self-Assessment Questions

1. What does IDE stand for?
 - A) Integrated Development Environment
 - B) Internal Development Engine
 - C) Internet Data Extension D) Integrated Data Editor
2. What does JDK stand for?
 - A) Java Design Kit B) Java Development Kit
 - C) Java Deployment Kit D) Java Debug Kit
3. Which command is used to check if Java is installed on Windows?
 - A) java -check B) java /version
 - C) java --version D) check java
4. What is Java's "Write Once, Run Anywhere" (WORA) capability primarily enabled by?
 - A) Java Compiler (javac) B) Java Development Kit (JDK)
 - C) Java Virtual Machine D) Java Runtime Environment
5. Where is the JDK typically installed by default on a Windows system?
 - A) C:\Java\jdk-<version>
 - B) C:\Program Files\Java\jdk-<version>\bin
 - C) C:\Windows\System32\Java
 - D) C:\Users\<username>\Java\JDK
6. What environment variable should be set to point to the JDK installation directory?
 - A) JAVA_PATH B) JDK_HOME
 - C) JAVA_HOME D) JRE_HOME
7. What is the purpose of the javac command?
 - A) To execute a Java program
 - B) To compile Java source code into bytecode
 - C) To install the JDK D) To check the Java version
8. The official source for downloading the JDK is?
 - A) <https://java.com> B) <https://github.com/java>
 - C) <https://www.oracle.com/java/technologies/downloads/>
 - D) <https://openjdk.org>

9. Which component of Java is responsible for converting bytecode into machine-specific code?
- A) Java Compiler (javac) B) Java Virtual Machine (JVM)
- C) Java Development Kit (JDK)
- D) Java Runtime Environment (JRE)
10. Which command is used to execute a compiled Java program?
- A) java MyProgram B) javac MyProgram
- C) run MyProgram D) execute MyProgram.class

Unit Activities

Activity 1: Installing and Verifying JDK

Objective: Practice downloading, installing, and verifying the Java Development Kit (JDK) on your computer

Instructions:

1. Visit the official Oracle Java download page.
2. Download the appropriate JDK version for your operating system.
3. Install the JDK following the installation prompts.
4. Open your command prompt or terminal
5. Run the command `java --version` to verify the installation.

Tasks:

1. Take a screenshot of the output showing the installed Java version.
2. Write a brief paragraph describing your experience installing the JDK and verifying the version.
3. Submit both your screenshot and description as part of your assignment

Activity 2: Configuring Environment Variables

Objective: Configure the environment variables needed to compile and run Java programs from any directory

Instructions:

1. Locate your JDK installation directory on your system.

2. Set the JAVA_HOME environment variable to point to the JDK installation folder.
3. Update the system's PATH variable to include the JDK bin directory.
4. Open a new command prompt or terminal
5. Test run the commands javac and java from any directory.

Tasks:

1. Document the steps you took to set the environment variables (include screenshots if possible).
2. Run the commands javac -version and java --version and capture their outputs.
3. Explain why setting these environment variables is important in Java development.
4. Submit your documentation, screenshots, and explanations for review

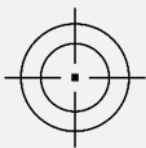
Module Three

Java Fundamentals

Unit 1 — Java Basics-Data Types, Variables and Operations in Java Part 1

INTRODUCTION

To develop efficient and reliable Java applications, mastering core programming concepts is essential. This unit provides a comprehensive introduction to Java fundamentals, including data types, variables, and operators. You will be introduced to the types of control structures for program flow management and learn the complete process of writing, compiling, and executing simple Java programs. The unit also covers practical input/output operations, teaching various methods for user interaction and data display.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Explain Java data types and their memory sizes.
- Declare and use variables with correct naming conventions.
- Identify various operators used in Java including arithmetic, relational, etc.
- Identify the various forms of control structures
- Write and compile basic Java programs.
- Handle user input and display out.
- Understand the structure of a Java program.

3.1.1 Basic Concepts in Programming

Programming and writing logical program begin with mastering basic concepts that serve as building blocks for all code. The core elements include variables, data types, operators, and control structures.

Variables

A variable is a container that stores data. Variables store values such as numbers, characters, text, or logical values. A variable in Java must be declared before it is used. You declare a variable by specifying its data type, followed by the variable name. For example:

```
int number;
```

where `int` is the data type, and `number` is the name of the variable.

The following are the rules for naming variables in Java:

- I. Variable names must start with a letter, underscore (`_`) or dollar sign (`$`). For instance `_myName`, `$fileName`, `person`, etc., are valid variable names.
- II. Variable name cannot start with a digit. `1variable`, `123LondonStreet`, `2name`, etc., are invalid variable names.
- III. Variable names are case-sensitive, meaning `age`, `AGE`, and `Age` are different.
- IV. A variable name can include letters (A to Z, a to z), digits (0 to 9), underscore (`_`), and dollar sign (`$`). `name2`, `variable$25`, `input22`, etc., are all valid variable names.
- V. Variable names cannot contain spaces, or special characters like exclamation (!), at-sign (@), hyphen (-), etc. `my-variable` and `my@name` are invalid.
- VI. Variable names cannot be keyword a. `int`, `class`, `float`, `if`, `public`, `else`, `private`, etc. are invalid variable names.

Data Types

Data types are used to specify the type of data a variable can hold (store). The type of data a variable can store determines the operations that can be performed on that variable. For instance, consider the code snippet in Figure 3.1.

```
public class Main {
    public static void main(String[] args) {
        int age = 25;
        // 'age' is the variable 'int' is the
        // data type and 25 is the value

        // Printing the value of the variable 'age'
        System.out.println("The age is: " + age);
    }
}
```

Figure 3.1 Data Type and Variable

The variable name is age and it is declared to have an integer data type, meaning age can only hold integer.

Data types in Java can be categorized into:

- Primitive Data Types
- Non-Primitive Data Types

Primitive data types are the most basic type of data in Java. They represent single values. Table 3.1 shows the primitive data types and their description.

Data type	Size	Default Value	Description
byte	1 byte	0	Stores 8-bit signed integers. Range is -128 to 127
short	2 bytes	0	Stores 16-bit signed integers. Range is -32,768 to 32,767
int	4 bytes	0	Stores 32-bit signed integers (whole numbers). Range is -2^{31} to $2^{31} - 1$ (that is, -2,147,483,648 to 2,147,483,648)
long	8 bytes	0L	Stores 64-bit signed integers (whole numbers). Range is 2^{63} to $2^{63}-1$
float	4 bytes	0.0f	Stores 32-bit floating-point numbers (decimal numbers). Precision is up to 7 digits.
double	8 bytes	0.0d	Stores 64-bit floating-point numbers (decimal numbers). Precision is up to 15 digits.
char	2 bytes	'\u0000'	Stores a single 16-bit Unicode character. It is used to store a single character or Unicode value. 16-bit unsigned ranging from 0 to 65,536 (Unicode). It is enclosed in single quotes ('A', '1').
boolean	1 bite	false	Stores true or false

Non primitive data types are also known as reference types because they refer to objects in memory. These data types can store objects and are user-defined. Examples include:

- **Classes:** A class is a template used to create objects. It defines the properties (variables) and behaviour (methods) that an object can have.
- **Interfaces:** An interface is an abstract type that defines a set of methods that a class must implement.
- **Arrays:** An array is a data structure that can hold multiple values of the same type.
- **Strings:** A string is an object and is used to represent a sequence of characters.

Operators

Operators are symbols that perform operations on variables, constants or values. The following are types of operators in Java:

- I. **Arithmetic Operators:** They are used to perform basic mathematics operations such as addition (+), subtraction (-), division (/), multiplication (*) and modulus (%).
- II. **Relational (Comparison) Operators:** They are used to compare two values. These operators return a boolean value (true or false) Examples include equal to (==), not equal to (!=), greater than (>), greater than or equal to (>=), less than (<), and less than or equal to (<=).
- III. **Logical Operators:** Logical operators are used to perform logical operations. Examples include Logical AND (&&), Logical OR (||) and Logical NOT (!)
- IV. **Assignment Operators:** Assignment operators are used to assign values to variables. These operators can also perform operations and assign the result back to the variable. Examples include simple assignment (=), addition assignment (+=), subtraction assignment (-=), multiplication assignment (*=) and division assignment (/=).
- V. **Unary Operators:** Unary operators are used with a single operand to perform operation such as increasing, decreasing or negating. Examples include unary plus (+a), unary minus (-a), increment (a++, ++a), decrement (a--, --a), bitwise complement (~a), etc.

- VI. Ternary Operator (?): The ternary operator is a shorthand for if-else statements. It takes three operands and returns one of two values depending on the condition. The syntax is condition? value1: value2
- VII. Instance of Operator: The instance of operator is used to check whether an object is an instance of a specific class or implements an interface. The Syntax follows the format objectName instanceof Class
- VIII. Bitwise Operators: Bitwise operators perform operations on the binary representations of integers.

Control Structure

Control structures in Java are fundamental building blocks that dictate the flow of execution in a program. By default, codes are executed sequentially (linearly) from top to bottom. There are three main types of control structures:

- I. Selection Control (Decision Making): Executes different codes based on conditions. Examples include if, if-else, and switch statements.
- II. Iteration Control (Loops): Repeats a block of code. Examples include for, while and do while loops.
- III. Branching Statements: Alter the normal flow using break, continue and return.

In-Text Questions (ITQs)

- 1. What are the rules for naming variables in Java?
- 2. What is the difference between primitive and non-primitive data types in Java?
- 3. What are the main types of control structures in Java, and what is their purpose?

In-Text Answers (ITAs)

- 1. Variable names in Java must start with a letter, underscore (_), or dollar sign (\$). They cannot start with a digit and are case-sensitive. Variable names may include letters, digits, underscores, and dollar signs, but cannot contain spaces or special characters like !, @, or -. Additionally, variable names cannot be Java keywords such as int, class, or if.
- 2. Primitive data types represent single basic values like integers, floats, characters, and booleans, each with a fixed size and default value. Non-primitive data types,

also called reference types, refer to objects stored in memory, including classes, interfaces, arrays, and strings, which can hold multiple values or complex data.

3. The main types of control structures are:
 - a. Selection Control (if, if-else, switch) for decision-making based on conditions.
 - b. Iteration Control (for, while, do-while loops) to repeat code blocks.
 - c. Branching Statements (break, continue, return) to alter the normal flow of program execution.

3.1.2 Steps to Write and Run a Java Program

In the Java programming language, all source code is written in a plain text file saved with a .java extension. These source codes are compiled into bytecodes (the machine language of the Java Virtual Machine) by the compiler. The bytecodes are saved in a file with a .class extension. A Java program goes through the following steps: writing the code, compiling the code and running it.

Writing the Code

The steps include:

- I. Open a text editor (e.g., Notepad, VS Code, IntelliJ IDEA, Eclipse).
- II. Type the Java program and save it with the same name as the Class and use the .java extension, e.g. Hello.java.

Compiling the Code

Java programs need to be compiled before execution.

- I. Open the Command Prompt and navigate to the directory where the file is saved.
- II. Compile the code with the command
`javac Hello.java`

If there are no syntax errors, a compiled file with a .class is created in the directory.

Running the Program

After successful compilation, run the program using:

```
java Hello
```

3.1.3 A Simple Java Program

This section provides an introduction to the structure of a Java program, including class declaration, main method, statements and execution flow.

Structure of a Simple Java Program

A basic Java program consists of the following components:

- I. Class Declaration: Every Java code must have at least one class.
- II. Main Method (main): The entry point of any Java application.
- III. Statements: Instructions to be executed by the compiler.
- IV. Comments (Optional): None executable statements.

Figure 3.2 shows a simple Java program called FirstProgram:

```

1  public class FirstProgram {
2      public static void main (String[] args) {
3          // Displays Hello Java
4          System.out.println("Hello Java!");
5      }
6  }
```

Figure 3.2 A Simple Java Program

Explanation of the Code

The explanation of the code can be split into the following:

1. **Class Declaration:** Line 1 defines the class with an access modifier of public and a class name of FirstProgram. By convention, class names start with an uppercase to distinguish them from variables/methods.

```
public class FirstProgram {
```

2. **The main Method:** Line 2 defines the main method, which is the entry point of the program execution. The keyword static means that the method belongs to the class, not an instance. void means that the method does not return any value, while Strings[] args indicates that the program accepts command-line arguments.

```
public static void main (String[] args) {
```

3. **Comment:** Line 3 is a comment that explains more about the line of code before it. Comments are not executed by the compiler.

4. **Printing Output:** Line 4 prints the string "Hello Java!" on the console.

In Java, program execution begins with the main method. The first line of a typical Java program defines the class itself.

3.1.4 Java Input

Input handling lets users provide data to an application from the keyboard, files, or networks. Common ways include:

Using Scanner Class (Most Common)

The Scanner class is part of java.util package. It can read various types of inputs like strings, integers, floats, etc.

Example 1

The program (Figure 3.3) uses the Scanner class to get user input. It asks for the name of the user (Line 6), saves it (Line 7), and then prints a welcome message with the name (Line 8).

```

1  import java.util.Scanner;
2  public class ScannerInput {
3      public static void main(String[] args) {
4          Scanner input = new Scanner(System.in);
5
6          System.out.print("Enter your name: ");
7          String name = input.nextLine();
8          System.out.print("Welcome " + name);
9
10         input.close();
11     }
12 }

```

Figure 3.3 Using of Scanner Class

When the code above run, it will display,

Enter your name: John Doe

Welcome John Doe

In Figure xxx, `nextLine()` reads a String input. Other methods include `nextBoolean()`, `nextByte()`, `nextDouble()`, `nextFloat()`, `nextInt()`, `nextLong()`, and `nextShort()`.

Example 2

A code (Figure 3.4) that accepts the name and the age of a user using the Scanner and prints the details out.

```
import java.util.Scanner;
public class NameAge {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter your name: ");
        String name = sc.nextLine();
        System.out.println("Enter your age: ");
        int age = sc.nextInt();

        System.out.println(name + " you're " + age + " years old");

        sc.close();
    }
}
```

Figure 3.4: Using Scanner to Accept Input

Using BufferedReader and InputStreamReader

BufferedReader is used to read text from a character-based input stream, and it can be used with InputStreamReader to read input from the keyboard.

Example 1

This program (Figure 3.5) uses BufferedReader to read an integer input, converts it from String to int and prints the result.

```
import java.io.*;
public class BufferedReaderInput {
    public static void main(String[] args) throws IOException {
        BufferedReader br =
            new BufferedReader(new InputStreamReader(System.in));

        System.out.print("Enter an integer: ");
        int n = Integer.parseInt(br.readLine());

        System.out.println("You entered: " + n);
        br.close();
    }
}
```

Figure 3.5: Using BufferedReader to Accept Input

The program in Figure 3.5 uses BufferedReader to read integer input from the user. It wraps InputStreamReader around System.in, converts the input to an integer with Integer.parseInt(), and prints the result. The throws IOException handles input errors, while br.close() prevents resource leaks. More efficient than Scanner for large inputs, this demonstrates basic console input handling in Java.

Example 2

The Java code in Figure 3.6 uses `BufferedReader` with `InputStreamReader` to read user input from the console. It prompts for a name (as a `String` via `readLine()`) and an age (converted to an `int` using `Integer.parseInt()`), then prints both values. Useful for basic console input, but note that `parseInt()` will throw an error if non-numeric input is entered. Requires throws `IOException` for input handling.

```
import java.io.*;
public class BufferedReaderExample {
    public static void main (String[] args) throws IOException {
        // Create BufferedReader and InputStreamReader
        BufferedReader reader =
            new BufferedReader(new InputStreamReader(System.in));

        // Reading a string
        System.out.print("Enter your name: ");
        String name = reader.readLine();

        // Reading an integer
        System.out.print("Enter your age: ");
        int age = Integer.parseInt(reader.readLine());

        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}
```

Figure 3.6: Accepting Input Using `BufferedReader`

Using Console (Secure for Passwords)

The `Console` class (from `java.io`) is useful for reading passwords securely (input is not displayed).

Example 1

The Java code in Figure 3.7 uses `System.console()` to securely read a username via `readLine()` and a password via `readPassword()` (as `char[]`). It then prints both (unsafely for demo purposes, real apps should avoid converting passwords to `String`). Useful for CLI login prompts.

```
import java.io.Console;

public class ConsoleExample {
    public static void main(String[] args) {
        Console console = System.console();

        if (console == null) {
            System.err.println("Console not available!");
            return;
        }

        String username = console.readLine("Enter username: ");
        char[] passwordChars = console.readPassword("Enter password: ");
        String password = new String(passwordChars);

        System.out.println("Username: " + username);

        // Just for demo (don't do this in real apps!)
        System.out.println("Password: " + password);
    }
}
```

Figure 3.7: Using Console for Accepting Password

3.1.5 Java Output

Java provides several ways to handle output, with the most common method being through the `System.out` stream. Here is a breakdown of different methods and concepts related to Java output:

Print with newline (`System.out.println()`)

The `println()` method prints the specified text or value to the console and then moves the cursor to the next line.

```
public class Println {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
        // Prints the text and moves to a new line
    }
}
```

Figure 3.8 Using print Statement for Output

Print without newline (`System.out.print()`)

Unlike `println()`, `print` (Figure 3.8) does not move the cursor to a new line after printing.

```
public class Print {
    public static void main(String[] args) {
        System.out.print("Hello, ");
        System.out.print("Java!");
    }
}
```

Figure 3.8: Code Snippet for Print

The above code will output "Hello Java!" on the same line.

Formatted Output (System.out.printf())

printf() (Figure 3.9) allows printing of variables with specific formatting (like setting the decimal places, padding numbers, etc.).

Example 1

```
public class PrintFormatOne {
    public static void main(String[] args) {
        double pi = 3.14159;
        System.out.printf("The value of Pi is %.2f", pi);
        // Prints pi with two decimal places
    }
}
```

Figure 3.9: Using printf for Output

printf works with format specifiers. In the code in Figure xxx, the format specifier is "%.2f". Table 3.2 shows more format specifiers.

Format	Description	Example
%s	String	"John"
%d	Integer	100
%f	Floating-point number	3.14159
%.2f	2 decimal places	3.14
%c	Character	'A'
%x or %X	Hexadecimal (lowercase or uppercase)	ff or FF
%e	Scientific notation (lowercase)	1.234568e+06
%E	Scientific notation (uppercase)	1.234568E+06
%05d	Padding with zeros	00005
%+d or %-d	Forces a sign (+ or -)	+10 or -10
%-5d	Left justified e.g. " %-5d "	10
%5d	Minimum number of character	5

Example 2:

The Java program in Figure 3.10 uses printf() to format and print variables: name (String), age (int), and height (double). It displays name and age, then prints height with two decimal places.

```

1 public class PrintFormatThree {
2     public static void main(String[] args) {
3         String name = "Alice";
4         int age = 25;
5         double height = 1.75;
6
7         System.out.printf("Name: %s, Age: %d", name, age);
8         System.out.println();
9         System.out.printf("Height: %.2f meters", height);
10    }
11 }

```

Figure 3.10: Using the printf Format

The above code will output:

Name: Alice, Age: 25

Height: 1.75 meters

Example 3

The code in Figure 3.11 uses printf() and format() to print a table with left-aligned text (%-10s) and right-aligned numbers (%5d). %s is for strings, %d for integers. The 5 in %5d means Use a minimum width of 5 characters.

```

1 public class PrintFormatTwo {
2     public static void main(String[] args) {
3         double num = 1234567.89;
4
5         System.out.printf("%E", num);
6         System.out.println();
7         System.out.format("%-10s %-10s %5d%n", "Name", "Role", 2025);
8         System.out.format("%-10s %-10s %5d%n", "Lara", "Dev", 36);
9         System.out.format("%-10s %-10s %5d%n", "John", "Tester", 28);
10    }
11 }

```

Figure 3.11: Using printf for Adding Formatted Spaces

The above code will output:

1.234568E+06

Name	Role	2025
Lara	Dev	36
John	Tester	28

Lara	Dev	36
John	Tester	28

In-Text Questions (ITQs)

1. Explain the steps involved in writing, compiling, and running a Java program.
2. What is the purpose of the main method in a Java program?
3. What are the differences between `System.out.print()`, `System.out.println()`, and `System.out.printf()`?

In-Text Answers (ITAs)

1. First, write the Java code in a plain text editor and save it with a .java extension using the class name as the filename. Next, compile the code using the `javac` compiler, which produces a .class file with bytecode. Finally, run the compiled bytecode with the `java` command followed by the class name to execute the program.
2. The main method serves as the entry point for program execution. It is where the Java Virtual Machine (JVM) begins running the program. Its signature is `public static void main(String[] args)`, indicating it is public, static, returns no value, and accepts an array of String arguments.
3. The differences are:
 - a. `print()` outputs text without moving to a new line.
 - b. `println()` outputs text and moves the cursor to the next line.
 - c. `printf()` allows formatted output using format specifiers (e.g., `%d`, `%s`, `%.2f`) for controlling the display of variables with precision, alignment, and padding

3.1.6 Conclusion

Mastering these Java fundamentals equips you with the basic tools needed to develop reliable and efficient Java applications. Understanding variables, data types, operators, and control structures forms the backbone of programming logic. With the ability to write, compile, and run Java code, and effectively manage user input

and output, you are well-prepared to tackle more advanced programming challenges in Java.

Summary

This unit introduced the foundational concepts of Java programming, covering essential topics such as data types, variables, operators, and control structures. You learned the rules for naming variables, the difference between primitive and non-primitive data types, and the various operators used to perform computations and decisions. The unit also explained how to write, compile, and run simple Java programs, highlighting the role of the main method. Additionally, you explored methods for handling user input using Scanner, BufferedReader, and Console classes, as well as different ways to display output with print, println, and printf

Self-Assessment Questions

1. How many bytes does a byte data type use?
A) 2 B) 1 C) 4 D) 8
2. What is the size of a short data type in memory?
A) 8 bits B) 16 bits C) 32 bits D) 64 bits
3. Which of the following data types is stored using 4 bytes?
A) long B) byte C) int D) double
4. What is the correct size of a long data type in bytes?
A) 2 B) 4 C) 8 D) 16
5. The float data type takes how many bytes in memory?
A) 2 B) 4 C) 8 D) 16
6. What is the precision level of a double compared to a float?
Less B) The same C) More D) None of the above
7. A double type variable takes how much space in memory?
A) 2 bytes B) 4 bytes C) 8 bytes D) 16 bytes
8. Which numeric data type would be the best choice for storing large integer values?
A) int B) short C) long D) float
9. Which data type is best suited for high-precision decimal values?
A) int B) float C) double D) short

10. Which real-value type is typically used when memory optimization is a priority?
 A) float B) double C) int D) long
11. 11. A file containing a class called Main should be saved as:
 A) Main.js B) Main.cls C) main.class D) Main.java
12. 12. What method is used to print without a newline in Java?
 A) System.out.line() B) System.print.out()
 C) System.out.print() D) System.out.println()
13. What is the correct sequence for running a Java program?
 A) Write → Run → Compile B) Compile → Write → Run
 C) Write → Compile → Run D) Run → Compile → Write
14. What is the output file extension after compiling a Java file?
 A) .exe B) .class C) .javac D) .obj
15. What is a variable in Java?
 A function B) A class
 C) A container to store data D) A library
16. Which of these is a valid variable name in Java?
 A) 1value B) my-variable C) \$amount D) class
17. What type of data type is int in Java?
 A) Object B) Float C) Primitive D) Non-primitive
18. Which of the following is NOT a primitive data type in Java?
 A) float B) char C) String D) None of the above
19. What is the purpose of the javac command?
 A) Run Java program B) Create a class
 C) Compile Java code D) Delete files
20. Which command is used to run a compiled Java class?
 A) run Hello.java B) javac Hello.class
 C) java Hello D) execute Hello
21. How do you write a single-line comment in Java?
 A) <!-- Comment --> B) ## Comment
 C) // Comment D) ** Comment
22. What is the entry point of a Java program?
 A) constructor B) Main class
 C) main method D) None of the above
23. Which of the following is NOT a valid Java keyword?
 A) class B) public C) input D) void

Unit Activities

Activity 1: Operators and Control Structures in Java

Objective: To demonstrate the use of various operators and control structures in Java programming.

Instructions:

Learn about arithmetic, relational, logical, assignment, unary, and ternary operators, as well as Java control structures like if-else, switch, loops, and branching statements

Task:

1. Write a Java program that demonstrates the use of different operators with comments explaining their function.
2. Extend the program to include examples of control structures: if-else, switch, for loop, while loop, and do-while loop.
3. Use branching statements (break, continue) inside loops and explain their effects in comments

Activity 2: Writing, Compiling, and Running Java Programs

Objective: To practice writing, compiling, and running Java programs using command line tool.

Instructions:

Follow the Java program development lifecycle by writing a basic Java program, compiling it, and running it from the terminal or command prompt

Tasks:

1. Write a simple Java program named HelloJava that prints "Hello Java!" to the console.
2. Compile the program using the javac command and run it using the java command.
3. Capture screenshots or document each step: code writing, compilation, and program output

Activity 3: Java Input and Output Techniques

Objective: To practice receiving user input and displaying output using different Java methods.

Instructions:

Explore how to handle user input via the Scanner class, BufferedReader, and Console, and output data using System.out.print, println, and printf.

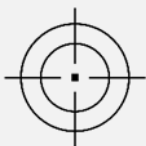
Tasks:

1. Write a Java program that uses the Scanner class to read a user's name and age, then print a greeting message.
2. Write another program that uses BufferedReader to read input, convert it to the appropriate data type, and display it.
3. Create a program that uses formatted output with printf() to display a table of names, ages, and heights aligned neatly with appropriate format specifiers.

Unit 2 — Java Basics-Data Types, Variables and Operations in Java Part II

INTRODUCTION

This unit delves into detailed aspects of data types (including primitive types like int, double, boolean and reference types like String and arrays), variables (covering declaration and initialisation), constants (using the final keyword), and operators (arithmetic, relational, logical, assignment, and their precedence rules). This unit focuses on the core elements of Java programming, starting with the usage of data types. You will learn how to declare and initialise variables, understand variable scope, and apply naming conventions. Additionally, the unit covers different types of operators (arithmetic, relational, logical, and assignment) and their roles in performing calculations, comparisons, and logical decisions



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Declare, initialise, and use variables while adhering to Java naming conventions.
- Apply arithmetic, relational, logical and assignment operators in Java programs.
- Understand how to use the final keyword when declaring constants.
- Understand operator precedence and its impact on expressions.
- Write and execute Java programs that demonstrate the proper use of data types, variables and operators.

3.2.1 Usage of Data Types in Java

Each data type serves a specific purpose. Consider the code sample in Figure 3.12:

```
public class PrimitiveTypes {
    public static void main (String[] args) {
        int age = 30;
        double salary = 85999.90;
        char grade = 'B';
        boolean married = false;

        System.out.println("Age: " + age);
        System.out.println("Salary: $" + salary);
        System.out.println("Grade: " + grade);
        System.out.println("Married: " + married);
    }
}
```

Figure 3.12: Primitive Data Types

3.2.2 Usage of Variables and Constants

Variables

A variable is a named memory location used to store a value. It must be declared with a data type before use. The syntax is:

```
dataType variableName = value;
```

Examples:

```
int age = 25; // Integer variable

String name = "John"; // String variable

boolean isActive = true; // Boolean variable

double price = 19.99; // Double variable
```

Constants

A constant is a data container whose value cannot change once it has been assigned. Constants improve code readability and prevent accidental modifications. In Java, constants are declared using the final keyword.

Syntax is:

```
final dataType CONSTANT_NAME = value;
```

Examples

```
final double PI = 3.14159;

final int MAX_USERS = 100;

final String COMPANY_NAME = "TechCorp"
```

Example of variables and constants in a program

The following are examples (in screenshots) of codes that show how variables and constants are being used.

Example 1:

The TaxInclusivePricing class (Figure xxx) calculates the total price of items including tax. It uses a constant tax rate of 15% (TAX_RATE). In the main method, the total is computed by multiplying the the quantity and the base price and the tax rate.

```
public class TaxInclusivePricing {
    // Constant (static final)
    public static final double TAX_RATE = 0.15;
    public static void main(String[] args) {
        // Variables
        int quantity = 10;
        double price = 29.99;
        double total = price * quantity * (1 + TAX_RATE);

        System.out.println("Total cost: $" + total);
    }
}
```

Figure 3.13: Using Different Data Types

Example 2

The TemperatureConverter class (Figure 3.14) converts Celsius to Fahrenheit using constants: FAH_FACTOR and FAH_OFFSET. It converts 25.0C to 77.0F, then updates the Celsius value to 30.0C and prints the new Fahrenheit value, 86.0F.

```

public class TemperatureConverter {
    // Constants for conversion
    static final double FAH_FACTOR = 9.0 / 5;
    static final double FAH_OFFSET = 32;
    public static void main(String[] args) {
        // Initial value for celsius
        double celsius = 25.0;
        double fahrenheit = (celsius * FAH_FACTOR) + FAH_OFFSET;
        System.out.println(celsius + "C = " + fahrenheit + "F");

        // Change the Value for Celsius
        celsius = 30;
        fahrenheit = (celsius * FAH_FACTOR) + FAH_OFFSET;
        System.out.println(celsius + "C = " + fahrenheit + "F");
    }
}

```

Figure 3.14: Using Different Data Types and Constants

Example 3:

The BankAccount class (Figure 3.15) calculates the interest earned on a bank balance. It defines a constant INTEREST_RATE of 0.05 (5%). In the main method, it initializes a balance of \$1000.0 and calculates the interest earned using the formula $\text{interestEarned} = \text{balance} * \text{INTEREST_RATE}$. The result is printed. After depositing an additional \$500 into the balance, the program recalculates the interest on the new balance and prints the updated interest.

```

public class BankAccount {
    // Constant
    public static final double INTEREST_RATE = 0.05;
    public static void main(String[] args) {
        // Variables
        double balance = 1000.0;
        double interestEarned = balance * INTEREST_RATE;

        System.out.println("Interest Earned: $" + interestEarned);

        // Deposit more money
        balance = balance + 500;
        interestEarned = balance * INTEREST_RATE;
        System.out.println("New Interest: $" + interestEarned);
    }
}

```

Figure 3.15: More Examples on Using Constants and Variable

3.2.3 Usage of Operators

As discussed in Module 3, Unit 2, operators in Java can be mathematical, relational, logical, assignment, etc.

Arithmetic Operators

Arithmetic operators are shown in Table 3.3

Table 3.3: Arithmetic Operators

Operator	Description	Example (a = 7, b = 2, x = 18.0, y = 9.0)	Result	Notes
+	Addition	a + b = 7 + 2	9	Works with int, long, float and double
-	Subtraction	a - b = 7 - 2	5	Works with all numeric types
*	Multiplication	a * b = 7 * 2	14	Works with all numeric types
/	Integer Division	a/b = 7/2	3	Truncates decimal part
	Floating-Point Division	x/y = 18.0/9.0	2.0	Keeps decimal precision
%	Modulus (Remainder)	a % b = 7%2	1	Returns remainder
		x % y = 18.0 % 9.0	0.0	Works with floating point
		-8 % 3	-2	Sign follows dividend

The following are common pitfalls with arithmetic operators

- I. Division by Zero
- II. Modulus with negative numbers
- III. String concatenation

```

1  public class ArithmeticOperatorsPitfalls {
2      public static void main(String[] args) {
3          System.out.println(10/0);
4          System.out.println(10.0/0);
5          System.out.println(-7%3);
6          System.out.println(7%-3);
7      }
8  }

```

Figure 3.16: Using Arithmetic Operators

In Figure 3.16, line 3 will throw an `ArithmeticException` error. If commented out, the remaining lines will output:

Line 4: Infinity

Line 5: -1

Line 6: 1

Relational (Comparative) Operators

Relational operators (as shown in Table 3.4) return boolean values (true or false) as their outputs.

Table 3.4: Relational Operators

Operator	Description	Example (a = 5, b = 3)	Return Value	Notes
<code>==</code>	Equal to	<code>a == b</code>	false	Compares values (for primitives) or references (for objects)
<code>!=</code>	Not equal to	<code>a != b</code>	true	Opposite of <code>==</code>
<code>></code>	Greater than	<code>a > b</code>	true	Works with numbers
<code><</code>	Lesser than	<code>a < b</code>	false	Works with numbers
<code>>=</code>	Greater than or equal to	<code>a >= b</code>	true	true, if either <code>></code> or <code>==</code>
<code><=</code>	Lesser than or equal to	<code>a <= b</code>	false	true, if either <code><</code> or <code>==</code>

Example 1: Primitive comparison

```

public class PrimitiveComparison {
    public static void main(String[] args) {
        int y = 5;
        int x = 5;
        double a = 0.1 + 0.2;
        double b = 0.3;
        System.out.println(y == x); // true
        System.out.println(a == b); // false
    }
}

```

Figure 3.17: Primitive Comparison

In Figure 3.17, the comparison between `a` and `b` is false because $0.1 + 0.2 = 0.30000000000000004$.

Example 2: Comparing different data types

```

1  public class CompareDifferentDataTypes {
2      public static void main(String[] args) {
3          int num = 10;
4          double decimal = 10.0;
5          char letter = 'A';
6          System.out.println(num == decimal);
7          System.out.println(letter > num);
8      }
9  }

```

Figure 3.18: Comparing Different Data Types

Line 6 outputs true because == compares values, not types, 10.0 (double) equals 10 (int). Line 7 is also true since 'A' has an ASCII value of 65, which is greater than 10.

Logical Operators

These operators determine the truth value of a condition or combined conditions, returning either true or false (Table 3.5).

Table 3.5: Logical Operatos and Examples

Operator	Name	Description	Example (a = true, b = false)
&&	Logical AND	Returns true only if both operands are true	a && b = false
	Logical OR	Returns true if at least one operand is true	a b = true
!	Logical NOT	Reverses the boolean value	!a = false
XOR	Logical XOR (Exclusive OR)	Returns true if only one operand is true	a ^ b = true

Example 1: Boolean operators

```

1  public class AdulthoodDriver {
2      public static void main(String[] args) {
3          boolean isAdult = true;
4          boolean hasLicense = false;
5
6          if (isAdult && hasLicense) {
7              System.out.println("Can drive");
8          } else {
9              System.out.println("Cannot drive");
10         }
11     }
12 }

```

Figure 3.19: Using Boolean Operators

In the code in Figure 3.19, line 7 is not executed. This is because the condition in line 6 is false. However, line 9 executes.

Example 2: Boolean with relational operators

```

1  public class BooleanWithRelational {
2      public static void main(String[] args) {
3          int x = 7;
4          int y = 12;
5
6          System.out.println(x < 10 && y > 10);
7          System.out.println(x == y || y != 0);
8          System.out.println(x == y && y != 0);
9      }
10 }

```

Figure 3.20: Using Boolean and Relational Operators

Line 6 is true as x is less than 10 and y is greater than 12. Line 7 is true because y is not equal 0. Line 8 is false because while y is not equal to 0 is true, x is equal to y is false.

In-Text Questions (ITQs)

1. What is the syntax to declare and initialize a variable in Java?
2. How do you declare a constant in Java?
3. How do logical operators affect conditional statements?
4. What is a common mistake when comparing variables using = instead of ==?
5. What error occurs when dividing by zero in Java?
6. How can parentheses change the evaluation of expressions?
7. Why is operator precedence important in Java expressions?

In-Text Answers (ITAs)

1. The syntax is
 dataType variableName = value;

For example, `int age = 25;` declares an integer variable named `age` with the value 25.

2. A constant is declared using the `final` keyword, like
`final double PI = 3.14159;`
3. Logical operators combine multiple conditions in `if`-statements, controlling program flow by evaluating the overall truth value.
4. Using `=` assigns a value instead of comparing, which causes logical errors or compilation failure.
5. Dividing by zero throws an `ArithmeticException` error at runtime.
6. Operator precedence determines the order in which parts of an expression are evaluated, affecting the final result.
7. Parentheses override default precedence, forcing Java to evaluate the expressions inside them first.

3.2.4 Conclusion

This unit provided the foundational knowledge needed to declare and initialize variables, enforce immutability with constants, and apply a variety of operators correctly. Understanding how these elements interact enables you to write clear, efficient, and bug-free code. The ability to handle arithmetic, relational, and logical operations confidently supports making decisions and controlling program behavior, skills that are indispensable as you progress in Java development. As you continue your learning journey, these core concepts will serve as the groundwork for building more sophisticated applications and solving complex programming problems.

Summary

The unit introduced the concept of constants using the `final` keyword, emphasizing their importance in creating immutable values that enhance code safety and readability. Practical examples, including tax calculations, temperature conversions, and bank interest computations, illustrated the effective use of variables and constants in real-world scenarios. Operators were examined extensively, covering arithmetic, relational, logical, and assignment categories. You learned how arithmetic operators perform calculations and the nuances of integer versus floating-point division, along with common pitfalls such as division by zero and modulus with negative numbers. Relational operators were shown to enable comparisons that return boolean values,

critical for decision-making in programs. Logical operators like AND, OR, NOT, and XOR were also discussed, demonstrating how to combine conditions to control program flow.

Self-Assessment Questions

1. Which operator has the highest precedence?
 A) * B) + C) () D) !
2. Which operator has the lowest precedence?
 A) && B) || C) == D) %
3. What will be the output of $10 / 2 + 3 * 2$?
 A) 9 B) 11 C) 12 D) 10
4. What is the result of $10 \% 3$?
 A) 1 B) 2 C) 3 D) 4
5. What is the result of $!(5 > 2 \ \&\& \ 3 < 4)$?
 A) true B) false
6. Which operator is used for logical "AND"?
 A) || B) && C) == D) !
7. Which operator is used for logical "OR"?
 A) && B) || C) != D) ==
8. What is the result of $\text{true} \ || \ \text{false} \ \&\& \ \text{false}$?
 A) true B) false
9. What does `==` check for?
 A) Value and type equality B) Only value equality
 C) Only type equality D) None of the above
10. What is the result of $!(3 == 3)$?
 A) true B) false
11. What does `!=` mean?
 A) Greater than B) Less than C) Not equal to D) Equals
12. What will $10 > 5 \ \&\& \ 3 < 2$ return?
 A) true B) false
13. What will $5 == 5 \ || \ 2 < 1$ return?
 A) true B) false
14. Which operator is used for NOT operation?
 A) && B) || C) ! D) ==
15. What is the output of $5 \% 2 == 1 \ \&\& \ 4 > 2$?
 A) true B) false
16. What is the precedence of `>=` in comparison to `+`?

- A) Higher B) Lower C) Equal
17. What is the precedence of == compared to &&?
 A) Higher B) Lower C) Equal
18. What will 10 % 4 + 6 / 2 evaluate to?
 A) 5 B) 4 C) 7 D) 6
19. Which operator has the highest precedence?
 A) % B) && C) + D) ==
20. What will 2 + 3 * 4 % 5 evaluate to?
 A) 4 B) 5 C) 6 D) 2
21. What does (true && false) || true evaluate to?
 A) true B) false
22. What is 6 / 3 + 8 % 3?
 A) 3 B) 4 C) 2 D) 5
23. What will !(false || true) return?
 A) true B) false
24. What will !(true && false) return?
 A) true B) false
25. What will 8 + 2 * 3 - 4 / 2 evaluate to?
 A) 10 B) 12 C) 11 D) 9
26. What will false && true || true return?
 A) true B) false
27. What will 10 > 2 && 5 <= 5 return?
 A) true B) false
28. What is the output of the code snippet?
 System.out.println(-7 % 3)
 1 B) ArithmeticException
 C) -1 D) None of the above
29. What is the output of the code snippet?
 System.out.println(5.0 / 0)
 1 B) ArithmeticException
 C) -1 D) Infinity
30. What is the output of the code snippet?
 System.out.println(5 / 0)
 1 B) ArithmeticException
 C) -1 D) Infinity
31. What Java operator is used to assign values to variables?
 A) == B) := C) = D) =>
 A) 2 B) 4 C) 8 D) 16

Unit Activities

Activity 1: Declaring Variables, Constants, and Basic Operations

Objective: Learn how to declare variables and constants, initialize them, and perform basic arithmetic operations

Instructions:

Write Java programs to declare variables and constants, then use arithmetic operators to perform calculations.

Task:

1. Declare variables of types int, double, boolean, and String, and initialize them.
2. Declare constants using the final keyword (e.g., PI, MAX_USERS).
3. Write expressions using arithmetic operators (+, -, *, /, %) on declared variables.
4. Calculate and print the circumference of a circle using a constant PI and a radius variable.

Activity 2: Using Relational and Logical Operators

Objective: Understand how to use relational and logical operators to evaluate expressions and control program flow

Instructions:

Create Java programs that compare variables using relational operators and combine conditions using logical operators

Task:

1. Use relational operators (==, !=, >, <, >=, <=) to compare integers, doubles, and characters, and print the results.
2. Write if-statements that combine multiple conditions using logical operators (&&, ||, !, ^).
3. Experiment with short-circuit evaluation in logical expressions and observe behavior.
4. Include examples that show the difference between comparing strings with == and .equals().

Activity 3: Troubleshooting and Best Practices with Variables and Operators

Objective: Identify common errors with variables, constants, and operators, and learn best practices to write clean and error-free Java code

Instructions:

Analyze buggy code snippets, find errors related to operator misuse or incorrect variable declarations, and fix them

Tasks:

1. Debug code containing division by zero, misuse of assignment = instead of equality ==, or improper use of constants.
2. Explain the importance of using constants for fixed values and how they improve code reliability.
3. Discuss naming conventions for variables and constants for better code readability.

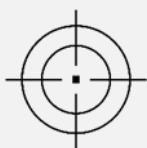
Module Four

Java Control Structures

Unit 1 — Conditional Statements and Decision Making

INTRODUCTION

Control structures are fundamental constructs that determine the flow of execution within a program. They allow the program to make decisions, execute code repeatedly, and control how the code moves from one part to another. Control structures are used to make decisions (decision-making structures), repeat actions (looping structures), and handle unexpected situations (error handling and debugging). In this unit, the focus is on decision-making control structures. Decision-making structures help the program respond intelligently to inputs or situations. Instead of running code sequentially, the program can test conditions and decide which code block to run. Java provides several conditional constructs like if, if-else, if-else-if, switch, and the ternary operator (?:), all of which allow the developer to manage program flow efficiently and logically.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Understand the concept of control structures.
- Identify and explain types of control structure.
- Apply conditional statements (if, if-else, if-else-if, switch, and the ternary operator) to control program flow based on specific conditions.
- Understand the role of Boolean expressions in decision-making.
- Write Java programs that incorporate conditional logic to solve real-world problems

4.1.1 What are Control Structures?

Control structures in programming are special instructions that manage the flow of execution in a program. They allow the program to make decisions, repeat actions, or choose between different paths based on certain conditions. Control structures are generally categorised into three groups: conditional, loop, and branching.

Conditional (Decision-Making) Structures

These execute specific blocks of code only when certain conditions are met. Common examples include:

- i. if statement
- ii. if-else statement
- iii. else-if ladder
- iv. switch statement

Looping (Repetition) Structures

These are used to repeatedly execute a block of code as long as a given condition is true. Examples include:

- i. for loop
- ii. while loop
- iii. do while loop

Jump (Control Transfer) Structures

These alter the normal flow of control, especially within loops or conditional blocks. Examples include:

- i. break (exits a loop or switch block)
- ii. continue (skips the current loop iteration and moves to the next)
- iii. return (exits from a method and optionally returns a value)

4.1.2 Decision-Making Structures

The if Statement

This evaluates a Boolean expression and executes a block of code only if the condition is true. The syntax is:

```
if (condition) {
```

```

        // Code to be executed if the condition is true
    }

```

Example 1:

Figure xxx, demonstrates a conditional statement that only executes if the condition is true. The code is skipped if the condition is false.

```

public class IfStatement {
    public static void main(String[] args) {
        int age = 19;
        if (age >= 18) {
            System.out.println("You are eligible to vote!");
        }
    }
}

```

Figure 4.1: Usage of If Statement

The code in Figure 4.1 will output: "You are eligible to vote!"

The if-else Statement

Executes one block if the condition is true, another block if false. The syntax is:

```

If (condition) {

    //Execute code if the condition is true

} else {

    // Executes code if the condition is false

}

```

Example 1

The code checks whether a given integer (number) is even or odd using a simple if-else conditional statement.

```
public class IfElseStatement {
    public static void main(String[] args) {
        int num = 10;
        if (num % 2 == 0) {
            System.out.println(num + " is even");
        } else {
            System.out.println(num + " is odd");
        }
    }
}
```

Figure 4.2: Usage of If-Else Statement

The output of the code in Figure 4.2 is: 10 is even

The else-if Ladder

This is used for multiple conditions. This is useful when you have more than two possible paths to take based on certain conditions. The program checks each condition in sequence until one is true. The syntax is:

```
if (condition1) {
    // Executes if condition1 is true
} else if (condition2) {
    // Executes if condition1 is false and condition2 is true
} else if (condition3) {
    // Executes if all previous conditions are false and condition3 is true
} else {
    // Executes if none of the above conditions are true
}
```

Example 1

Figure 4.3 demonstrates how if-else if structure helps in decision making when multiple outcomes (grade categories) are possible.

```
public class IfElseStatementTwo {
    public static void main(String[] args) {
        int score = 75;

        if (score >= 90) {
            System.out.println("You grade is A");
        } else if (score >= 80) {
            System.out.println("You grade is B");
        } else if (score >= 70) {
            System.out.println("You grade is C");
        } else if (score >= 60) {
            System.out.println("You grade is D");
        } else {
            System.out.println("You grade is F");
        }
    }
}
```

Figure 4.3: Usage of If-Else-If Statement

The code in Figure 4.3 will output

Your grade is C

The Switch Statement

The switch statement streamlines multiple if-else checks by evaluating an expression once and executing the block of code that matches the corresponding case value. The break statement is used to terminate a specific case, immediately jumping to the end of the switch block. Without a break, the program continues executing the subsequent case(s), a behavior known as "fall-through," until it encounters a break or reaches the end of the switch. The syntax is:

```
switch (expression) {

    case value 1:

        // code block

        break;

    case value 2:

        // code block

        break;

    default:

        // code block
}
```

}

Example 1

The code in Figure 4.4 uses a switch statement to display a message based on the value of the category variable. It checks the value of category and matches it to one of the defined case blocks: if the value is 1, it prints "Pre-teen"; if it is 2, it prints "Teenage"; and if it is 3, it prints "Adult". If the value does not match any of these cases, the default block is executed, which prints "Senior Adult".

```
public class Switch1 {
    public static void main(String[] args) {
        int category = 1;
        switch (category) {
            case 1:
                System.out.println("Pre-teen");
                break;
            case 2:
                System.out.println("Teenager");
                break;
            case 3:
                System.out.println("Adult");
                break;
            default:
                System.out.println("Senior Adult!");
                break;
        }
    }
}
```

Figure 4.4: Usage of the Switch Statement

Example 2

The code in Figure 4.5, displays welcome message based on the value for the level.

```

public class SwitchStringExample {
    public static void main(String[] args) {
        String level = "intermediate";
        switch (level) {
            case "beginner":
                System.out.println("Welcome, beginner!");
                break;
            case "intermediate":
                System.out.println("You're doing well!");
                break;
            case "advanced":
                System.out.println("You're almost an expert!");
                break;
            default:
                System.out.println("Unknown level.");
        }
    }
}

```

Figure 4.5: Usage of Switch, Example 2

The output of the code in Figure 4.5 is You're doing well!

4.1.3 Ternary Operator

The ternary operator is a compact way of writing an if-else statement. It is often used for simple conditional expressions where you want to assign a value or return a result based on a condition. The syntax is:

dataType variable = (condition) ? value_if_true : value_if_false;

Example 1: Basic Assignment

The code (Figure 4.6) uses the ternary operator to find and print the maximum of two integers.

```

public class MaxNumbers {
    public static void main(String[] args) {
        int a = 10, b = 20;
        int max = (a > b) ? a : b;
        System.out.println("Maximum is: " + max);
    }
}

```

Figure 4.6: Usage of Ternary Operator

Example 2: Ternary Operator Inside Print Statement

Figure 4.7 illustrates the usage of the ternary operator inside the print statement.

```
public class TemperatureCheck {
    public static void main(String[] args) {
        int temp = 25;
        System.out.println(temp > 30 ? "It's hot" : "It's cool");
    }
}
```

Figure 4.7: Usage of Ternary Operator Inside a Print Statement

The code in Figure 4.7 will output: It's cool

Example 3: Nested Ternary Operator Inside Print Statement

This code (Figure 4.8) assigns a letter grade based on the score using nested ternary operators and prints the result.

```
public class GradeCalculator {
    public static void main(String[] args) {
        int score = 55;
        String grade = (score >= 90) ? "A" :
            (score >= 80) ? "B" :
            (score >= 70) ? "C" : "F";

        System.out.println("Grade: " + grade);
    }
}
```

Figure 4.8: Usage of Nested Ternary Operator

When you run the code, the output is: Grade: F

Example 4: Inline Conditional Return

In Figure 4.9, the code defines a method that uses a ternary operator to evaluate whether a number is even or odd and returns the corresponding string ("Even" or "Odd") to the caller.

```
public class EvenOddChecker {
    public static String evenOdd(int number) {
        return (number % 2 == 0) ? "Even" : "Odd";
    }

    public static void main(String[] args) {
        int num = 7;
        System.out.println(num + " is " + evenOdd(num));
    }
}
```

Figure 4.9: Inline Conditional Return Statement

The output of the code is: 7 is Odd

4.1.4 Nested if Statement

Nested if statements occur when an if or else block contains another if or else statement inside it. This allows you to test multiple conditions in a hierarchical or sequential manner. It is used to check complex conditions that depend on multiple criteria and to perform different actions based on combinations of conditions. The syntax is:

```

if (condition1) {

    if (condition2) {

        // Executes if both conditions are true

    } else {

        // Executes if condition1 is true and condition 2 is false

    }

    } else {

        // Executes if condition1 is false

    }

```

Example 1: Basic Nested If

The code in Figure 4.10 checks if a number is positive and then determines if it's even or odd, printing the result accordingly.

```

public class NumberCheck {
    public static void main(String[] args) {
        int num = 10;

        if (num > 0) {
            if (num % 2 == 0) {
                System.out.println("Positive even number");
            } else {
                System.out.println("Positive odd number");
            }
        } else {
            System.out.println("Non-positive number");
        }
    }
}

```

Figure 4.10 Basic Nested if Statement

The output of the code is: Positive even number

Example 2: Checking Age and Eligibility

The code in Figure 4.11 checks if a person is 18 or older and has an ID; if both conditions are true, it prints "Eligible to vote," otherwise it prompts for an ID or states they're not eligible.

```
public class VotingEligibility {
    public static void main(String[] args) {
        int age = 20;
        boolean hasID = true;

        if (age >= 18) {
            if (hasID) {
                System.out.println("Eligible to vote");
            } else {
                System.out.println("ID required to vote");
            }
        } else {
            System.out.println("Not eligible to vote");
        }
    }
}
```

Figure 4.11: Basic Nested if Statement, Example 2

The output of the code is: Eligible to vote

Example 3: User Login Validation

In Figure 4.12 if username and password are correct, it prints "Login successful". If password is wrong, it prints "Incorrect password". If username is incorrect, it prints "User not found".

```
public class LoginCheck {
    public static void main(String[] args) {
        String username = "admin";
        String password = "12345";

        if (username.equals("admin")) {
            if (password.equals("1234")) {
                System.out.println("Login successful");
            } else {
                System.out.println("Incorrect password");
            }
        } else {
            System.out.println("User not found");
        }
    }
}
```

Figure 4.12: Login Validation with Nested If Statement

The code will output: Incorrect password

Example 4: Leap Year Checker

In Figure 4.13, the code checks if a given year is a leap year based on the rules: divisible by 4, not divisible by 100 unless also divisible by 400. It prints the result accordingly.

```
public class LeapYearCheck {
    public static void main(String[] args) {
        int year = 2024;

        if (year % 4 == 0) {
            if (year % 100 == 0) {
                if (year % 400 == 0) {
                    System.out.println(year + " is a leap year");
                } else {
                    System.out.println(year + " is not a leap year");
                }
            } else {
                System.out.println(year + " is a leap year");
            }
        } else {
            System.out.println(year + " is not a leap year");
        }
    }
}
```

Figure 4.13: Leap Year Checker Using Nested If Statement

The code will output: **2024 is a leap year**

In-Text Questions (ITQs)

1. What are control structures in Java, and why are they important?
2. What types of control structures are commonly used?
3. How does the if-else statement work?
4. What is the purpose of the switch statement, and how does it differ from an if-else ladder?
5. Explain how the ternary operator simplifies decision-making in Java Which part of the electromagnetic spectrum has the longest wavelengths?

In-Text Answers (ITAs)

1. Control structures are programming constructs that manage the flow of execution in a program. They are important because they allow the program to make decisions, repeat actions, and control which parts of the code run based on certain conditions.
2. The common types are conditional structures (if, if-else, else-if ladder, switch), looping structures (for, while, do-while), and jump structures (break, continue, return).
3. The if-else statement executes one block of code if a condition is true, and another block if the condition is false, allowing two possible paths based on the condition.
4. The switch statement evaluates an expression once and executes the matching case block, which makes it more efficient for multiple fixed values. Unlike an if-else ladder, switch uses break statements to avoid fall-through.
5. The ternary operator is a shorthand for simple if-else statements. It evaluates a condition and returns one value if true and another if false, using the syntax: `condition? value_if_true : value_if_false`.

4.1.5 Conclusion

Understanding and applying decision-making structures is essential in programming. Java provides versatile conditional constructs (if, switch, ternary, etc.) that enable developers to control program flow based on logic and input. By mastering these tools, programmers can build responsive, flexible, and intelligent applications capable of handling a wide range of real-life scenarios.

Summary

This unit introduces control structures, focusing specifically on decision-making statements. These structures allow programs to make logical decisions and execute different blocks of code based on specific conditions. Essential concepts covered under conditional statements include:

- i. If: Executes code when a condition is true.
- ii. if-else: Chooses between two paths based on a condition.

- iii. else-if ladder: Checks multiple conditions in sequence.
- iv. Switch: An efficient way to handle multiple specific values of a variable.
- v. Ternary Operator: A compact, one-line if-else alternative for simple conditions.
- vi. Nested if: Allows checking conditions within conditions for more complex logic.

Each concept was illustrated with examples showing real-world logic such as checking eligibility, grading systems, login validation, and leap year determination.

Self-Assessment Questions

1. What is the main purpose of control structures in Java?
 A) To store data B) To control the flow of execution
 C) To declare variables D) To define classes
2. Which control structure is used for decision-making in Java?
 A) Loop B) Conditional C) Class D) Array
3. Which of the following is a conditional statement in Java?
 A) IF B) FOR C) BREAK D) CONTINUE
4. Which keyword is used to handle multiple conditions?
 A) LOOP B) REPEAT C) SWITCH D) JUMP
5. What does the if statement evaluate?
 A) A boolean condition B) A string
 C) A number D) An object
6. What is the output of the code if (5 > 3) System.out.println("Yes");?
 A) No B) Error C) Yes D) Nothing
7. Which statement is true about the if-else structure?
 A) It executes one block if true, another if false
 B) It repeats code C) It handles errors D) It breaks loops
8. How many conditions can be tested in an else-if ladder?
 A) Only one B) As many as needed
 C) Two D) None
9. Which is the correct syntax for a ternary operator?
 A) condition: true ? false B) if ? else :
 C) condition? true_value: false_value D) ? : ;
10. Which statement is equivalent to `x = (a > b) ? a : b;`?
 A) If `a > b` then `x = a` else `x = b` B) `x = a` or `b`
 C) `x = b` if `a < b` D) `x = b` if `a == b`

11. What does switch evaluate?
 - A) An expression or variable B) A condition only
 - C) A boolean D) A method
12. Which keyword is used to exit a switch case block?
 - A) CONTINUE B) RETURN C) BREAK D) EXIT
13. What is the default block in a switch statement used for?
 - A) First case B) Error handling
 - C) When no case matches D) Last case
14. What is the result of switch(level) when level = 2 and case 2: System.out.println("Intermediate");?
 - A) Error B) Nothing C) Intermediate D) Advanced
15. What is the output of: int x = 4; System.out.println((x % 2 == 0) ? "Even" : "Odd");
 - A) Odd B) Even C) Error D) None
16. Which of the following is NOT a decision-making structure?
 - A) IF B) FORC) SWITCH D) TERNARY
17. What is a valid use of if-else if ladder?
 - A) To evaluate multiple exclusive conditions B) To loop
 - C) To handle exceptions D) To declare classes
18. Which is true about the ternary operator?
 - A) It requires four operands B) It replaces switch
 - C) It simplifies simple if-else conditions
 - D) It's slower than if
19. Which is best for fixed value-based decisions?
 - A) IF B) SWITCH C) WHILE D) FOR
20. Which block in a switch executes when no cases match?
 - A) First B) Middle C) DEFAULT D) BREAK
21. What does this print?


```
if (age >= 18 && hasID) { System.out.println("Eligible"); }
```

 with age = 20 and hasID = true
 - A) Not eligible B) Eligible C) Error D) Undetermined
22. Which of the following is a jump control statement?
 - A) IF B) FORC) BREAK D) SWITCH
23. What type of expression is required in an if statement?
 - A) STRING B) BOOLEAN C) INTEGER D) CHAR
24. What happens if no break is used in a switch case?
 - A) All following cases execute ("fall-through")
 - B) Error C) Switch exits D) Nothing happens

Unit Activities

Activity 1: Design a Smart Decision-Making System

Objective: Apply if, if-else, and else-if statements to design and implement a real-world decision-making program using multiple inputs and nested conditions

Instructions:

Work in pairs to design a Java program that simulates an automated system (choose one from below or propose your own):

1. A health risk assessment tool based on temperature, age, and symptoms
2. A basic loan eligibility checker using salary, credit score, and employment type
3. A school fee discount calculator based on grade, performance, and sibling status

Tasks:

1. Create a flowchart representing your decision logic.
2. Implement the logic in Java using appropriate conditional statements.
3. Your program must accept at least three different user inputs and use nested conditions.
4. Submit the following:
 - a. Java code with comments
 - b. Screenshot of program output
 - c. Flowchart or pseudocode of logic

Activity 2: Algorithm Optimization: If-Else vs Switch

Objective: Evaluate and optimize decision logic by comparing if-else-if ladders with switch statements for maintainability and performance

Instructions:

In groups of 3-4, analyze a large-scale decision tree (e.g., course registration based on department, level, and credit load). Implement it using both if-else-if and switch

Tasks:

1. Implement the full logic in both structures.

2. Refactor your code to reduce redundancy.
3. Write a 200-word comparison report discussing:
 - a. Code readability and maintainability
 - b. Suitability of each structure
 - c. Potential performance impact
4. Submit Java code files, output screenshots, and the written comparison

Activity 3: Conditional Logic Debugging Challenge

Objective: Enhance debugging and tracing skills by analysing faulty conditional logic and correcting it using reasoning and systematic testing

Instructions:

Individually, you'll be given three faulty Java programs containing logical errors in if-else or nested conditions.

Task:

1. Identify all logic errors and describe the problem.
2. Fix the bugs and rewrite the code to work as intended.
3. Create a test table with at least 5 input cases per program, showing:
 - a) Inputs
 - b) Expected output
 - c) Actual output (before and after fixing)
4. Submit the corrected code, your test table, and a short explanation of each fix.

Activity 4: Implement a Dynamic Menu Using Switch and Ternary

Objective: Design an interactive Java application that uses switch and ternary operators effectively in real-world menus and interfaces.

Instructions:

Create a console-based menu-driven application for any of the following:

1. Restaurant ordering system
2. Library service portal
3. ATM transaction system

Task:

1. Use a switch to handle user-selected options (minimum of 5 options).

2. Use the ternary operator in at least 3 places for simple decisions (e.g., "balance >= amount? Proceed: Decline").
3. Include input validation and meaningful feedback for invalid selections.
4. Submit:
 - a. Complete Java source code
 - b. Screenshot of 2 full program runs
 - c. Brief documentation on your use of switch and ternary. Identify all logic errors and describe the problem.

Activity 5: Build a Multi-Level Access Control System (Nested Ifs)

Objective: Design a secure multi-condition validation system using nested if statements to simulate layered access control.

Instructions:

Individually design a role-based login system that checks:

1. Username and password
2. User role (admin, student, teacher)
3. User-specific privileges (e.g., read-only, edit, delete access)

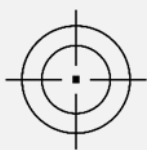
Tasks:

1. Accept multiple inputs: username, password, and role.
2. Implement layered checks using nested if statements (at least 3 levels deep).
3. Grant or deny access based on the combinations.
4. Submit:
 - a. Java source code
 - b. Test case table with different user roles and outcomes
 - c. Explanation of how nesting helped organise access levels

Unit 2 — Loops and Branches

INTRODUCTION

In everyday programming, running a piece of code just once often is not enough. Think about tasks like processing multiple user in-puts, working through lists of data, or repeating an action until something changes, these all need the program to repeat steps or make decisions along the way. Java gives us simple but powerful tools to handle these situations: loops to repeat actions and branch-ing statements to decide what to do next. In this unit, you will get to know the different kinds of loops, like for, while, and do-while, and learn how to use branching commands such as break, contin-ue, and re-turn to guide the flow of your program. By the time you finish this lesson, you will feel confident using these tools to build programs that can act in more flexible, intelligent ways.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Understand the purpose of loops and when to use them.
- Differentiate between for, while, and do-while loops.
- Use branching statements like break, continue and return appropriately.
- Write Java programs that use loops and branching to solve problems.

4.2.1 Introduction to Loops

Loops are used to execute a block of code repeatedly based on a condition. Java provides three loop structures:

The for Loop

Best suited for situations where the number of iterations is known. The syntax is:

```
for (initialization; condition; update) {  
  
    // Code to execute  
  
}
```

where:

- I. initialization: Sets the starting point (e.g. `int i = 0;`).
- II. condition: Loop continues while this is true (e.g. `i < 10;`).
- III. update: Changes the loop variable each iteration (e.g. `i++;`).

Example 1:

```
for(int i = 1; i <= 4; i++) {  
  
    System.out.println("Iteration: " + i);  
  
}
```

The output of the code is as follows:

Iteration: 1

Iteration: 2

Iteration: 3

Iteration: 4

The while Loop

It is used when the number of iterations is not known in advance and depends on a condition. The syntax is:

```
initialization;  
  
while (condtion) {
```

```
        // Statements to execute

        Update;

    }
```

Example 1:

```
int count = 1;

while (count <= 5) {

    System.out.println("Count: " + count);

    count++;

}
```

The do-while Loop

This is similar to the while loop, but it guarantees that the body of the loop executes at least. The syntax is:

```
    initialization;

    do {

        // Statements to execute

        update;

    } while (condition);
```

Example 1

```
int i = 1;

do {

    System.out.println("Do-While Value: " + i);

    i++;

} while(i <= 5);
```

4.2.2 Branching Statements

Branching statements alter the normal flow of control in loops and methods. They are used to:

- I. Exit loops prematurely (break).
- II. Skip the current iteration and proceed to the next (continue).
- III. Exit from a method early (return).

The break Statement

It is used to exit a switch statement immediately or to exit the loop entirely, even if the loop condition is still true.

Example 1: Usage of break in Switch Statement

```
public class Main {  
  
    public static void main(String[] args) {  
  
        char grade = 'B';  
  
        switch(grade) {  
  
            case 'A':  
  
            case 'B':  
  
            case 'C':  
  
                System.out.println("Pass");  
  
                break;  
  
            case 'D':  
  
            case 'F':  
  
                System.out.println("Fail");  
  
                break;  
  
            default:  
  
                System.out.println("Invalid grade");  
  
        }  
    }  
}
```

```
}  
  
}  
  
}
```

The output of the code is: Pass

Example 2: Usage of break in for Statement

```
for (int i = 1; i <= 10; i++) {  
    if (i == 5) {  
        break; // Exit the loop when i is 5  
    }  
    System.out.println("i = " + i);  
}
```

The output of the code is:

```
i = 1  
i = 2  
i = 3  
i = 4
```

The continue Statement

This skips the current iteration and proceeds to the next.

Example 1

```
for (int i = 1; i <= 4; i++) {  
    if (i == 3) {  
        continue; // Skips printing 3  
    }  
    System.out.println("i = " + i);  
}
```

The output is:

i = 1

i = 2

i = 4

The return Statement

Used to exit a method immediately and optionally returns a value. Useful in both branching logic and error handling.

Example 1

```
public int sum(int a, int b) {  
    return a + b; // Returns the sum and exits  
}
```

Example 2:

```
public class Main {  
    public static void checkNumber(int number) {  
        if(number < 0) {  
            System.out.println("Negative number!");  
            return; // Exit method  
        }  
        System.out.println("Valid number: " + number);  
    }  
  
    public static void main(String[] args) {  
        checkNumber(8);  
    }  
}
```

The output of the code is: Valid number: 8

In-Text Questions (ITQs)

1. How does the continue statement affect loop execution?
2. Why is understanding loops and branching statements important in Java programming?

In-Text Answers (ITAs)

1. The continue statement skips the current iteration of the loop and proceeds to the next iteration, effectively bypassing any code after it within the loop body for that cycle. How does the continue statement affect loop execution?
2. Understanding loops and branching statements is crucial because they allow programs to repeat actions and make decisions dynamically, enabling efficient handling of complex logic and data processing.

4.2.3 Conclusion

Mastering loops and branching statements is essential for any Java programmer aiming to write robust and dynamic applications. Loops allow repeated execution of code blocks without redundancy, while branching statements give you fine control over the flow of your program. These constructs together enable handling complex tasks such as processing multiple inputs, navigating data structures, and implementing conditional logic. With practice, you will be able to apply these concepts confidently to solve real-world problems, making your programs smarter and more efficient.

Summary

In this unit, we explored the fundamental control structures in Java that allow programs to repeat actions and make decisions dynamically. We learned about three main types of loops:

- i. For loop: Ideal when the number of iterations is known beforehand, with initialization, condition checking, and update all in one statement.

- ii. While loop: Used when the number of iterations depends on a condition and may not be known in advance; the condition is checked before executing the loop body.
- iii. Do-while loop: Similar to the while loop, but guarantees the loop body executes at least once because the condition is checked after the body.
- iv. We also studied branching statements that control the flow within loops and methods:
- v. Break: Exits a loop or switch statement immediately.
- vi. Continue: Skips the current iteration and moves to the next loop cycle.
- vii. Return: Exits a method immediately, optionally returning a value.

By understanding how to effectively use these loops and branching commands, you can write more flexible and efficient Java programs that respond intelligently to varying inputs and conditions.

Self-Assessment Questions

1. What is the main purpose of loops in Java?
 - A) To decide which code block to execute
 - B) To execute a block of code repeatedly
 - C) To stop program execution
 - D) To print output once
2. Which loop is best when the number of iterations is known in advance?
 - A) while loop
 - B) do-while loop
 - C) for loop
 - D) if-else statement
3. In the syntax for (initialisation; condition; update), what is the purpose of the 'update' part?
 - A) To initialise the loop variable
 - B) To check if the loop should continue
 - C) To change the loop variable after each iteration
 - D) To execute the loop body
4. Which loop guarantees the body is executed at least once?
 - A) for loop
 - B) while loop
 - C) do-while loop
 - D) if statement
5. What is the output of this code?

```
for(int i = 1; i <= 4; i++) {  
    System.out.println("Iteration: " + i);  
}
```

- A) Iteration: 1 to Iteration: 3 B) Iteration: 0 to Iteration: 4
 C) Iteration: 1 to Iteration: 4 D) Compilation error
6. In a while loop, when is the condition checked?
 A) Before executing the loop body
 B) After executing the loop body
 C) At the end of the program D) Inside the loop body
7. What will happen if the condition in a while loop is initially false?
 A) The loop executes once B) The loop executes infinitely
 C) The loop body is skipped entirely D) The program crashes
8. Which statement exits a loop immediately, even if the loop condition is still true?
 A) continue B) break C) return D) exit
9. What does the continue statement do inside a loop?
 A) Exits the loop completely B) Exits the program
 C) Repeats the current iteration
 D) Skips the current iteration and proceeds to the next
10. In a switch statement, what happens if you omit the break statement?
 A) Syntax error B) The program will stop running
 C) The switch statement will exit immediately
 D) Execution will fall through to the next case(s)
11. Which of these is NOT a Java branching statement?
 A) break B) continue C) return D) loop
12. What does the return statement do inside a method?
 A) Exits the loop
 B) Exits the method immediately/optionally returns a value
 C) Skips to the next iteration D) Starts a new method call

Consider the code:

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        break;
    }
    System.out.println("i = " + i);
}
```

13. What is the last value printed?
 A) 4 B) 5 C) 10 D) 1
14. What will be printed by this code?

```
for (int i = 1; i <= 4; i++) {
    if (i == 3) {
        continue;
    }
}
```

```

    }
    System.out.println("i = " + i);
  }

```

- A) i = 1, i = 2, i = 3, i = 4 B) i = 1, i = 2, i = 4
 C) i = 3 only D) i = 4 only
15. When should you use a do-while loop over a while loop?
 A) When you know the number of iterations in advance
 B) When the loop should execute at least once regardless of the condition
 C) When the condition must be checked first
 D) When you want to skip iterations
16. Which part of the for loop sets the starting value?
 A) update B) condition
 C) initialization D) loop body
17. What is the correct syntax for a while loop?
 A. while { condition } (statements);
 B. while (condition) { statements; }
 C. while: (condition) { statements; }
 D. while (condition) statements;
18. Which branching statement is used to skip the current iteration of a loop?
 A) break B) return C) continue D) skip
19. What will be the output of the following code snippet?
- ```

int i = 1;
do {
 System.out.println(i);
 i++;
} while (i <= 3);

```
- A) 1 2                      B) 1 2 3                      C) 2 3                      D) No output
20. Can continue be used outside loops?  
 A) Yes                      B) No                      C) None of the above

## Unit Activities

---

### Activity 1: Multiplication Table Creator

**Objective:** Practice using a for loop and conditional branching.

**Instructions:**

Write a Java program that asks the user to enter a positive integer  $n$ . Use a for loop to print the multiplication table of  $n$  from 1 to 10. If the user enters a non-positive number, display a message saying "Please enter a positive number" and stop the program. Use appropriate branching statements to handle this. Write a Java program that asks the user to enter a positive integer  $n$ .

**Tasks:**

1. Prompt the user to enter a positive integer  $n$ .
2. Check if  $n$  is positive; if not, print "Please enter a positive number" and terminate the program.
3. Use a for loop to iterate from 1 to 10.
4. Print the multiplication result for each iteration in the format:  $n \times i = \text{result}$ .
5. Test your program with both valid and invalid inputs

**Expected Output Example:**

Enter a number: 4

$4 \times 1 = 4$

$4 \times 2 = 8$

...

$4 \times 10 = 40$

**Activity 2: Number Sign Checker**

**Objective: Use a while loop combined with continue and break statements**

**Instructions:**

Create a Java program that repeatedly asks the user to input an integer. For each number entered, print "Positive" if the number is greater than zero, and print "Negative" if the number is less than zero, then skip the rest of the current loop iteration using continue. If the user enters zero, exit the loop using break and display "Program ended."

**Tasks:**

1. Set up a while loop to repeatedly prompt for an integer input.
2. If the number is greater than zero, print "Positive."
3. If the number is less than zero, print "Negative" and use continue to skip the rest of the loop iteration.
4. If the number is zero, use break to exit the loop and print "Program ended."
5. Test the program with positive, negative, and zero inputs.

### Activity 3: Password Verification

**Objective:** Use a do-while loop and branching to validate input.

**Instructions:**

Write a program that asks the user to enter a password. The program should keep prompting the user until the correct password "Java123" is entered. Once the password is correct, print "Access Granted" and end the program.

**Tasks:**

1. Use a do-while loop to prompt the user for a password.
2. Compare the input with the correct password "Java123".
3. If the password is incorrect, prompt the user to try again.
4. When the password is correct, print "Access Granted" and exit the loop.
5. Test by entering both right and wrong passwords.

### Activity 4: Student Grade Evaluator

**Objective:** Practice loops and switch statements for decision making.

**Instructions:**

Write a program that allows the user to enter student grades one at a time. Use a switch statement to print "Pass" for grades A, B, and C, and "Fail" for grades D and F. If an invalid grade is entered, print "Invalid grade." Continue accepting grades until the user inputs 'Q' to quit the program.

**Tasks:**

1. Create a loop that repeatedly prompts the user for grade.
2. Use a switch statement to evaluate the grade:
  - a. Create a loop that repeatedly prompts the user for grade.
  - b. Use a switch statement to evaluate the grade:
  - c. Print "Invalid grade" for any other input except 'Q'
3. If the user inputs 'Q', exit the loop and terminate the program.
4. Test the program with valid grades, invalid inputs, and the quit command.

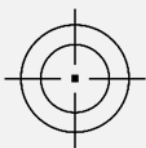
# Module Five

# **Methods**

# Unit 1 — Methods and Parameter Testing

## INTRODUCTION

In this unit, we dive into one of the core building blocks of Java programming, methods. You will learn how methods are defined, how to pass data into them using parameters, and how to return results. By exploring user-defined methods, you will begin to see how modular code can make your programs more organised, reusable, and easier to manage. We will also break down key concepts such as method signatures, variable scope, and how methods are invoked



### LEARNING OUTCOMES

#### At the end of this unit, you should be able to:

- Understand the purpose and structure of methods.
- Define and invoke both void and non-void methods.
- Differentiate between formal parameters and actual arguments.
- Differentiate between static and instance methods.
- Apply method overloading for flexible code.

### 5.1.1 What is a Method?

A method is a reusable block of code that performs a specific task. Methods help in:

- i. Modularising code: Breaking code into smaller, manageable parts.
- ii. Improving readability: Logical separation of tasks.
- iii. Reducing redundancy: Avoid repetition, write once, and you use many times.

The concept of write once and use many times can be demonstrated with the code block in Figure 5.1.

```
public class ShapeDetails {
 static void shapeInfo(int len, int width) {
 int area = len * width;
 int perimeter = 2 * (len + width);
 System.out.println("Length of the shape: " + len);
 System.out.println("Width of the shape: " + width);
 System.out.println("Area: " + area);
 System.out.println("Perimeter: " + perimeter);
 System.out.println();
 }

 public static void main(String[] args) {
 shapeInfo(5, 10); // First use
 shapeInfo(3, 7); // Second use
 shapeInfo(8, 4); // Third use
 shapeInfo(6, 6); // Fourth use (Square)
 }
}
```

Figure 5.1: Modularization Using Method

In the provided code (Figure 5.1), the method `shapeInfo` demonstrates how methods in Java help modularise code, improve readability, and reduce redundancy. The method is written once and reused multiple times with different values for length and width, allowing for calculations and printing of area and perimeter for various shapes without repeating the same code. By modularizing the task into a single method, the code becomes more manageable and maintainable, as changes to the logic (like how the area or perimeter is calculated) only need to be made in one place. Instead of rewriting the same logic for each shape, the method is reused, showcasing how Java methods can help write clean, concise, and maintainable code.

In Java, methods can be categorized based on return type, functionality and parameter usage.

### 5.1.2 Classification of Methods Based on Return Type

#### **Void Methods**

These methods perform actions but do not return any value. They use void as the return type (Figure 5.2). The syntax is:

```
accessModifier void methodName (parameterList) {

 // method body

}

1 public class Main {
2 public static void sayHello (String name) {
3 System.out.print("Hello " + name);
4 }
5
6 public static void main(String[] args) {
7 sayHello("James");
8 }
9 }
```

Figure 5.2: Simple Void Method

The method does not return any value. In line 7, the method, sayHello() is called with "James" as argument. It will output:

Hello James

#### **Non-Void (Return Type) Methods**

These methods return a value (Figure 5.3). The syntax is:

```
accessModifier dataType methodName (parameterList) {

 // method logic

 return value;

}
```

```
public class MethodAdd {
 public static int add(int a, int b) {
 return a + b;
 }

 public static void main(String[] args) {
 System.out.println("Sum: " + add(6, 3));
 }
}
```

Figure 5.3: Simple Return Method

### 5.1.3 Classification of Methods Based on Parameters

#### ***Parameterized Methods***

These methods take one or more input values (called parameters) to perform operation(s). The syntax is:

```
public static returnType methodName (dT1 p1, dT2 p2) {

 // Java statement

}
```

where: p1 is the first parameter with datatype of dT1

p2 is the second parameter with datatype of dT2

```
1 public class NameAge {
2 public static String Details (int age, String name) {
3 return name + " is " + age + " years old.";
4 }
5
6 public static void main(String[] args) {
7 System.out.println(Details(45, "Blake"));
8 }
9 }
```

Figure 5.4: Parameterized Method

#### ***Non-Parameterized Methods***

They do not require parameters (Figure 5.5). They perform tasks that are self-contained or use internal values only. The syntax is:

```
public static returnType methodName () {
```

```

 // Logic
 }

1 public class NonParaMethod {
2 static void whoAreYou() {
3 System.out.print("I am John Doe");
4 }
5
6 public static void main (String[] args) {
7 whoAreYou();
8 }
9 }

```

Figure 5.5: Non-Parameterised Method

#### 5.1.4 Static vs Non-Static Methods

Static methods belong to the class and can be called without creating an object. Non-static methods require creating an object of the class before calling it (Figure 5.6). The static method is `greet()`, while `saySomething()` is the non-static method. Most of the methods we have created so far, are static methods, we will discuss and provide more examples of non-static methods in the Object-Oriented Programming section.

```

1 public class NonStaticAndStatic {
2 public void saySomething() {
3 System.out.println("Hello World");
4 }
5
6 public static void greet() {
7 System.out.println("Good morning my friend!");
8 }
9
10 public static void main(String[] args) {
11 greet(); // Static method call
12
13 // Object creation
14 NonStaticAndStatic obj = new NonStaticAndStatic();
15 obj.saySomething(); // Instance method call
16 }
17 }

```

Figure 5.6: Static and Non-Static Methods

### 5.1.5 Parameters Passing in Java

Parameter passing refers to the mechanism of transferring data between methods. The types of parameter passing are:

- I. Pass-by-value (primitive types)
- II. Pass-by-reference (objects/arrays)

#### ***Pass-by-Value (Primitive Types)***

A copy of the value is passed (Figure 5.7). Changes inside the method do not affect the original variable.

```
public class PassByValue {
 static void modifyValue(int number) {
 number = number + 10;
 System.out.println("Inside method: " + number);
 }

 public static void main(String[] args) {
 int myNumber = 15;
 System.out.println("Before method call: " + myNumber);
 modifyValue(myNumber);
 System.out.println("After method call: " + myNumber);
 }
}
```

Figure 5.7: Pass-by-Value Parameter Passing 1

The output of the above code is as follows:

Before method call: 15

Inside method: 25

After method call: 15

We started with myNumber set to 15 and passed it to modifyValue. Since Java uses pass-by-value, only a copy of the value (15) is changed (to 25), while the original myNumber stays 15. Figure 5.8 also shows another example of pass-by-value.

```

1 public class PassByValueChar {
2 static void modifyChar(char x) {
3 x = 'Z';
4 System.out.println("Inside method: " + x);
5 }
6
7 public static void main(String[] args) {
8 char y = 'A';
9 System.out.println("Before method call: " + y);
10 modifyChar(y);
11 System.out.println("After method call: " + y);
12 }
13 }

```

Figure 5.8: Pass-by-Value Parameter Passing 2

The output for the code in Figure 5.8 will output:

Before method call: A

Inside method: Z

After method call: A

The `modifyChar(char c)` method receives a copy of the `char` value, changes it to 'Z', and prints it. However, the original variable `y` in `main` remains unchanged because Java passes primitives by value.

### ***Pass-by-Reference (Objects and Arrays)***

The memory address (reference) is passed. Changes inside the method affect the original object.

#### ***Example 1***

In Figure 5.9, the array `numbers` is passed to the `modifyArray` method, where its first element is set to 99. Because arrays are passed by reference, this change affects the original array in the `main` method. However, reassigning the array inside the method won't affect the original. We'll demonstrate this using a class (explained further in the Object-Oriented Programming section).

```

1 public class PassByReferenceArray {
2 static void modifyArray(int[] arr) {
3 arr[0] = 99;
4 System.out.println("Inside Method: " + arr[0]);
5 }
6
7 public static void main(String[] args) {
8 int[] numbers = {1, 2, 3};
9 System.out.println("Before Method Call: " + numbers[0]);
10 modifyArray(numbers);
11 System.out.println("After Method Call: " + numbers[0]);
12 }
13 }

```

Figure 5.9: Pass by Reference Parameter Passing

The output of the code in Figure 5.9 is as follows:

Before Method Call: 1

Inside Method: 99

After Method Call: 99

Example 2

```

1 class Person {
2 String name;
3
4 Person(String name) {
5 this.name = name;
6 }
7 }
8
9 public class PassByReferenceReassign {
10 static void reAssignObject(Person personVar) {
11 personVar = new Person("John");
12 System.out.println("Inside Method: " + personVar.name);
13 }
14
15 public static void main(String[] args) {
16 Person person1 = new Person("Charles");
17 System.out.println("Before method call: " + person1.name);
18 reAssignObject(person1);
19 System.out.println("After method call: " + person1.name);
20 }
21 }

```

The code in Figure 5.10 will output the following:

Before method call: Charles

Inside Method: John

After method call: Charles

In `reAssignObject`, reassigning `personVar` to a new object doesn't affect `person1` in `main` because only a copy of the reference is passed, so the original still points to the object with the name "Charles."

The output of the code is: Valid number: 8

### ***In-Text Questions (ITQs)***

---

1. What is the main purpose of using methods in Java?
2. What is the difference between a void method and a non-void method?
3. What happens when you pass a primitive type to a method?
4. Why doesn't changing an object reference inside a method affect the original object?

### ***In-Text Answers (ITAs)***

---

1. Methods in Java are used to modularise code, improve readability, and eliminate redundancy by allowing us to write code once and reuse it multiple times.
2. A void method performs an action but does not return any value, while a non-void method performs an action and returns a value of a specified data type
3. When a primitive type is passed to a method, only a copy of the value is used. Any changes inside the method do not affect the original variable.
4. Because Java passes object references by value, changing the reference inside a method only affects the local copy and not the original reference in the caller. The `continue` statement skips the current iteration of the loop and proceeds to the next iteration, effectively bypassing any code after it within the loop body for that cycle. How does the `continue` statement affect loop execution?

### 5.1.6 Method Overloading

This is a feature that allows a class to have more than one method with the same name but with different parameter lists (Figure 5.11). The Java compiler selects the appropriate method based on:

- I. iMethod signature (name + parameter list).
- II. Arguments passed during the method call.

```
public class PrintDisplay {
 static void print (String name) {
 System.out.println("I am " + name);
 }

 static void print (int age) {
 System.out.println("I am " + age + " years old");
 }

 public static void main(String[] args) {
 print ("John"); // Calls the first method
 print (18); // Calls the second method
 }
}
```

Figure 5.11: Overloading of Print Method

Consider the code snapshot in Figure 5.12. The class PlusCalculator defines three add methods:

- I. add (int a, int b): Adds two integers.
- II. add (int a, int b, int c): Adds three integers.
- III. add (double a, double b): Adds two doubles.

The main method creates a PlusCalculator object and calls all three overloaded add methods with different parameters.

```

class PlusCalculator {
 // Method 1: Adds two integers
 int add(int a, int b) {
 return a + b;
 }

 // Method 2: Adds three integers (different parameter count)
 int add(int a, int b, int c) {
 return a + b + c;
 }

 // Method 3: Adds two doubles (different parameter types)
 double add(double a, double b) {
 return a + b;
 }
}

public class Calculator {
 public static void main(String[] args) {
 PlusCalculator calc = new PlusCalculator();
 System.out.println(calc.add(5, 10)); // Calls Method 1
 System.out.println(calc.add(2, 3, 4)); // Calls Method 2
 System.out.println(calc.add(1.5, 2.5)); // Calls Method 3
 }
}

```

Figure 5.12: Overloading the add() Method

### 5.1.7 Type Promotion

Java automatically promotes smaller data types to larger ones if an exact match is not found:

byte => short => int => long => float => double

char => int

#### Example 1

```

public class TypePromotion {
 static void display (int x) {
 System.out.println("Int: " + x);
 }

 public static void main(String[] args) {
 display('A');
 }
}

```

Figure 5.13: Automatic Type Conversion Example 1

In the code in Figure 5.13, the output is 65, as the char 'A' was promoted to int.

### Example 2

```
class ExampleOne {
 void show(int x) {
 System.out.println("int method: " + x);
 }

 public static void main(String[] args) {
 byte b = 5;
 new ExampleOne().show(b);
 }
}
```

Figure 5.14: Automatic Type Conversion Example 2

In Figure 5.14, Java promotes byte to int since int is the closest matching type, despite the absence of a show(byte) method.

### Example 3

```
class ExampleFive {
 void test(double x) {
 System.out.println("double method: " + x);
 }

 public static void main(String[] args) {
 short s = 2;
 new ExampleFive().test(s);
 }
}
```

Figure 5.15: Type Conversion through Multiple Steps

In Figure 5.15, short is promoted through multiple steps. Short is promoted through int, then through long, through float and finally to double. Since only double is defined, Java promotes step-by-step until it fits.

### ***In-Text Questions (ITQs)***

---

1. What is method overloading?
2. What is the significance of static methods?
3. Can Java promote data types automatically in method overloading?

### ***In-Text Answers (ITAs)***

---

1. Method overloading allows a class to have multiple methods with the same name but different parameter lists. The correct method is chosen based on the arguments passed.
2. Static methods belong to the class rather than an instance and can be called without creating an object of the class.
3. Yes, Java can promote smaller data types to larger ones (like byte to int) if no exact match is found for the method parameters during overloading.

#### **5.1.8 Conclusion**

Understanding methods and parameter passing is essential for writing clean, modular, and efficient Java programs. Methods enable developers to reuse code, separate concerns, and manage complexity effectively. Mastering the differences between void and return-type methods, static and non-static methods, as well as parameter passing techniques, lays a strong foundation for more advanced topics such as object-oriented programming. Additionally, method overloading enhances code flexibility and usability by allowing multiple variations of a method tailored to different inputs.

### ***Summary***

---

This unit introduced methods in Java, explaining how they help organize code into reusable blocks that perform specific tasks. You learned about void and non-void methods, parameterised and non-parameterised methods, and the difference between static and non-static methods. The unit covered parameter passing:

1. Pass-by-value for primitives, where changes inside methods don't affect the original variables.
2. Pass-by-reference for objects and arrays, where changes inside methods affect the original data.

Finally, we explored method overloading and how Java uses type promotion to match method calls.

## Self-Assessment Questions

---

1. Which does not represent method overloading?  
A. `void display(int a) { ... }`  
`void display(int a, int b) { ... }`  
B. `int calculate() { ... }`  
`double calculate() { ... }`  
C. `void print(int a, double b) { ... }`  
`void print(double a, int b) { ... }`  
D. `void show(int num) { ... }`  
`void show(String text) { ... }`
2. Which is the correct syntax to return a value from a method?  
A. `return value;` B. `value;`  
C. `return;` D. `return dataType;`
3. Which type of method must be invoked on an instance of a class?  
A. static method B. non-parameterized method  
C. instance method D. void method
4. What happens if you pass a char to a method that expects an int in Java?  
A. The code will cause a compilation error.  
B. The char will be converted automatically to int.  
C. The method will fail at runtime  
D. The char will be treated as a string
5. Which of the following will pass an object reference to a method?  
A. Passing an array B. Passing an integer  
C. Passing a boolean D. Passing a float
6. What is the return type of a method that does not return any value?  
A. int B. void C. boolean D. String
7. What is the correct syntax for a method with parameters?  
A. `public static void methodName() { }`  
B. `public static void methodName(int param) { }`  
C. `public static methodName() { }`  
D. `public void methodName(int param) { }`
8. What does a method signature include?  
A. Only the method name  
B. Method name and return type  
C. Method name and parameter list  
D. Method name, return type and parameter list.
9. What does the "static" keyword in a method mean?  
A. The method can only be used in the same class

- B. The method belongs to the class (not object)
  - C. The method returns a value
  - D. The method cannot be overloaded
10. What happened if you try to modify the value of a primitive type passed to a method?
- A. The original value is modified
  - B. A new variable is created in the method
  - C. The original value is returned
  - D. None of the above
11. What is the output when an integer is promoted to a double in method overloading?
- A. The integer value remains the same
  - B. The integer value is truncated
  - C. The integer value is automatically converted to double
  - D. None of the above
12. How do you invoke a static method in Java?
- A. By creating an object of the class
  - B. By calling it directly using the class name
  - C. By passing arguments to the class
  - D. None of the above
13. Which of the following is an example of pass-by-reference?
- A. Passing an int variable
  - B. Passing a char variable
  - C. Passing an array or object
  - D. Passing a string
14. In Java, how are primitive types passed to method? By:
- A. By reference
  - B. By value
  - C. By address
  - D. By object
15. Which of the following methods cannot be overloaded?
- A. Methods with different return type
  - B. Methods with the same parameter types and different numbers of parameters
  - C. Methods with different names
  - D. Methods sharing a name but different parameter types
16. Which is true about passing an array to a method in Java?
- A. The array is passed by value
  - B. The array is passed by reference
  - C. The array is passed by address
  - D. The array cannot be passed to a method
17. Which statement is true about static methods?
- A. They can access instance variables directly
  - B. They cannot be called without an object
  - C. They can be accessed directly using the class name
  - D. They cannot be overloaded

18. How does Java handles type promotion in method overloading?

- A. Java promotes smaller data types to larger ones
- B. Java automatically converts all data types to double
- C. Java will not allow type promotion in overload methods
- D. Java promotes larger data types to smaller ones

19. What is the output?

```
void mmodifyArray(int[] arr) { arr[0] = 99; }
public static void main (String[] args) {
 int [] nums = {1, 2, 3};
 modifyArray(nums);
 System.out.println(nums[0]);
}
```

- A. 1
- B. Compilation error
- C. 99
- D. NullPointerException

20. A static method is called using:

- A. An object
- B. The class name
- C. A constructor
- D. A return statement

21. Which is true about static method?

- A. They can access instance variables directly
- B. They belong to the class, not objects
- C. They cannot be overloaded
- D. They must return void

22. What is the purpose of modularizing code with methods?

- A. To make the code longer
- B. To improve readability and reduce redundancy
- C. To slow down program, execution
- D. To hide errors

23. What is the return type of this method?

```
int add (int a, int b) { return a + b; }
```

- A. void
- B. int
- C. double
- D. None

24. Which method correctly returns a String?

- A. void getName() { return "Alice"; }
- B. String getName() { return "Alice"; }
- C. String getName() { System.out.println("Alice"); }
- D. getName() { return "Alice"; }

25. Type promotion in overloading converts:

- A. int to String
- B. byte to int
- C. double to float
- D. long to byte

26. What is the output of the code

```
void greet() { System.out.println("Hello"); }
public static void main(String[] args) {
 greet();
}
```

- A. Compilation error    B. "Hello"    C. No output  
D. Runtime error

## ***Unit Activities***

---

### **Activity 1: Defining and Calling Methods**

**Objective: Learn to define simple methods and invoke them**

**Instructions:**

1. Define a void method called greetUser that takes a String parameter for the user's name and prints a greeting message.
2. In the main method, call greetUser with different names

**Task:**

1. Write the greetUser method.
2. Call it at least three times with different names.
3. Observe the output

### **Activity 2: Using Return Values from Methods**

**Objective: Understand how to define non-void methods that return values.**

1. Write a method called square that takes an integer and returns its square.
2. Call the method in main and print the result for numbers 4, 7, and 10

**Task:**

1. Implement the square method.
2. Use the returned value in the main method.

### **Activity 3: Pass-by-Value Demonstration**

**Objective: Explore how primitive types are passed by value in methods.**

**Instructions:**

1. Create a method `increaseByTen` that takes an `int` parameter and adds 10 to it inside the method.
2. Print the value before the method call, inside the method, and after the method call to observe changes

**Tasks:**

1. Implement `increaseByTen`.
2. Call it from main with a variable initialized to 20.
3. Record the output and explain why the original variable remains unchanged

**Activity 4: Pass-by-Reference with Arrays**

**Objective: Understand how arrays are passed by reference to methods.**

**Instructions:**

1. Write a method `modifyFirstElement` that takes an `int` array and sets the first element to 100.
2. In the main method, create an array and print its contents before and after calling `modifyFirstElement`

**Tasks:**

1. Implement `modifyFirstElement`.
2. Demonstrate how the original array changes after the method call Implement the square method.

**Activity 5: Method Overloading Practice**

**Objective: Practice creating multiple methods with the same name but different parameters.**

**Instructions:**

1. Create a class with three overloaded methods named `add`:
  - a. `add(int a, int b)` returns the sum of two integers.
  - b. `add(double a, double b)` returns the sum of two doubles.
  - c. `add(int a, int b, int c)` returns the sum of three integers.
2. Call each method with appropriate arguments and print the results.
3. before and after calling `modifyFirstElement`

**Tasks:**

1. Implement `modifyFirstElement`.
2. Demonstrate how the original array changes

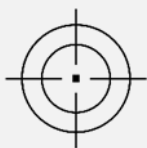
Module Six

# **Object-Oriented Programming (OOP)**

# Unit 1 — Object-Oriented Programming (OOP) Basics

## INTRODUCTION

In this unit, we begin our journey into Object-Oriented Programming (OOP), a fundamental concept in Java and a widely adopted programming paradigm in modern software development. OOP emphasizes structuring code around classes and objects, allowing us to build reusable, modular, and scalable applications. This unit also introduces the core principles of OOP, such as encapsulation (restricting direct access to data and exposing only necessary parts) and abstraction (focusing on essential features while hiding implementation details). Mastering these foundational concepts prepares you for more advanced topics.



### LEARNING OUTCOMES

#### At the end of this unit, you should be able to:

- Understand the meaning of Object-Oriented Programming.
- Define and explain the key principles of OOP: classes, objects, encapsulation and abstraction.
- Create simple Java classes and objects.
- Demonstrate encapsulation using access modifiers.
- Apply abstraction using methods and class design

### 6.1.1 What is Object-Oriented Programming?

Object-Oriented Programming (OOP) is a paradigm that organizes software design around objects (instances of classes) rather than functions and logic to model real-world entities. Objects represent real-world entities with attributes (data) and behaviours (methods). The key concepts of OOP are (the last four are considered the pillars of OOP):

- i. Class
- ii. Object
- iii. Encapsulation
- iv. Abstraction
- v. Inheritance (covered in later lessons)
- vi. Polymorphism (covered in later lessons)

#### ***Why is OOP Important?***

The following are the importance of OOP:

- i. **Modularity and Reusability:** Organizes code into reusable, self-contained components (such as classes or objects), making it easier to maintain and scale.
- ii. **Real-World Modelling:** Represents real-world entities through objects, making it easier to conceptualise.
- iii. **Powerful Features:** Supports key principles like inheritance, polymorphism, encapsulation, and abstraction, enabling robust and flexible code design.

### 6.1.2 Class

A class is a blueprint or template for creating objects. It defines the structure (which includes the attributes (variables) and behaviour (methods)) of an object. The syntax is:

```
public class ClassName {

 // Fields (attributes)

 // Methods (behaviour)

}
```

### Example 1: Person Class

Figure 6.1 represent a screenshot of a Person class that contains two fields name (String) and age (int) with a method introduce() that prints the person's details.

```
public class Person {
 // Attributes (fields)
 String name;
 int age;

 // Method (Behaviour)
 void introduce() {
 System.out.println("My name is " + name);
 System.out.println("I am " + age + " years old");
 }
}
```

Figure 6.1: Person Class Definition

### Example 2: Car Class

In the example below, the Car class models a car with attributes (model, color, year) and a start() method.

```
public class Car {

 String model, color;

 int year

 void start() {

 System.out.println("Car started!")

 }

}
```

#### 6.1.3 What is OOP?

An object is a specific instance of a class. Once you have defined a class, you can create objects from it. The syntax is:

```
Classname objectName = new Classname();
```

### Example 1: Student Object

In Figure 6.2 a Student class with two attributes: name and age was defined. It creates an object of the Student class in the main method and prints the values of these fields.

```
// Define the Student class
public class Student {
 // Fields of the Student class: name and age
 String name = "Alice"; // Default name
 int age = 20; // Default age

 // main method to run the program
 public static void main(String[] args) {
 // Create an object of the Student class
 Student s1 = new Student();

 // Print the name and age of the Student object
 System.out.println(s1.name); // Output: Alice
 System.out.println(s1.age); // Output: 20
 }
}
```

Figure 6.2: Simple Code Illustrating the Concept of OOP

### Example 2: Car Object

A Car class (Figure 6.3) with make and model fields is created. In the main method, an object myCar is created, and its properties are accessed and printed.

```
public class Student {
 String name;
 int age;

 void displayInfo() {
 System.out.println("Hi there! I am " + name);
 System.out.println("I am " + age + " years old.");
 }

 public static void main(String[] args) {
 Student s1 = new Student();
 s1.name = "Lara";
 s1.age = 20;
 // Call the displayInfo method
 s1.displayInfo();
 }
}
```

Figure 6.3: Car Class and Car Object

### Example 3: Student with Method

A Student class (Figure 6.4) with two fields: name and age and a method displayInfo() to print the student's details. In main(), an object s1 is created, values are assigned, and the info is displayed.

```
public class Student {
 String name;
 int age;

 void displayInfo() {
 System.out.println("Hi there! I am " + name);
 System.out.println("I am " + age + " years old.");
 }

 public static void main(String[] args) {
 Student s1 = new Student();
 s1.name = "Lara";
 s1.age = 20;
 // Call the displayInfo method
 s1.displayInfo();
 }
}
```

Figure 6.4: Student Class and Student Object

### Example 4: Calculator with User Input

The Calculator class (Figure 6.5) has an add method that adds two numbers. It takes user input using the Scanner class, creates a Calculator object, calls the add method, and displays the sum.

```
import java.util.Scanner;
public class Calculator {
 // Method to add two numbers
 public int add(int a, int b) {
 return a + b;
 }

 public static void main(String[] args) {
 Scanner input = new Scanner(System.in);

 // Prompting user to enter the first number
 System.out.print("Enter the first number: ");
 int num1 = input.nextInt();

 // Prompting user to enter the second number
 System.out.print("Enter the second number: ");
 int num2 = input.nextInt();

 // Creating an object of Calculator class
 Calculator calc = new Calculator();

 // Call the add method and print the result
 int result = calc.add(num1, num2);
 System.out.println("Sum: " + result);
 }
}
```

Figure 6.5: Calculator Class

### 6.1.4 Constructor

A constructor is a special method used to initialise objects of a class. It is called automatically when an object is created. The following are key features of a constructor:

- i. The name of a constructor must be the same as the class.
- ii. It does not have a return type, not even void.
- iii. If none is written, Java provides a default constructor.
- iv. You can have multiple constructors in one class (constructor overloading).

#### **Default Constructor**

A default constructor is one that takes no parameters. If you do not define a constructor, Java provides a default one.

##### *Example 1: Default Constructor (CarBrand)*

In the CarBrand class (Figure 6.6), the default constructor sets the brand to "Toyota". When a CarBrand object (c1) is created, the constructor is called, setting the brand to Toyota. The main() method then prints the brand value directly using System.out.println(). This demonstrates how a default constructor assigns a default value to an object's property during instantiation.

```
public class CarBrand {
 String brand;

 // Default constructor
 public CarBrand() {
 brand = "Toyota";
 }

 public static void main(String[] args) {
 // Constructor is automatically called
 CarBrand c1 = new CarBrand();
 System.out.println("Brand: " + c1.brand);
 }
}
```

Figure 6.6: CarBrand Class

##### *Example 2: Default Constructor (Book)*

In Figure 6.7, the Book class has a default constructor that sets title to "Java Basics" and author to "John Doe". When a Book **object** is created, the constructor runs automatically.

```

public class Book {
 String title;
 String author;

 // Default constructor
 Book() {
 title = "Java Basics";
 author = "John Doe";
 }

 void display() {
 System.out.println("Title: " + title);
 System.out.println("Author: " + author);
 }

 public static void main(String[] args) {
 // Calls the default constructor
 Book book1 = new Book();
 book1.display();
 }
}

```

Figure 6.7: Non-parameterised Book Constructor

### ***Parameterized Constructor***

A parameterised constructor accepts arguments to initialise an object with custom values.

#### ***Example 1***

The Book class (Figure 6.8) has two instance variables: title and author. A parameterized constructor sets the author and title when creating an object. b1 is initialised with "Java Basics" and "John Doe" as title and author, respectively.

```

class Book {
 String title, author;
 Book(String t, String a) { // Parameterized constructor
 title = t;
 author = a;
 }
 void display() {
 System.out.println(title + " by " + author);
 }
 public static void main(String[] args) {
 Book b1 = new Book("Java Basics", "John Doe");
 b1.display();
 }
}

```

Figure 6.8: Parameterised Book Constructor

### ***In-Text Questions (ITQs)***

---

1. What is the difference between a class and an object?
2. What is the role of a constructor in a Java class?

### ***In-Text Answers (ITAs)***

---

1. A constructor initialises new objects of a class. It sets initial values for fields when an object is created. Java provides a default constructor if none is defined.
2. A class is a blueprint or template that defines the attributes and behaviors of an object. An object is an instance of a class, created using the new keyword, and represents a specific implementation of that blueprint.

#### **6.1.5 Encapsulation**

Encapsulation is the mechanism of bundling data (fields) and methods (behavior) into a single unit (class) while restricting direct access to some components.

#### ***How to Achieve Encapsulation***

To achieve encapsulation:

- I. Declare fields as private (restrict direct access).
- II. Provide public getter/setter methods to control access.

#### ***Why Encapsulation?***

- i. Data Hiding: Prevents unauthorised access/modification.
- ii. Validation: Setters can validate input (e.g., prevent negative balance).
- iii. Flexibility: Internal logic can change without affecting other code.

#### ***Example 1: Encapsulated BankAccount Class***

In the BankAccount class (Figure 6.9), accountNumber and balance are private, preventing direct access and protecting the integrity of the data. Public methods like getBalance(), deposit(), and withdraw() control how the balance is accessed and modified, ensuring only valid operations (e.g., positive deposits, withdrawal not exceeding balance).

```
public class BankAccount {
 // Private fields (hidden from outside)
 private String accountNumber;
 private double balance;

 // Public constructor
 public BankAccount(String accountNo, double bal) {
 this.accountNumber = accountNo;
 if (balance >= 0) {
 this.balance = bal;
 }
 }

 // Public getter (read-only access)
 public double getBalance() {
 return balance;
 }

 // Public setter (controlled modification)
 public void deposit(double amount) {
 if (amount > 0) {
 balance += amount;
 }
 }

 // Public method (secure withdrawal)
 public void withdraw(double amount) {
 if (amount > 0 && amount <= balance) {
 balance -= amount;
 } else {
 System.out.println("Invalid amount!");
 }
 }
}
```

```
public class BankAccountMain {
 public static void main(String[] args) {
 BankAccount acc = new BankAccount("12345", 1000);
 acc.deposit(500);
 acc.withdraw(200);
 System.out.println("Balance: $" + acc.getBalance());
 // acc.balance = -1000; Error: balance is private!
 }
}
```

Figure 6.9: Encapsulated Bank Account

### Example 2: Encapsulated Car Class

```
class Car {
 private String model; // Private field
 private int year;

 // Getter method for model
 public String getModel() {
 return model;
 }

 // Setter method for model
 public void setModel(String model) {
 this.model = model;
 }

 // Getter method for year
 public int getYear() {
 return year;
 }

 // Setter method for year
 public void setYear(int year) {
 if (year > 1885) { // Validating the year
 this.year = year;
 } else {
 System.out.println("Invalid year");
 }
 }
}
```

Figure 6.10: Encapsulated Car Class

This Java code in Figure 6.10 defines a class named `Car`, which encapsulates `model` and `year`. The class provides getter and setter methods for accessing and modifying these fields. The `getModel()` and `getYear()` methods return the values of the `model` and `year`, respectively. The `setModel()` and `setYear()` methods allow setting these fields, with the `setYear()` method including validation to ensure the year is greater than 1885, which is considered the birth year of the modern car.

#### 6.1.6 Abstraction

##### **Abstraction**

Abstraction is the process of hiding internal implementation details and showing only the essential features of an objects.

## Basic Abstraction Using Methods

Method-level abstraction hides complex logic inside private methods and exposes only essential functionality through simple, public method calls.

### Example 1: Car Starting System

CarStartSystem class (Figure 6.11) demonstrates abstraction, where the start() method provides an interface for the user to start the car, while steps like turning on the engine, checking fuel, and starting movement are hidden in private methods.

```
class CarStartSystem {
 // public method abstracts complex internal steps
 public void start() {
 System.out.println("Starting the car...");
 turnEngineOn();
 checkFuel();
 startMoving();
 }

 // These methods are hidden from the user
 private void turnEngineOn() {
 System.out.println("Turning Engine On");
 }

 private void checkFuel() {
 System.out.println("Checking Fuel");
 }

 public void startMoving() {
 System.out.println("Car is Moving!");
 }
}

public class CarStartSystemMain {
 public static void main(String[] args) {
 CarStartSystem car = new CarStartSystem();
 car.start();
 }
}
```

Figure 6.11: Car Start System Class

### Example 2: ATM Operation

The code (Figure 6.12) demonstrates method-level abstraction in the ATM class, where the withdrawCash() method simplifies the cash withdrawal process. Internal steps like inserting the card, entering the PIN, and dispensing cash are hidden in private methods, allowing users to interact only with the high-level withdrawCash() method.

```

class ATM {
 public void withdrawCash() {
 insertCard();
 enterPin();
 processTransaction();
 dispenseCash();
 }

 private void insertCard() {
 System.out.println("Card inserted.");
 }

 private void enterPin() {
 System.out.println("PIN entered.");
 }

 private void processTransaction() {
 System.out.println("Processing transaction");
 }

 private void dispenseCash() {
 System.out.println("Cash dispensed.");
 }
}

public class ATMOperation {
 public static void main(String[] args) {
 ATM atm = new ATM();
 atm.withdrawCash();
 }
}

```

Figure 6.12: Class Simulating the Operations of ATM

### ***Abstraction Using Abstract Class***

An abstract class cannot be instantiated directly. It provides partial implementation of methods and leave other methods abstract.

#### ***Example 1:***

Vehicle (Figure 6.13) is an abstract class that has an abstract method start() (no implementation). The Car and Bike classes must provide their own implementations of start(). The stop() method is already implemented in the Vehicle class.

```

abstract class Vehicle {
 // Abstract method (no body)
 public abstract void start();

 // Concrete method
 public void stop() {
 System.out.println("Vehicle stopped!");
 }
}

class Car extends Vehicle {
 @Override
 public void start() {
 System.out.println("Car started!");
 }
}

class Bike extends Vehicle {
 @Override
 public void start() {
 System.out.println("Bike started!");
 }
}

public class Main {
 public static void main(String[] args) {
 Vehicle myCar = new Car();
 myCar.start(); // Prints: Car started!
 myCar.stop(); // Prints: Vehicle stopped!

 Vehicle myBike = new Bike();
 myBike.start(); // Bike started!
 }
}

```

Figure 6.13: Abstraction Using Abstract Class

### ***Abstraction Using Interfaces***

Abstraction using interfaces allows you to define a set of methods that must be implemented by any class that uses the interface, without specifying how those methods should work. This hides the implementation details and focuses on what an object can do, not how it does it.

#### ***Example 1: Startable Interface***

The Startable interface defines a contract with a single method `start()`. The Car and Bike classes both implement this interface and provide their own version of the `start()` method.

```

interface Startable {
 void start(); // Abstract method, no body
}

class Car implements Startable {
 @Override
 public void start() {
 System.out.println("Starting the car");
 }
}

class Bike implements Startable {
 @Override
 public void start() {
 System.out.println("Starting the bike");
 }
}

public class StartableMain {
 public static void main(String[] args) {
 Startable myCar = new Car();
 myCar.start(); // Prints: Starting the car

 Startable myBike = new Bike();
 myBike.start(); // Prints: Starting the bike
 }
}

```

Figure 6.14: Abstraction Using Interface

### 6.1.7 Abstraction Using: Interface Vs Abstract Class

The differences between abstraction using interface and using abstract class are listed in Table 6.1:

| Feature                      | Interface                                            | Abstract Class                                      |
|------------------------------|------------------------------------------------------|-----------------------------------------------------|
| Purpose                      | Defines a contract for the implementing class.       | Provides both a contract and some common behaviour. |
| Inheritance                  | A class can implement multiple interfaces.           | A class can extend only one abstract class.         |
| Implementation Requirement   | Must implement all methods defined in the interface. | Can inherit some methods (from abstract class).     |
| Access Modifiers for Methods | All methods are implicitly public.                   | Can have different access modifiers for methods.    |
| Use Case                     | For defining behaviour for unrelated classes.        | For related classes with shared functionality.      |

### Example 1: Abstraction Using an Abstract Class

Abstract classes hide implementation details while enforcing a common structure for subclasses. In the Figure 6.15, FlyingCreature defines a general behavior breathe() and requires subclasses to implement the fly() method.

```
abstract class FlyingCreature {
 // Concrete method (common behavior)
 void breathe() {
 System.out.println("Breathing in air");
 }

 // Abstract method
 // (must be implemented by subclasses)
 abstract void fly();
}

// Subclass Bird extends FlyingCreature
class Bird extends FlyingCreature {
 @Override
 void fly() {
 System.out.println("Bird is flying");
 }
}

public class Main {
 public static void main(String[] args) {
 FlyingCreature bird = new Bird();
 bird.breathe(); // (inherited method)
 bird.fly(); // (implemented method)
 }
}
```

Figure 6.15: Abstraction Using Abstract Class 2

### Example 2: Abstraction Using an Interface

Flyable interface defines fly() and breathe(), which Bird implements with its own behaviour (Figure 6.16). This shows how interfaces support abstraction by enforcing structure while allowing custom behaviour.

```

interface Flyable {
 // The abstract methods (fly() and
 // breathe(), must be implemented
 void fly();
 void breathe();
}

// Class that implements the Flyable interface
class Bird implements Flyable {
 @Override
 public void fly() {
 System.out.println("Bird is flying");
 }

 @Override
 public void breathe() {
 System.out.println("Bird is breathing");
 }
}

public class Main {
 public static void main(String[] args) {
 Flyable bird = new Bird();
 bird.fly();
 bird.breathe();
 }
}

```

Figure 6.16: Abstraction Using Interface

### ***In-Text Questions (ITQs)***

---

1. Why is encapsulation important in Java programming?
2. How does abstraction simplify code for users?
3. What is the difference between an abstract class and an interface?
4. How can you demonstrate encapsulation in a Java class?

### ***In-Text Answers (ITAs)***

---

1. Encapsulation helps protect data by restricting direct access to class fields using private access modifiers and exposing controlled access through public getter and setter methods. This ensures data integrity and supports validation logic.

2. Abstraction hides internal implementation details and shows only essential features. It allows users to interact with simple interfaces or methods without worrying about the underlying complexity, making code easier to use and maintain.
3. An abstract class can contain both fully implemented and abstract methods, and it allows for shared functionality among related classes. An interface, on the other hand, defines a contract of methods that must be implemented and is typically used for unrelated classes.
4. Encapsulation is demonstrated by declaring class fields as private and providing public getter and setter methods to access and modify the data, such as in a BankAccount class where balance is modified only through methods.

### 6.1.8 Conclusion

Understanding OOP is critical for writing clean, organized, and scalable Java applications. This unit has equipped you with the skills to create and manage classes, objects, and methods effectively while applying encapsulation and abstraction for better code design. These concepts form the core of modern software development and will serve as the foundation for more complex programming topics ahead. A constructor is a special method used to initialize objects of.

### ***Summary***

---

This unit introduced the foundational concepts of Object-Oriented Programming (OOP) in Java. It explained how OOP helps structure code around classes and objects, enabling modularity and reusability. Core principles covered include encapsulation, where data is hidden using private fields and accessed via public methods, and abstraction, which exposes essential features while hiding complex implementation. You also learned how to create objects, use constructors (default and parameterized), and apply abstraction through methods, abstract classes, and interfaces. These principles lay the groundwork for more advanced OOP concepts such as inheritance and polymorphism, to be covered later. A constructor is a special method used to initialize objects of.

## Self-Assessment Questions

- Which of these class types can be used to define method abstraction without providing implementation?  
A. Static class                      B. Concrete Class  
C. Abstract Class                      D. None of the above
- What does a user sees interacting with a class that uses abstraction properly?  
A. The internal logic of every method  
B. Only the public method signatures and behaviour  
C. The private variables and helper methods  
D. Bytecode and compiled binaries
- The state of an object (also known as its properties or attributes) is represented by \_\_\_\_\_ with their current value  
A. Data field                      B. method  
C. instance method                      D. void method
- A \_\_\_ serves as a template that defines the properties (data) and behaviors (methods) common to all objects of that type.  
A. Data field                      B. Class  
C. Field                      D. None of the above
- Which OOP concept involves bundling data and methods together, while restricting direct access to an object's internal state?  
A. Inheritance                      B. Polymorphism  
C. Encapsulation                      D. Abstraction
- Which programming concepts involves exposing only essential features of an object while hiding the complex implementation details?  
A. Inheritance                      B. Polymorphism  
C. Encapsulation                      D. Abstraction
- Given the name of a class is Book, what should be the name of its constructor?

- A. createBook()   B. book()   C. Book()   D. newBook()
8. Which of the following is true about a constructor?
- A. A constructor must always have a return type.
  - B. A constructor is called when an object is created.
  - C. A constructor can only be declared once per class.
  - D. A constructor cannot be overloaded.
9. Which of the following statements is true about interface?
- A. An interface can have both abstract and concrete methods.
  - B. An interface can have only concrete methods.
  - C. An interface can only have abstract methods.
  - D. An interface can have only constructors.
10. Which of the following allows you to define methods without providing their implementation?
- A. Concrete class                      B. Abstract class
  - B. Interfaces                              D. Final class
11. Which of the following can have default methods with implementation?
- A. Abstract class   B. Interface                      C. Enum              D. Package
12. Which of the following statements about an abstract class is false?
- A. It can have constructor              B. It can have private methods
  - C. It can have fields              D. It can be instantiated directly
13. What is the main difference between an abstract class and an interface?
- A. An abstract class can have method implementations, but an interface cannot.
  - B. A class can implement multiple interfaces, but only one abstract class.
  - C. An interface contains only method signatures, while an abstract class can have implementations.
  - D. Abstract classes and interfaces are different.

14. Which keyword is used to create a reference variable for an object in Java?  
A. New                      B. new C. ref                      D. Ref
15. What method is automatically called when an object is created?  
A. main    B. method()                      C. constructor()                      D. class
16. Abstraction focuses on:  
A. Speeding up code                      B. Reducing object size  
C. Hiding complex details                      D. Repeating code
17. Which of the following benefits does NOT come from encapsulation?  
A. Input validation                      B. Faster execution  
C. Data hiding                      D. Flexibility
18. Which method access level is typically used for abstraction?  
A. private                      B. public                      C. static                      D. final
19. Which is used to implement abstraction ins Java?  
A. Classes only                      B. Abstract classes and interfaces  
C. Methods only                      D. Constructors
20. Interfaces differ from abstract classes in that:  
A) They allow constructors                      B) They can be instantiated  
C) They support multiple inheritance  
D) They do not require Unit Activities

### ***Unit Activities***

---

#### **Activity 1: Create and Use Classes and Objects**

**Objective: To practice creating classes and objects and understand how to define attributes and methods**

**Instructions:**

1. Define a class named Book with attributes title, author, and a method displayInfo() that prints these details.
2. In the main() method, create at least two objects of the class and call the displayInfo() method.

**Tasks:**

1. Create the Book class with fields and method.
2. Instantiate two Book objects in the main() method.
3. Call and test the displayInfo() method.

**Activity 2: Implement Encapsulation**

**Objective: To demonstrate understanding of encapsulation through the use of private fields and public getters and setters.**

1. Create a class called Account with private fields: accountNumber and balance.
2. Write getBalance(), deposit(amount), and withdraw(amount) methods.
3. Ensure withdraw() does not allow balance to go negative.

**Task:**

1. Define private fields and public methods.
2. Use the main() method to test deposits and withdrawals.
3. Print balance before and after operations

**Activity 3: Constructor Practice (Default and Parameterized)**

**Objective: To understand how constructors are used to initialize objects.**

**Instructions:**

1. Create a Car class with attributes: model, color, and year.
2. Create both a default constructor and a parameterized constructor.
3. Instantiate objects using both constructors

**Tasks:**

1. Implement two constructors.
2. Create at least two Car objects in the main() method.
3. Print the values of each object

## **Activity 4: Apply Abstraction Using Methods and Classes**

**Objective: To understand and apply abstraction through method design and abstract classes.**

### **Instructions:**

1. Create an abstract class Appliance with an abstract method turnOn() and a concrete method plugIn().
2. Create subclasses TV and Fan that implement turnOn().
3. Use main() to demonstrate the behavior

### **Tasks:**

1. Define the abstract class and its method.
2. Implement subclasses with specific behaviors.
3. Demonstrate polymorphism in main() by calling methods on the Appliance reference

## **Activity 5: Interface Implementation**

**Objective: To demonstrate abstraction and behavior sharing using interfaces.**

### **Instructions:**

1. Create an interface Playable with a method play().
2. Create classes Piano and Guitar that implement Playable.
3. Use main() to create objects and call play()

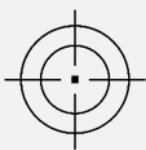
### **Tasks:**

1. Define the interface and implement it in both classes.
2. Instantiate each class in main() and call play() on each.

## Unit 2 — Object-Oriented Programming (OOP) Advanced

### INTRODUCTION

OOP is a powerful paradigm that models real-world entities using objects and classes. While the basics of OOP, such as classes, objects, and encapsulation, lay the foundation, this unit explores the advanced features, such as inheritance and polymorphism. In this unit, you will learn how to create class hierarchies using inheritance to promote code reuse and organisational structure. You will also learn how polymorphism enables flexible and dynamic method execution, making your code more scalable and maintainable.



#### LEARNING OUTCOMES

#### At the end of this unit, you should be able to:

- Explain and implement inheritance.
- Apply polymorphism to write flexible code.
- Differentiate between method overriding and overloading.
- Understand and use the super keyword for accessing a parent class member.

### 6.2.1 What is Inheritance?

Inheritance is a fundamental concept in Object-Oriented Programming (OOP) that allows a class (called the subclass, derived class, or child class) to inherit attributes (fields/variables) and behaviors (methods) from another class (called the superclass, base class, or parent class). This promotes code reuse, enhances scalability, and helps model real-world relationships in software design.

#### ***Benefits of Inheritance***

The following are the benefits of inheritance:

- i. Code Reusability: Shared methods and variables in the parent class are re-used in child classes.
- ii. Extensibility: You can build on existing functionality without modifying existing code.
- iii. Maintenance: Updates made to the parent class automatically apply to child classes.
- iv. Hierarchy Modelling: Helps to logically organise classes and represent real-world relationships.

The syntax of inheritance is as follows:

```
class Superclass {

 // fields and methods

}

class Subclass extends Superclass {

 // Additional fields and methods

}
```

### 6.2.2 Types of Inheritance

The following are the types of inheritance:

#### ***Single Inheritance***

A subclass inherits from one superclass. In Figure 6.17, the Dog class inherits eat() from the Animal class. This means any Dog object can call both the eat() and bark() methods.

```

class Animal {
 void eat() {
 System.out.println("Animal is eat...");
 }
}

class Dog extends Animal {
 void bark() {
 System.out.println("Dog is bark...");
 }
}

```

Figure 6.17: Single Inheritance

### ***Multilevel Inheritance***

A class is derived from a derived class. In Figure 6.18, Puppy class inherits features from the Dog class and by extension from the Animal class (the eat() method) without duplicating code.

```

class Animal {
 void eat() {
 System.out.println("Eating...");
 }
}

class Dog extends Animal { // Level 1
 void bark() {
 System.out.println("Barking...");
 }
}

class Puppy extends Dog { // Level 2
 void weep() {
 System.out.println("Weeping...");
 }
}

```

Figure 6.18: Multi-Level Inheritance

### ***Hierarchical Inheritance***

Multiple subclasses inherit from a single class. In Figure xxx, both Lion and Cat inherit eat() method from the Animal class.

```

class Animal {
 void eat() {
 System.out.println("Eating...");
 }
}

class Cat extends Animal {
 void meow() {
 System.out.println("Meowing...");
 }
}

class Lion extends Animal {
 void roar() {
 System.out.println("Roaring...");
 }
}

```

Figure 6.19: Hierarchical Inheritance

### 6.2.3 Using the Super Keyword

The super keyword is a special reference variable used within a subclass to refer directly to its immediate superclass. It plays a crucial role in inheritance, allowing a subclass to access members (variables and methods) and constructors of the superclass.

#### ***Calling the Superclass Constructor***

When a subclass extends a superclass, the constructor of the superclass must be called to initialise the inherited attributes. If the subclass constructor does not explicitly call a superclass constructor, Java automatically calls the no-argument constructor in the superclass.

#### ***Example 1: Default Behavior - No super() Needed***

If a superclass has a no-argument constructor (Figure 6.20), Java automatically calls it when creating a subclass object.

```

class Parent {
 Parent() {
 System.out.println("Parent constructor");
 }
}

class Child extends Parent {
 Child() {
 // Java automatically calls super() here
 System.out.println("Child constructor");
 }
}

public class Main {
 public static void main(String[] args) {
 Child c = new Child();
 }
}

```

**Figure 6.20: Default Behaviour of a Class**

The output of the code is:

Parent constructor

Child constructor

#### *Example 2: Using super() explicitly*

In Example (Figure 6.20), while the `super()` is implicitly called, it can also be called explicitly by adding `super()`.

#### *Example 3: Parent class has a parameterised constructor*

If the superclass does not have a no-arg constructor, the subclass must explicitly call a parameterised constructor using `super(arguments)`.

```

class Parent {
 Parent(String name) {
 System.out.println(
 "Parent constructor. Name: " + name);
 }
}

class Child extends Parent {
 Child(String name) {
 super(name); // Must call param. constructor
 System.out.println("Child constructor");
 }
}

public class Main {
 public static void main(String[] args) {
 new Child("Alice");
 }
}

```

**Figure 6.21: Parent and Child Classes**

## Accessing Superclass Methods

When a subclass overrides a method from its superclass, it provides its own version of that method. Sometimes, inside the subclass, you might still want to call the original (superclass) version of the method, for example:

- i. To reuse the behavior of the superclass method before or after adding subclass-specific behavior.
- ii. To avoid code duplication.
- iii. To extend the functionality of the superclass method.

The keyword `super` lets you do this

**`super.methodName();`**

This calls the superclass version of the method, even if it's overridden in the subclass.

### Examples 1: Calling Overridden Method

In Figure 6.22, `start()` is overridden in `Car`. Using `super.start()` allows the subclass to call the superclass version of the method.

```
class Vehicle {
 void start() {
 System.out.println("Vehicle is starting");
 }
}

class Car extends Vehicle {
 void start() {
 System.out.println("Car is starting");
 }

 void display() {
 start(); // Calls Car's start()
 super.start(); // Calls Vehicle's start()
 }
}

public class TestSuperMethod {
 public static void main(String[] args) {
 Car c = new Car();
 c.display();
 }
}
```

Figure 6.22: Calling Overridden Method

The output of the code is:

Car is starting

Vehicle is starting

### ***In-Text Questions (ITQs)***

---

1. What is inheritance in Object-Oriented Programming?
2. What are the main benefits of using inheritance?
3. What is the role of the super keyword in Java inheritance?

### ***In-Text Answers (ITAs)***

---

1. Inheritance is a feature in OOP where a class (called the subclass) inherits properties and methods from another class (called the superclass). This allows the subclass to reuse code from the superclass, promoting modularity and easier maintenance.
2. The main benefits include code reusability, extensibility, easier maintenance, and the ability to model real-world hierarchical relationships logically.
3. The super keyword allows a subclass to call its immediate superclass's constructors and methods. It is used to access or extend the functionality of the parent class, especially when constructors or methods are overridden

#### **6.2.4 What is Polymorphism**

Polymorphism is the ability of different objects to respond to the same method in different ways. It allows objects to be treated as instances of their parent class rather than their actual class. For instance, suppose we have the hierarchy, Animal (parent class) and a Dog class (child class) that extends Animal, we can write:

```
Animal myAnimal = new Dog()
```

Here, the actual object is a Dog, but the reference variable is of type Animal. This is useful because:

- I. The program treats myAnimal as an Animal (parent type).
- II. But at runtime, when you call a method that is overridden in Dog, the Dog's version of the method runs.
- III. This means you do not need to know the exact subclasses while writing the code. The correct method for the actual object type is called automatically.

### 6.2.5 Types of Polymorphism

There are two main types of polymorphism in Java:

- I. Compile-time (static) polymorphism
  - a. Achieved through method overloading
  - b. The method to be called is resolved during compilation
  - c. Methods have the same name but different parameter lists.
- II. Runtime (dynamic) polymorphism
  - a. Achieved through method overriding.
  - b. The method to be called is resolved at runtime.
  - c. A subclass provides a specific implementation of a method that is already defined in its superclass.

#### ***Compile-time Polymorphism (Method Overloading)***

Method overloading occurs when multiple methods in the same class share the same name but have different parameters (different types, number or both) (Figure 6.23).

```

class Calculator {
 int add(int a, int b) {
 return a + b;
 }

 double add(double a, double b) {
 return a + b;
 }

 int add(int a, int b, int c) {
 return a + b + c;
 }
}

public class Main {
 public static void main(String[] args) {
 Calculator calc = new Calculator();
 System.out.println(calc.add(5, 10));
 System.out.println(calc.add(5.5, 6.3));
 System.out.println(calc.add(1, 2, 3));
 }
}

```

Figure 6.23: Method Overloading Showing Polymorphism

In Figure xxx, the add method is overloaded three times with different parameter lists. The compiler determines which method to call based on the arguments.

### ***Runtime Polymorphism (Method Overriding)***

Method overriding happens when a subclass provides its own implementation of a method declared in its superclass. The method signature (name, return type, parameters) must be the same.

In the code snippet in Figure 6.24, Appliance is the superclass while WashingMachine and Microwave are subclasses that override the operate(). Although myAppliance is of type Appliance, it behaves differently at runtime depending on the object assigned to it.

```

class Appliance {
 void operate() {
 System.out.println("Appliance is operating");
 }
}

class WashingMachine extends Appliance {
 @Override
 void operate() {
 System.out.println("Washing Machine is washing");
 }
}

class Microwave extends Appliance {
 @Override
 void operate() {
 System.out.println("Microwave is working");
 }
}

public class Main {
 public static void main(String[] args) {
 Appliance myAppliance;

 myAppliance = new WashingMachine();
 myAppliance.operate();

 myAppliance = new Microwave();
 myAppliance.operate();

 myAppliance = new Appliance();
 myAppliance.operate();
 }
}

```

Figure 6.24: Method Overridden with Polymorphism

### 6.2.6 Polymorphism with Abstract Classes and Interfaces

Polymorphism is commonly used with abstract classes and interfaces, where methods are declared but not implemented in the parent type and implemented in subclasses.

```
interface Shape {
 void draw();
}

class Circle implements Shape {
 public void draw() {
 System.out.println("Draws Circle");
 }
}

class Rectangle implements Shape {
 public void draw() {
 System.out.println("Draws Rectangle");
 }
}

public class ShapeMain {
 public static void main(String[] args) {
 Shape shape;

 shape = new Circle();
 shape.draw();

 shape = new Rectangle();
 shape.draw();
 }
}
```

Figure 6.25: Polymorphism with Abstract Class and Interface

The output of the code in Figure 6.25, is as follows:

Draws Circle

Draws Rectangle

Vehicle is starting

### ***In-Text Questions (ITQs)***

---

1. How does polymorphism improve flexibility in Java programs?
2. What is the significance of abstract classes in polymorphism?
3. When should you use an interface instead of an abstract class?
4. Can a subclass override all the methods of a superclass?
5. What is the difference between method overloading and method overriding?

### ***In-Text Answers (ITAs)***

---

1. Polymorphism allows objects of different subclasses to be treated as instances of their superclass. This means the same method call can invoke different behaviors depending on the actual object type, making the code more flexible and scalable.
2. Abstract classes define methods that must be implemented by subclasses. This enforces a contract while allowing each subclass to define its own behavior, supporting polymorphism.
3. Use an interface when you want to define a contract that can be implemented by unrelated classes. Use an abstract class when classes share some common functionality and structure.
4. Yes, a subclass can override all non-final, non-static methods of its superclass. However, it's often useful to retain or extend some inherited behaviors using `super`.
5. Method overloading is compile-time polymorphism where multiple methods have the same name but different parameter lists within the same class. Method overriding is runtime polymorphism where a subclass provides its own implementation of a method already defined in its superclass.

#### **6.2.7 Conclusion**

Understanding inheritance and polymorphism is crucial for mastering Object-Oriented Programming. These concepts enhance code reuse, simplify maintenance, and allow for dynamic method behavior, making programs more flexible and easier to extend. The use of the `super` keyword helps maintain clear relationships between classes, while polymorphism enables writing generalized yet adaptable code. Mastery of these advanced OOP features prepares you to design robust and scalable applications that effectively model real-world scenarios.

### ***Summary***

---

This unit explores advanced Object-Oriented Programming concepts in Java, focusing on inheritance and polymorphism. Inheritance allows a class (subclass) to inherit attributes and methods from another class (superclass), promoting code reuse, ex-

tensibility, and hierarchical organization. You learned about different types of inheritance such as single, multilevel, and hierarchical inheritance. The `super` keyword was introduced as a way for subclasses to access superclass constructors and methods.

Polymorphism, the ability of different objects to respond uniquely to the same method call, was discussed in detail. Two types of polymorphism were explained: compile-time polymorphism through method overloading and runtime polymorphism through method overriding. This unit also highlighted the role of polymorphism when working with abstract classes and interfaces, enabling flexible and scalable code design.

### ***Self-Assessment Questions***

---

1. The class that inherits from another class is called:  
A. Superclass    B. Interface    C Subclass    D. Abstract class
2. What is polymorphism?  
A. A way to create multiple classes  
B. The ability of objects to take many forms  
C. A type of loop                      D. A method to hide data
3. Which of the following is a benefit of inheritance?  
A. Code duplication                      B. Code reusability  
C. Increased compilation time                      D Slower performance
4. What concept is demonstrated here?  

```
Animal a = new Dog();
a.sound(); // Calls Dog's sound()
```

  
A. Encapsulation                      B. Polymorphism  
C. Abstraction                      D. Inheritance
5. Method overriding occurs when:  
A. Methods share a name but have different parameters  
B. A subclass provides its own version of a method defined in the superclass  
C. A method is called from multiple classes

- D. Methods have different return types
6. What is dynamic method dispatch?
- A. The process of selecting which overloaded method to call
  - B. The process of selecting which overridden method to call at runtime
  - C. Creating objects dynamically
  - D. Dispatching methods statically
7. What is the output when `super.method()` is called inside an overridden method?
- A. It calls the subclass version of the method
  - B. It calls the superclass version of the method
  - C. It calls the method twice
  - D. It results in an error
8. What happens when you assign a subclass object to a superclass reference?
- A. Compile-time error
  - B. Polymorphic behavior is enabled
  - C. Runtime error
  - D. Object is converted to superclass
9. Which keyword prevents method overriding?
- A. static
  - B. final
  - C. private
  - D. abstract
10. The signature of a method includes:
- A. Method name and return type
  - B. Method name only
  - C. Method name and parameters
  - D. Return type and parameters
11. What keyword is used to inherit an interface in Java?
- A. extends
  - B. implements
  - C. inherits
  - D. interface
12. In Java, can a subclass access private members of the superclass directly?
- A. Yes
  - B. No
13. Which of the following is NOT true about method overloading?
- A. Methods must have different parameter lists

B. Methods can have different return types

C. Methods must have the same name

D. Methods must be in different classes

14. What does it mean to say Java supports polymorphism "by reference"?

B. The reference type decides method call

B. The actual object type decides method call

C. Both reference and object type are irrelevant

D. Polymorphism cannot happen by reference

15. Which feature of OOP allows hiding of data?

A. Inheritance

B. Encapsulation

C. Polymorphism

D. Abstraction

16. What is the output of the following code?

```
class Alpha {
 Alpha() { System.out.println("Alpha"); }
}

class Beta extends Alpha {
 Beta() { System.out.println("Beta"); }
}

public class Main {
 public static void main(String[] args) {
 Beta b = new Beta();
 }
}
```

A. Alpha

B. Alpha followed by Beta

C. Beta

D. Beta followed by Alpha

17. Can constructors be inherited?

A. Yes

B. Only if marked final

- C. No
  - D. Only in interfaces
- What if a subclass doesn't override an abstract method?
- A. It becomes abstract.
  - B. It inherits the superclass method
  - C. Compiler error
  - D. Runtime exception
- Can static methods be overridden?
- A. Yes
  - B. Only in interfaces
  - C. No
  - D. Only with @Override
- Which of these best describes an abstract class?
- A. A class that cannot be instantiated
  - B. A class without any methods
  - C. A class with no fields
  - D. A class that can inherit only one class
- What is the output of the following code?

```
class X {
 void print() { System.out.println("X"); }
}

class Y extends X {
 void print() { System.out.println("Y"); }
}

public class Main {
 public static void main(String[] args) {
 X x = new Y();
 x.print();
 }
}
```

- A. X                      B. Y                      C. Compiler error                      D. null

## **Unit Activities**

---

### **Activity 1: Exploring Inheritance**

**Objective: Understand and implement different types of inheritance in Java (single, multilevel, hierarchical)**

#### **Instructions:**

1. Create a base class Animal with a method eat() that prints: "This animal eats food."
2. Create a subclass Dog that extends Animal and includes a method bark().
3. Create a class Puppy that extends Dog and includes a method weep().
4. Create another subclass Cat that extends Animal and includes a method meow().
5. In the main() method, create objects of each class and call their methods.

#### **Task:**

Write the complete code for the classes. Annotate which inheritance type each example represents and explain how methods are inherited in your own words

### **Activity 2: Working with the super Keyword**

**Objective: Learn how to access parent class constructors and methods using super.**

#### **Instructions**

1. Create a class Person with a constructor that prints "Person constructor called." and a method displayInfo().
2. Create a class Student that extends Person. In its constructor, use super() and print "Student constructor called."
3. Override displayInfo() in Student and use super.displayInfo() within the method.
4. In the main() method, create a Student object and observe the constructor and method behavior

#### **Task:**

Write a report describing:

1. The role of super() in constructor chaining.
2. The effect of calling super.methodName() inside an overridden method.

### Activity 3: Method Overloading (Compile-Time Polymorphism)

**Objective:** Demonstrate compile-time polymorphism through method overloading.

**Instructions:**

1. Create a class MathOperations.
2. Define multiple multiply() methods with different parameter types:
  - a. multiply(int a, int b)
  - b. multiply(double a, double b)
  - c. multiply(int a, int b, int c)
3. In the main() method, call each version of multiply() and print the results.

**Tasks:**

Explain in comments or a brief write-up:

1. Why these methods are considered overloaded.
2. When Java decides which method version to use

### Activity 4: Method Overriding and Runtime Polymorphism

**Objective:** Understand and implement runtime polymorphism using method overriding.

**Instructions:**

1. Create a class Appliance with a method operate() that prints: "Appliance is operating." Create subclasses WashingMachine and Microwave, both overriding operate() to print class-specific messages.
2. In the main() method:
  - a. Declare a reference of type Appliance.
  - b. Assign it to a WashingMachine object and call operate().
  - c. Assign it to a Microwave object and call operate()

**Tasks:**

Write a reflection explaining how Java determines which method to execute at runtime, even when using a superclass reference.

### Activity 5: Abstraction Using Abstract Classes and Interfaces

**Objective:** Demonstrate abstraction using both abstract classes and interfaces

**Instructions:**

1. Create an interface Flyable with a method fly().
2. Create an abstract class Animal with:
  - a. A concrete method breathe() that prints "Animal is breathing."
  - b. An abstract method makeSound().
3. Create a class Bird that extends Animal and implements Flyable.
  - a. Implement fly() and makeSound().
4. In the main() method, create an object of Bird and call fly(), breathe(), and makeSound().

**Tasks:**

1. What is the difference between an abstract class and an interface?
2. Why is abstraction useful in object-oriented programming?

Module Seven

# Strings and String Processing

## Unit 1 — Data Structures and String Handling

### INTRODUCTION

Strings are one of the most commonly used data types in programming. They represent sequences of characters and are essential for storing and manipulating text data. This lesson covers string manipulation techniques in Java and explores classes like String, StringBuilder, and StringBuffer for efficient string processing.



#### LEARNING OUTCOMES

#### At the end of this unit, you should be able to:

- Understand the concept of strings in Java.
- Perform common string operations (concatenation, substring, searching, etc.).
- Differentiate between String, StringBuilder and StringBuffer.
- Use StringBuilder and StringBuffer for efficient string manipulation.

### 7.1.1 What is a String?

A String is an object representing a sequence of characters (chars values) enclosed within a double quotes. String is a final class in java.lang package.

#### ***Creating a String***

Strings can be created using:

- I. String Literals: Stored in String Pool for memory efficiency.
- II. The new Keyword: Creates a new object in Heap Memory.

Examples of String Creation using String Literals:

```
String s1 = "Hello";
```

```
String s2 = "Hello"; // Reuses the same object from String Pool
```

Examples of String Creation Using the new Keyword:

```
String s3 = new String("Hello");
```

```
String s4 = new String("Hello"); // Creates a new object
```

#### ***String Immutability***

Strings are immutable, meaning once created, their values cannot be changed. Any operation to modify a string will actually create a new String object. Example:

```
String str = "Java"
```

```
str.concat(" Programming") // Original 'str' remains "Java"
```

```
String newStr = str.concat(" Programming");
```

```
// newString = "Java Programming"
```

#### ***Common String Methods***

Table 7.1 shows some String methods and their descriptions

Tables 7.1: Common String Method

| Method                              | Description                               |
|-------------------------------------|-------------------------------------------|
| length()                            | Returns the length of the string          |
| charAt(int index)                   | Returns character at specified index      |
| substring(int start, int end)       | Returns a substring                       |
| toLowerCase() or toUpperCase()      | Converts string to lowercase or uppercase |
| trim()                              | Removes leading and trailing spaces       |
| equals(String str)                  | Compares string for exact match           |
| equalsIgnoreCase(String str)        | Compares string ignoring case             |
| indexOf(String str)                 | Finds the index of a substring            |
| replace(char oldChar, char newChar) | Replaces characters                       |
| contains(CharSequence s)            | Checks if sequence exist in string)       |
| split(String regex)                 | Splits string into array based on regex   |

### ***String Concatenation***

String concatenation can occur using the + operator or the concat() method as shown in the code snippets below.

#### ***Example 1: Using the + Operator***

```
String firstName = "John";

String lastName = "Doe";

String fullName = firstName + " " + lastName;

System.out.println(fullName); // Output: John Doe
```

#### ***Example 2: Using the concat() Method***

```
String result = "Hello".concat(" World");

System.out.println(result); // Output: Hello World
```

### ***Searching in Strings***

When working with strings, it is common to search for specific characters, substrings, or patterns within a larger string. Java provides several useful methods in the String class namely contains(), startsWith(), endsWith(), indexOf(), etc.

**Example 1: Searching in String Using startsWith()**

```
String str = "Data Structures";

System.out.println(str.startsWith("Data")); // Output: true

System.out.println(str.startsWith("Structures")); // Output: false

System.out.println(str.startsWith("Structures", 5)); // Output: true
```

The substring of str starting at index 5 is Structures and does Structures starts with Structures

**Example 2: Searching in String Using endsWith()**

```
String str = "Hello.java";

System.out.println(str.endsWith(".java")); // Output: true

System.out.println(str.endsWith("Java"));

// Output: false (case-sensitive)
```

**Example 3: Searching in String Using contains()**

```
String str = "Hello, World!";

System.out.println(str.contains("World")); // Output: true

System.out.println(str.contains("world"));

// Output: false (case-sensitive)
```

**Example 4: Searching in String Using indexOf()**

```
String str = "Hello, World!";

System.out.println(str.indexOf('o')); // Output: 4

System.out.println(str.indexOf('o', 5));

// Output: 8 (search starts from index 5)

System.out.println(str.indexOf("World")); // Output: 7

System.out.println(str.indexOf("world"));

// Output: -1 (case-sensitive)
```

### ***String Comparison***

We use `equals()` (case-sensitive) vs `equalsIgnoreCase()`. `compareTo()` is used for lexicographical comparison.

```
String a = "Java";

String b = "java";

System.out.println(a.equals(b));

System.out.println(a.equalsIgnoreCase(b));

System.out.println(a.compareTo(b));
```

The output is:

```
false

true

-32
```

In the comparison `a.compareTo(b)`, 'J' (Unicode value of 74) is lexicographically less than 'j' (Unicode value of 106)

### ***Extracting Substring***

We use `substring (int start)` and `substring (int start, int end)` to extract a portion of the string. Start is inclusive while end is exclusive. Given the code snippet below:

```
String str = "Covenant University";

System.out.println(str.substring(3));

System.out.println(str.substring(0, 4));
```

The output is as follows:

```
enant University

Cove
```

## **7.1.2 StringBuilder and StringBuffer**

The `String` class is immutable. Every modification creates a new `String` object, which can be inefficient when dealing with frequent or large-scale string operations. To overcome this, Java provides two mutable alternatives:

- I. `StringBuilder`
- II. `StringBuffer`

### ***StringBuilder***

`StringBuilder` is a mutable sequence of characters introduced in Java 5. It is not synchronized, meaning it is not thread-safe, but it offers better performance in single-threaded environments.

The following are common methods used with `StringBuilder`: `append(String str)`, `insert(int offset, String str)`, `replace(int start, int end, String str)`, `delete(int start, int end)`, `reverse()`, `toString()`, etc. Consider the code block below:

```
StringBuilder sb = new StringBuilder("Java");

sb.append(" Programming");

sb.insert(5, "Language ");

sb.replace(5, 13, "Core ");

sb.delete(5, 10);

sb.reverse();

System.out.println("Result: " + sb);
```

The output of the code is Result: gnimmargorP avaJ

### ***StringBuffer***

`StringBuffer` is similar to `StringBuilder` but is synchronized. This makes it suitable for multi-threaded environments where multiple threads may access the same string data. Methods are similar to those in `StringBuilder`.

### ***String vs StringBuilder vs StringBuffer***

The comparison in terms of mutability, thread-safe, performance and use cases are shown in Table 7.2.

Table 7.2: String vs StringBuilder vs StringBuffer

| Feature     | String                       | StringBuilder                         | StringBuffer                            |
|-------------|------------------------------|---------------------------------------|-----------------------------------------|
| Mutability  | Immutable                    | Mutable                               | Mutable                                 |
| Thread-safe | No                           | No                                    | Yes                                     |
| Performance | Slow (many modification)     | Fast (single-threaded)                | Slower (due to synchronization)         |
| Use Case    | When content does not change | For fast operations in single thread. | For fast operations in multi-threading. |

### 7.1.3 Conclusion

This unit introduced the essential concepts of string handling in Java, emphasizing that strings are immutable objects representing sequences of characters. You learned how to create strings using literals and the new keyword, and explored common operations such as substring extraction, searching, and comparison. The unit also highlighted the limitations of using immutable String objects for frequent modifications and presented StringBuilder and StringBuffer as efficient, mutable alternatives suited for single-threaded and multi-threaded environments respectively.

### Summary

---

Strings in Java are immutable sequences of characters that can be created via literals or the new keyword. Common string methods include `substring()`, `charAt()`, `split()`, and search functions like `contains()` and `indexOf()`, which enable effective manipulation and querying of text. For comparison, methods like `equals()` and `compareTo()` allow case-sensitive and lexicographical checks. To improve performance in scenarios involving multiple string modifications, `StringBuilder` (non-synchronized and faster) and `StringBuffer` (synchronized and thread-safe) provide mutable alternatives for efficient string processing depending on the application's concurrency needs. The comparison in terms of mutability, thread-safe.

## Self-Assessment Questions

---

1. What is a String in Java?
  - A. A primitive data type
  - B. An object representing a sequence of characters
  - C. A collection of integers
  - D. A type of array
2. How are string literals stored in Java?
  - A) In heap memory
  - B) In stack memory
  - C) In the String Pool
  - D) In the CPU cache
3. Which keyword creates a new String object?
  - A) create
  - B) new
  - C) String
  - D) object
4. What does it mean that strings are immutable in Java?
  - A) Strings can be changed anytime
  - B) Strings cannot be changed once created
  - C) Strings are primitive data types
  - D) Strings are mutable objects
5. What will be the output of this code?
 

```
String str = "Java";
str.concat("Programming");
System.out.println(str);
```

  - A) Java Programming
  - B) Java
  - C) Compilation error
  - D) NullPointerException
6. How can you get the character at index 2 in a string str?
  - A) str.get(2)
  - B) str.charAt(2)
  - C) str[2]
  - D) str.character(2)
7. Which method extracts a substring from a string?
  - A) substring(int start)
  - B) substring(int start, int end)
  - C) Both A and B
  - D) split()

8. What is the result of "Hello".toUpperCase()?  
A) "hello"      B) "HELLO"      C) "Hello"      D) Error
9. Which method compares two strings for exact equality?  
A) equals()      B) equalsIgnoreCase()  
C) compareTo()      D) contains()
10. What does indexOf() return if the substring is not found?  
A) 0      B) -1      C) null      D) Throws exception
11. Which class provides a mutable sequence of characters?  
A) String      B) StringBuilder  
C) StringBuffer      D) Both B and C
12. The difference between StringBuilder and StringBuffer is?  
A) StringBuilder is thread-safe      B) StringBuffer is thread-safe  
C) StringBuffer is faster than StringBuilder      D) No difference
13. Which is faster in a single-threaded environment?  
A) String      B) StringBuffer  
C) StringBuilder      D) StringTokenizer
14. What is printed by this code snippet?  

```
String str = "Covenant University";
System.out.println(str.substring(3, 9));
```

  
A. nant U      B. enant U      C. enant      D. nant
15. Which of the following methods can be used to concatenate two strings?  
A) + operator      B) concat() method  
C) append() method in StringBuilder      D) All of the above
16. Why is StringBuilder preferred over String for multiple modifications?  
A) StringBuilder is immutable  
B) StringBuilder creates fewer objects and is more efficient  
C) String is mutable      D) StringBuilder is slower

17. Which of the following is true about StringBuffer?

- A) Not synchronized                      B) Thread-safe
- C) Cannot be modified                      D) Immutable

18. Which method converts a StringBuilder to a String?

- A) toString()                      B) convert()                      C) string()
- D) toStringBuilder()

### ***Unit Activities***

---

#### **Activity 1: Creating and Understanding Strings**

**Objective: To learn how to create and differentiate between string literals and objects.**

##### **Instructions:**

Write a Java program that demonstrates the creation of strings using both string literals and the new keyword. Observe how Java handles them.

##### **Tasks:**

1. Create two string literals with the same value and print whether they refer to the same object using ==.
2. Create two string objects using new String() with the same value and compare them with == and .equals().

#### **Activity 2: String Method Exploration**

**Objective: To practice basic string manipulation methods.**

##### **Instructions**

Use different String class methods to analyze and manipulate text.

##### **Task:**

Given the string: String text = " Java Programming Language ";

1. Print the length of the string.
2. Trim the string and print the result.
3. Convert it to uppercase and lowercase.

4. Extract the word "Programming" using `substring()`.
5. Replace "Java" with "C++".

### **Activity 3: Searching Within Strings**

**Objective: To explore string search operations**

**Instructions:**

Use methods like `contains()`, `startsWith()`, `endsWith()`, and `indexOf()` to perform search operations on a string.

**Tasks:**

Given String `sentence = "Welcome to Covenant University!"`;

- a. Check if the string starts with "Welcome".
- b. Check if it ends with "!".
- c. Find the index of the word "Covenant".
- d. Check if it contains the word "University"

### **Activity 4: Comparing Strings**

**Objective: To understand string comparison using various methods.**

**Instructions:**

Write code to compare two strings using `equals()`, `equalsIgnoreCase()`, and `compareTo()`.

**Tasks:**

1. Declare String `a = "Java"`; and String `b = "java"`;
2. Compare `a` and `b` using `equals()`.
3. Compare them using `equalsIgnoreCase()`.
4. Compare them using `compareTo()` and interpret the result.

### **Activity 5: Use `StringBuilder` and `StringBuffer`**

**Objective: To demonstrate efficiency and usage of mutable string classes.**

**Instructions:**

Use `StringBuilder` and `StringBuffer` to manipulate strings and observe differences.

**Tasks:**

1. Create a `StringBuilder` object with "Data". Append " Structures", insert " & Algorithms", and reverse the result.
2. Repeat the same using `StringBuffer`.
3. Print and compare the outputs of both.

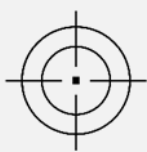
Module Eight

# **Arrays and Arrays Manipulation**

## Unit 1 — Arrays and Arrays Manipulation

### INTRODUCTION

In this unit, we explore one of the most essential data structures in structures in programming: arrays. Arrays allow you to store and manage collections of data efficiently using a single variable. You will learn how to declare, initialize, and access elements in both one-dimensional and multi-dimensional arrays. As we dive deeper, you'll see how arrays help organize data, simplify repetitive tasks, and support more complex operations. We will also cover common array manipulation techniques such as sorting and searching, giving you practical tools to process and analyze data effectively.



#### LEARNING OUTCOMES

#### At the end of this unit, you should be able to:

- Define arrays and explain their uses.
- Declare and manipulate different dimensions of arrays.
- Use array methods for sorting and searching data
- Carry out various array operations.

### 8.1.1 What is an Array?

An array is a data structure used to store multiple values of the same data type in a single variable. Arrays are useful when you need to store a collection of data and perform operations on it, such as sorting, searching, and updating values

### 8.1.2 One-Dimensional Array

A one-dimensional (1D) array in Java is a linear collection of elements of the same data type, stored in contiguous memory locations and accessed using an index

#### ***1D Array Declaration and Initialization***

The syntax for declaring an array is as follows:

```
dataType[] arrayName;
```

The syntax for initializing an array is as follows:

```
arrayName = new int[size];
```

The following code block demonstrate the array declaration, allocation and value assignment, each on separate lines:

```
int[] numbers; // Array Declaration
```

```
numbers = new int[3]; // Array Allocation
```

```
numbers[0] = 10; // Assigning value
```

```
numbers[1] = 45; // Assigning value
```

```
numbers[2] = -2; // Assigning value
```

The syntax for both array declaration and initialization is:

```
dataType[] arrayName = {<Elements separated by commas>}
```

For example

```
String [] names = {"James", "Charles", "Alice"};
```

## Accessing, Modifying and Sorting Elements

Figure 8.1 shows array declaration, modification, and sorting.

```
import java.util.Arrays;

public class ArrayManipulation {
 public static void main(String[] args) {
 // Declaration and Initialization
 int[] values = {10, 20, 30, 40, 50};

 // 1. Accessing elements
 System.out.println("Element at index 2: " + values[2]);
 // Output: 30

 // 2. Modifying an element
 values[1] = 25; // Change the value at index 1
 System.out.println("Updated value at index 1: " + values[1]);
 // Output: 25

 // 3. Sorting the array
 Arrays.sort(values); // Sorts the array in ascending order
 System.out.println("Sorted array: " + Arrays.toString(values));
 // Output: [10, 20, 25, 30, 40]
 }
}
```

Figure 8.1: Array Manipulations

The output of the code in Figure 8.1 is as follows:

Element at index 2: 30

Updated value at index 1: 25

Sorted array: [10, 25, 30, 40, 50]

### 8.1.3 Multi-Dimensional Array

Arrays of arrays. For instance, a two-dimensional (2-D) array for matrices, grids, or multiplication tables.

#### 2D Array Declaration and Initialization

The syntax for a 2D array is as follows:

```
dataType[][] arrayName;
```

For example:

```
int[][] numbers;
```

The syntax for allocating memory to a 2D array is as follows:

```
arrayName = new dataType[rows][columns];
```

**Example 1: 2D Array Declaration, Allocation and Assignment**

Figure 8.2 demonstrates how to declare a 2D array, allocate memory, and assign values to its elements, on separate lines:

```
import java.util.Arrays;

public class TwoDArrayOne {
 public static void main(String[] args) {
 int[][] nums; // Array Declaration
 // Array Allocation (3 by 2)
 nums = new int[3][2];

 // Assigning values to the array
 nums[0][0] = 10; // Row 0, Column 0
 nums[0][1] = 20; // Row 0, Column 1
 nums[1][0] = 30; // Row 1, Column 0
 nums[1][1] = 40; // Row 1, Column 1
 nums[2][0] = 50; // Row 2, Column 0
 nums[2][1] = 60; // Row 2, Column 1

 System.out.println(Arrays.deepToString(nums));
 }
}
```

Figure 8.2: Array Declaration Allocation and Assignment

The output of the code is as follows:

```
[[10, 20], [30, 40], [50, 60]]
```

#### 8.1.4 Array Methods and Operations in Java

Arrays support various operations through:

- I. Built-in methods from Arrays class.
- II. Manual implementation of algorithms
- III. Utility methods for common tasks.

##### **Sorting Arrays**

Sorting is the process of arranging the elements in a specific order. The built-in method for sorting array is `Arrays.sort()`

### Example 1:

In Figure 8.3 the array {5, 2, 9, 1, 5, 6} and sub-array {12, 7, 1} are sorted using `Arrays.sort()`

```
import java.util.Arrays;

public class ArraySort {
 public static void main(String[] args) {
 int[] numbers = {5, 2, 9, 1, 5, 6};

 // Sort in Ascending Order
 Arrays.sort(numbers);
 System.out.println(Arrays.toString(numbers));

 // Sort a SubArray
 int[] partialSort = {4, 12, 7, 1, 2};
 Arrays.sort(partialSort, 1, 4);
 System.out.println(Arrays.toString(partialSort));
 }
}
```

Figure 8.3: Sorting an Array and Sub-array Using `Arrays.sort()`

### Simple Search

Searching refers to the process of finding a specific in an array. Simple search that checks each element sequentially. In a simple linear search, you iterate through an array checking each element until you find the target value.

#### Example 1: Linear Search in a One-Dimensional Array

This code (Figure 8.4) performs a linear search to find the target (40) in the array and prints its index or a not-found message.

```

public class LinearSearch {
 public static void main(String[] args) {
 int[] numbers = {50, 20, 10, 40, 30};
 int target = 40;

 boolean found = false;
 for(int i = 0; i < numbers.length; i++) {
 if(numbers[i] == target) {
 found = true;
 System.out.println(
 "Element's index: " + i);
 break;
 }
 }

 if (!found) {
 System.out.println("Element not found");
 }
 }
}

```

Figure 8.4: Linear Search

The output of the code is:

Element's index: 3

### 8.1.5 Common Array Operations

Here are some common arrays operations:

#### ***Length of an Array***

To get the length of an array, we use the length property

```
int[] values = {5, 4, 34, -3, 0, 12, 0};
```

```
System.out.print(values.length);
```

The length of the array, values is 7.

#### ***Sorting an Array***

Java provides built-in Arrays.sort method for sorting an array.

```
int[] arr = {40, 10, 50, 20, 30};
```

```
Arrays.sort(arr);
```

```
System.out.println("Sorted Array: " + Arrays.toString(arr));
```

The code snippet will output a sorted array as shown below:

Sorted Array: [10, 20, 30, 40, 50]

### ***Accessing an Element from an Array***

To access an element, you can use the index:

```
int[] arr = {10, 20, 30, 40, 50};
```

System.out.println("Element at index 2: " + arr[2]); // Output: The output is as follows:

Element at index 2: 30

### ***Copying an Array***

Copying an array means creating a new array that contains the same elements as an existing array (as seen in Figure 8.5).

```
import java.util.Arrays;

public class ArrayOps {
 public static void main(String[] args) {
 // Original Array
 int[] original = {1, 2, 3, 4, 5};

 // Using Arrays.copyOf()
 int[] c1 = Arrays.copyOf(original, original.length);
 System.out.println("Using Arrays.copyOf: "
 + Arrays.toString(c1));

 // Using System.arraycopy()
 int[] c2 = new int[original.length];
 System.arraycopy(original, 0, c2, 0, original.length);
 System.out.println("Using System.arraycopy(): "
 + Arrays.toString(c2));

 // Using .clone()
 int[] c3 = original.clone();
 System.out.println("Using .clone: "
 + Arrays.toString(c3));
 }
}
```

Figure 8.5: Copying Array

The output of the code is:

Using Arrays.copyOf: [1, 2, 3, 4, 5]

Using System.arraycopy(): [1, 2, 3, 4, 5]

Using .clone: [1, 2, 3, 4, 5]

### ***Comparing Arrays***

Given the arrays arr1 and arr2 as shown below, the boolean operation below will return true

```
import java.util.Arrays;

public class ArraysMan {

 public static void main(String[] args) {

 int[] arr1 = {1, 2, 3};

 int[] arr2 = {1, 2, 3};

 boolean isEqual = Arrays.equals(arr1, arr2);

 System.out.println(isEqual);

 }

}
```

### ***Converting Arrays to String***

To convert array to string we use the Arrays.toString()

```
import java.util.Arrays;

public class ArraysMan {

 public static void main(String[] args) {

 int[] numbers = {10, 20, 30, 40, 50};

 System.out.print(Arrays.toString(numbers));

 }

}
```

The output of the code is [10, 20, 30, 40, 50]

### ***In-Text Questions (ITQs)***

---

1. What is the output of `System.out.println(values.length);` if `values` is defined as `int[] values = {5, 4, 3, 2, 1};`?
2. How do you declare and initialize a one-dimensional array in Java?
3. What is the syntax to declare and allocate a 2D array of size 3x2?
4. How do you perform a linear search in an array?
5. How do you convert an array to a string for easy printing?

### ***In-Text Answers (ITAs)***

---

1. The output is 5, which is the number of elements in the array. What is an array in programming?
2. `int[] numbers = {10, 20, 30};`
3. `int[][] matrix = new int[3][2];`
4. A linear search involves iterating through each element of the array until the target value is found.
5. Use `Arrays.toString(arrayName)` to get a string representation of the array.

#### **8.1.6 Conclusion**

In this unit, we explored arrays as one of the fundamental data structures in programming. You learned how to declare, initialise, and manipulate both one-dimensional and multidimensional arrays in Java. Arrays provide an efficient way to store and manage large volumes of data using a single variable name. We also covered important operations such as sorting, searching, updating, and comparing arrays. Mastering these concepts will help you write cleaner, more efficient code when dealing with structured data collections.

## Summary

---

In this unit, you learned that arrays are essential data structures used to store multiple values of the same data type under a single variable name. Java supports both one-dimensional (1D) and two-dimensional (2D) arrays, allowing for the organization of data in linear and tabular formats. Arrays are zero-indexed, meaning the first element is accessed at index 0. You explored how to declare, initialize, access, and modify array elements. Common operations such as sorting with `Arrays.sort()`, searching using linear search techniques, and comparing arrays with `Arrays.equals()` were also covered. Additionally, the unit highlighted array cloning using `.clone()` and converting arrays to strings using `Arrays.toString()`. Understanding and applying these array manipulation techniques will significantly enhance your ability to manage and process data efficiently in Java programming.

## Self-Assessment Questions

---

1. What is an array in programming?
  - A. A single value holder
  - B. A data type
  - C. A data structure that holds values of different types
  - D. A data structure that hold values of the same type.
2. Which of the following best describes arrays?
  - A. Dynamic in size
  - B. Immutable
  - C. Fixed-size, homogenous data structure
  - D. Only used for numbers
3. Which Java keyword is used to create a new array?
  - A. create
  - B. make
  - C. new
  - D. init
4. Which of the following correctly declares an integer array?
  - A. `int array[ ];`
  - B. `array int[ ];`
  - C. `int[ ] array;`
  - D. `init array`
5. How do you initialize an array of size 3?

- A. `int [ ] arr = int[3];`                      B. `int arr = new int [3];`  
 C. `int [ ] arr = new int[3];`                      D. `int arr[ ] = 3;`
6. What is the output of the `System.out.println(values[2]);` for `int [ ] values = {10, 20, 30, 40};`?
- A. 10                      B. 20                      C. 30                      D. 40
7. What is the index of the fifth element of the array?
- A. 4                      B. 5                      C. 6                      D. 0
8. How do you change the value at index 1 of an array to 25.
- A. `numbers[1] = 25;`                      B. `numbers[1] == 25;`  
 C. `Numbers[0] = 25;`                      D. `numbers [0] == 25;`
9. What will `System.out.println(numbers.length);` print for `int [ ] numbers = new int[4];`?
- A. 0                      B. 3                      C. 4                      D. Error
10. What is the default value of an integer array element?
- A. undefined      B. 0                      C. -1                      D. undefined
11. Which of the following declares a 2D integer array?
- A. `Int[ ][ ] matrix`                      B. `int matrix[ ][ ]`  
 C. Both A and B                      D. `int [2][2] matrix`
12. How do you allocate memory for a 2D array with 3 rows and 2 columns?
- A. `new int[2][3];`                      B. `new int[3][2];`  
 C. `int[3][2]`                      D. `allocate[3][2]`
13. How do you access the second element in the third row in `int [ ][ ] matrix`?
- A. `matrix [3][2]`                      B. `matrix[2][1]`  
 C. `Matrix [2][2]`                      D. `matrix [1][2]`
14. Which method is used to sort arrays in Java?
15. `sortArray()`                      B. `Arrays.sort()`  
 C. `sort()`                      D. `arraySort()`

16. What is the output of the following array: [[10, 20], [30, 40], [50, 60]] at matrix[1][1]?  
 A. 30                                      B. 40                                      C. 50                                      D. 20
17. What is the output after sorting {5, 2, 9, 1, 5, 6}?  
 A. {9, 6, 5, 5, 2, 1}                                      B. {1, 2, 5, 5, 6, 9}  
 C. {5, 5, 6, 9, 1, 2}                                      D. {2, 5, 5, 6, 9, 1}
18. Which method is used for linear search in an array?  
 A. Arrays.find()                                      B. Arrays.search()  
 C. Iterate manually                                      D. arrayLocate()
19. What happens when you try to access index 10 of an array with length 5?  
 A. Returns 0                                      B. Returns null  
 C. Runtime error                                      D. Compiles successfully
20. How do you get the length of an array arr?  
 A. arr.size()                                      B. arr.getLength()                                      C. arr.length                                      D. arr.length()
21. Which Java class provides utility methods for arrays?  
 A. ArrayList                                      B. Arrays                                      C. Collections                                      D. ArrayHelper
22. Which method is used to convert an array to a string?  
 A. Arrays.toString()                                      B. array.toString()  
 C. convertToString()                                      D. Arrays.toStr()
22. How do you compare two arrays for equality?  
 A. arr1 == arr2                                      B. Arrays.equal(arr1, arr2)  
 C. Arrays.equals(arr1, arr2)                                      D. arr1.equals(arr2)
23. What is the output of comparing {1,2,3} and {1,2,3} using Arrays.equals()?  
 A. false                                      B. true                                      C. equal                                      D. error
24. Which of the following creates a copy of an array?  
 A. arr.copy()                                      B. Arrays.copyOf()  
 C. arr.clone()                                      D. Both B and C

25. What does `System.out.println(arr[2]);` print for `int[] arr = {10, 20, 30, 40};`?
- A. 10                      B. 20                      C. 30                      D. 40
26. What is the result of: `Arrays.toString(new int[]{1,2,3})`?
- A. 123                      B. {1, 2, 3}                      C. [1, 2, 3]                      D. Error
27. This will throw an `ArrayIndexOutOfBoundsException`?
- A. `arr[0]`    B. `arr[arr.length]`    C. `arr[arr.length - 1]`    D. `arr[1]`
28. Which method is NOT used to copy arrays?
- A. `System.arraycopy()`                      B. `Arrays.copyOf()`
- C. `.clone()`                                      D. `Arrays.match()`
29. What does `int [ ] arr = {10, 20, 30}; System.out.println(arr.length);` output?
- A. 2                      B. 3                      C. 30                      D. Error
30. Which of these methods is used to copy array content efficiently in Java?
- A. `Arrays.copyArray()`                      B. `System.arraycopy()`
- C. `Arrays.duplicate()`                      D. `clone.array()`
31. What is the main advantage of using arrays?
- A. They are dynamic    B. They allow multiple data types
- C. They simplify repetitive operations
- D. They are always sorted
32. Which of the following statements is false?
- A. Array index starts at 0                      B. Arrays have a fixed length
- C. Arrays can contain mixed data types
- D. Arrays are contiguous blocks in memory
33. Which built-in class must be imported to use array utility functions in Java?
- A. `java.array.*`                      B. `java.util.Arrays`
- C. `java.lang.Array`                      D. `java.arrays.Utils`

## **Unit Activities**

---

### **Activity 1: Declare and Initialize Arrays**

**Objective:** To understand how to declare, allocate, and initialize one-dimensional and two-dimensional arrays.

#### **Instructions:**

Work in pairs. Each person writes code to create 1D and 2D arrays. Compare and discuss your outputs.

#### **Task:**

1. Declare and initialize a one-dimensional array with the values {10, 20, 30, 40, 50}.
2. Print the third element (index 2) of the array.
3. Declare and initialize a 3×2 two-dimensional array with any integer values.
4. Print the contents of the 2D array using nested loops.

### **Activity 2: Perform Common Array Operations**

**Objective:** To practice accessing, modifying, and sorting array elements.

#### **Instructions**

Use built-in and manual methods to manipulate array data.

#### **Task:**

1. Given the array `int[] numbers = {5, 15, 25, 35, 45};`
2. Change the value at index 1 to 20.
3. Sort the array using `Arrays.sort()`.
4. Print the modified and sorted array using `Arrays.toString()`.

### **Activity 3: Search an Array**

**Objective:** To implement a linear search algorithm to find specific values in an array.

#### **Instructions:**

You will write a method to perform a linear search and return the index of a target value.

**Tasks:**

1. Create an array `int[] data = {23, 56, 12, 78, 45};`
2. Write code to search for the value 78 in the array using a for loop.
3. Print the index if found, otherwise print "Not found"

**Activity 4: Compare and Clone Arrays**

**Objective: To compare two arrays and clone an array using built-in methods**

**Instructions:**

Use Java's Arrays utility class to perform the comparison and cloning operations.

**Tasks:**

1. Create two arrays: `int[] a = {1, 2, 3}` and `int[] b = {1, 2, 3}`
2. Compare them using `Arrays.equals()` and print the result.
3. Clone array a using `.clone()` and print the cloned array.

**Activity 5: Work with Multi-Dimensional Arrays**

**Objective: To enhance understanding of 2D array allocation and manipulation**

**Instructions:**

Manually fill in and print a 2D array using nested loops.

**Tasks:**

4. Declare a 3x2 2D array of integers.
5. Populate the array with any six values.
6. Print each element with its row and column index (e.g., Element at `[0][1]`: 25).

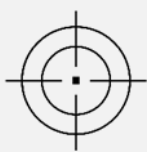
Module Nine

# **Advanced Programming Techniques**

## Unit 1 — Introduction to Recursion

### INTRODUCTION

In this unit, we explore the concept of recursion, a technique in programming where a method calls itself to solve smaller instances of a problem. Recursion is widely used for tasks like traversing data structures (e.g., trees and graphs), solving mathematical problems, and breaking down complex problems into simpler ones. Understanding recursion often leads to more elegant and concise solutions compared to iterative methods. We will also cover the key components of recursion, such as the base case (the condition that stops the recursion) and the recursive case (where the function calls itself with modified parameters). Mastering recursion will enable you to solve complex problems more efficiently.



#### LEARNING OUTCOMES

#### At the end of this unit, you should be able to:

- Understand the concept of recursion
- Write recursive methods to solve problems.
- Identify base cases and recursive cases.
- Recognize problems where recursion is advantageous

### 9.1.1 Overview of Recursion

Recursion is a programming technique in which a method calls itself in order to solve a problem. In recursion, a problem is broken down into smaller sub-problems of the same type. These sub-problems are solved independently until they reach a base case, which stops further recursive calls.

### 9.1.2 Key Concepts of Recursion?

Base case and recursive case are two key concepts of recursion we must understand to proceed in this unit.

#### **Base Case**

The base case is a condition in a recursive function that stops the recursion. Without a base case, the function would continue calling itself indefinitely, leading to a stack overflow error.

#### **Recursive Case**

The recursive case involves the function calling itself, but with a reduced or simpler version of the original problem. The recursive case should always move closer to the base case to avoid infinite recursion.

#### **Example: Factorial Calculation**

```
public class Factorial {
 // Recursive Function
 public static int factorial(int n) {
 // Base Case
 if (n == 0 || n == 1) return 1;
 // Recursive Case
 return n * factorial(n - 1);
 }

 public static void main(String[] args) {
 System.out.println(factorial(6));
 }
}
```

Figure 9.1: Illustrating Recursion Using Factorial

In Figure 9.1, the output is 720. Note, the if statement used is the short-hand version of the if statement.

### 9.1.3 How does Recursion Work

#### ***Call Stack Visualisation***

Each recursive call adds a new frame to the call stack until the base case is reached.

Example: factorial(3).

```

3 * factorial(2)
 2 * factorial(1)
 1 (base case reached)
 2 * 1 = 2
3 * 2 = 6

```

To find factorial(3), the method calls factorial(2), factorial(1) and finally factorial(0). When the base case is reached (factorial(0)), the function starts returning values back up the call stack.

```

factorial(3) => 3 * factorial(2)
factorial(2) => 2 * factorial(1)
factorial(1) => 1 * factorial(0)
factorial(0) => returns 1 (base case)

```

After getting to the base case, values are multiplied back up:

```

factorial(1) = 1 * 1 = 1
factorial(2) = 2 * 1 = 2
factorial(3) = 3 * 2 = 6

```

#### ***Call Stack Code Simulation***

```

public class RecursionStack {

 public static void recursiveMethod(int n) {

```

```
 if (n <= 0) {
 System.out.println("Base Case");
 return;
 }

 System.out.println("Recursive Method");
 recursiveMethod(n - 1);
 }

 public static void main(String[] args) {
 recursiveMethod(3);
 }
}
```

The above code will output:

```
Recursive Method
Recursive Method
Recursive Method
Base Case
```

#### 9.1.4 Common Recursive Algorithms/Examples

##### ***Fibonacci Sequence***

The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones, typically starting with 0 and 1. The sequence goes: 0, 1, 1, 2, 3, 5, 8,...

The formula to find the nth Fibonacci number is:

$$f(n) = f(n - 1) + f(n - 2)$$

Base Cases: if  $n == 2$ , return 0; if  $n == 1$ , return 1;

Recursive Case: return  $f(n - 1) + f(n - 2)$

```

public class Fibonacci {
 public static int fib (int n) {
 if (n == 0) return 0;
 if (n == 1) return 1;

 return fib(n - 1) + fib(n - 2);
 }

 public static void main(String[] args) {
 int n = 6;
 System.out.print(
 "Fibonacci of " + n + ": " + fib(6));
 }
}

```

Figure 9.2: Fibonacci Value of 6

The output is as follows: Fibonacci of 6: 8

### ***Power of a Number (Exponentiation)***

Evaluate  $x^n$  ( $x$  raised to  $n$ ) using recursion. The formula is

$$x^n = x + x^{n-1}$$

```

public class Exponent {
 public static int power(int x, int n) {
 // Base Case
 if (n == 0) return 1;

 // Recursive Case
 return x * power(x, n - 1);
 }

 public static void main(String[] args) {
 int x = 2, n = 3;
 System.out.print(x + " exp " + n + " = ");
 System.out.println(power(x, n));
 }
}

```

Figure 9.2: Power of a Number Using Recursion

### ***Sum of Natural Numbers***

The sum of the first  $n$  numbers can be calculated recursive using the formula:

$$\text{sum}(n) = n + \text{sum}(n - 1)$$

```

public class SumNaturalNumbers {
 public static int sum(int n) {
 // Base Case (n = 0)
 if (n == 0) return 0;
 //Recursive Case
 return n + sum(n - 1);
 }

 public static void main(String[] args) {
 int n = 7;
 System.out.print(
 "Sum of first " + n + " numbers: " + sum(n));
 }
}

```

Figure 9.3 Sum of Natural Numbers Using Recursion

The output of the code is: Sum of first 6 numbers: 21

### ***Sum of Digits***

To find the sum of the digits of a number, we recursively extract the last digit and then add it to the sum of the remaining digits. The formula to achieve is as follows:

$\text{sumOfDigits}(n) = n \% 10 + \text{sumOfDigits}(n/10);$

```

public class SumOfDigits {
 public static int sumOfDigits(int n) {
 // Base Case
 if (n == 0) return 0;
 // Recursive Case
 return n % 10 + sumOfDigits(n / 10);
 }

 public static void main(String[] args) {
 int n = 34501;
 System.out.print(
 "Sum of digits in " + n + ": ");
 System.out.println(sumOfDigits(n));
 }
}

```

Figure 9.4 Sum of Digits Using Recursion

The output of the code is: Sum of digits in 34501: 13

### ***Greatest Common Divisor (GCD)***

The GCD of integers can be calculated by using the Euclidean Algorithm (Figure 9.5). Given two integers, a and b (where  $a \geq b$ ), then:

$$\text{GCD}(a, b) = \text{GCD}(b, a \% b)$$

You keep replacing the bigger number with the remainder ( $a \% b$ ), and repeat this until the remainder is 0. At that point, the other number is the GCD

```
public class GCD {
 public static int gcd(int a, int b) {
 // Base Case
 if (b == 0) return a;

 // Recursive Case
 return gcd(b, a % b);
 }
 public static void main(String[] args) {
 int a = 48, b = 18;
 System.out.print(
 "GCD of " + a + " and " + b + ": ");
 System.out.println(gcd(a, b));
 }
}
```

Figure 9.5: GCD using Recursion

The output of the code is: GCD of 48 and 18: 6

### 9.1.5 Recursion vs Iteration

#### ***When to Use Recursion***

- I. Recursion is ideal for problems that naturally break down into subprograms, such as tree traversal, factorial, Fibonacci series, etc.
- II. Recursion tends to have cleaner, more readable code for certain problems.

#### ***When to Avoid Recursion***

- I. Recursion can lead to stack overflow if the recursion depth is too high (e.g. calculating Fibonacci number for large number).
- II. Iterative solutions may be more efficient in some cases, especially when working with large datasets.

### ***In-Text Questions (ITQs)***

---

1. What is recursion in programming?
2. What is the purpose of the base case in a recursive function?
3. How does the recursive case differ from the base case?
4. How does recursion use the call stack to solve problems?

5. What are some disadvantages of using recursion?
6. How does the Euclidean algorithm use recursion to find the GCD?

### ***In-Text Answers (ITAs)***

---

1. Recursion is a programming technique where a method calls itself to solve smaller instances of a problem until it reaches a base case that stops the recursion.
2. The base case is a stopping condition that prevents infinite recursion by terminating the recursive calls once a simple, solvable instance of the problem is reached.
3. The recursive case is where the function calls itself with a modified argument, progressively moving toward the base case, whereas the base case stops the recursion.
4. Each recursive call adds a new frame to the call stack. When the base case is reached, the calls return values back down the stack, resolving the problem step-by-step.
5. Recursion can lead to high memory usage and stack overflow errors if the recursion depth is too large. Iterative solutions may sometimes be more efficient.
6. The algorithm repeatedly replaces the larger number with the remainder of the division of the two numbers until the remainder is zero. The other number at that point is the GCD, implemented recursively as  $\text{GCD}(a, b) = \text{GCD}(b, a \% b)$ .

#### **9.1.6 Conclusion**

In conclusion, recursion is a powerful and elegant tool in a programmer's arsenal for solving complex problems that exhibit repetitive or nested structure. By mastering the concept of base and recursive cases and understanding how the call stack manages recursive calls, students can apply recursion to a wide range of real-world programming challenges. However, developers must also recognise the limitations of recursion and know when iterative solutions may be more appropriate. With a firm grasp of recursion, you are well-equipped to write clearer, more concise code and tackle problems involving repetition, hierarchy, and mathematical computations more effectively.

## Summary

---

This unit introduced recursion, a fundamental programming technique in which a method calls itself to solve smaller sub-problems of the original task. Recursion is especially useful for problems that can be naturally divided into similar sub-problems, such as computing factorials, generating Fibonacci numbers, calculating powers, summing digits, or determining the greatest common divisor (GCD). The two essential components of recursion, the base case and the recursive case, were clearly explained, as they form the control structure that prevents infinite loops. The unit also demonstrated how recursion works through the call stack, and how the stack unwinds as each recursive call returns a result. Real-life examples helped illustrate the concept, including factorials, Fibonacci numbers, and number summation. The advantages of recursion in simplifying code for problems involving hierarchical or repeated substructures were contrasted with the potential performance issues, such as stack overflow in deeply recursive calls. Overall, recursion is a powerful technique that can make code more elegant and efficient when used correctly.

## Self-Assessment Questions

---

1. What is recursion in programming?
  - A. Infinite loop
  - B. One method calling another
  - C. A method
  - D. A method calling itself
2. What ensures that a recursive method to terminate?
  - A. Infinite loop
  - B. Return statement
  - C. Base case
  - D. Print statement
3. What is the purpose of the recursive case in recursion?
  - A. Ends the recursion
  - B. Reduces the problem size
  - C. Initializes variables
  - D. Creates an infinite loop
4. What happens if a recursive function lacks base case?
  - A. Runs once
  - B. Syntax error
  - C. Stack overflow
  - D. Loops infinitely
5. In recursion, what is the 'call stack'?
  - A. A list of variables
  - B. A structure holding all loops
  - C. A tree of base cases
  - D. A stack that stores method calls

6. What is the output of factorial(3) using recursion?  
 A. 3            B. 5            C. 6            D. 9  
 Which is the correct base case for factorial recursion?  
 A. if (n == 0) return 1;            B. if (n == 1) return 0;  
 C. if (n == 2) return 1;            D. if (n < 0) return n;
7. What does the formula  $f(n) = f(n - 1) + f(n - 2)$  calculate?  
 A. Factorial    B. Fibonacci            C. Sum of digits            D. GCD
8. What are the base cases in the Fibonacci sequence ?  
 A.  $n == 1$  and  $n == 0$             B.  $n == 0$  and  $n == 2$   
 C.  $n == 2$  and  $n == 1$             D.  $n == 2$  and  $n == 3$
9. What is the Fibonacci of 6?  
 A. 6            B. 13            C. 8            D. 5
10. The recursive formula for the sum of first n natural numbers?  
 A.  $sum(n) = n - 1 + sum(n)$             B.  $sum(n) = n + sum(n - 1)$   
 C.  $sum(n) = n + 1$             D.  $sum(n) = sum(n - 2)$
11. What is the sum of first 6 natural numbers using recursion?  
 A. 15            B. 20            C. 21            D. 18
13. What is the recursive formula for sum of digits?  
 A.  $sum(n) = sum(n * 10)$             B.  $sum(n) = n / 10 + sum(n \% 10)$   
 C.  $sum(n) = n \% 10 + sum(n / 10)$   
 D.  $sum(n) = n \% 100 + sum(n * 10)$
14. What is the sum of digits for 34501 using recursion?  
 A. 12            B. 14            C. 13            D. 11
15. Which algorithm finds GCD using recursion?  
 A. Newton's method            B. Euclidean algorithm  
 C. Fermat's method            D. Pythagorean algorithm
16. What is the output of sum(3) using  $sum(n) = n + sum(n - 1)$ ?  
 A. 3            B. 5            C. 6            D. 9
17. What is the GCD of 48 and 18 as shown in the module?  
 A. 8            B. 6            C. 12            D. 4
18. When is recursion preferred over iteration?

- A. When performance is critical B. For simpler logic  
 C. For problems easily broken into sub-problems  
 D. When loops are unavailable
19. Which of these is a downside of recursion?  
 A. Complex code B. Less readability  
 C. Stack overflow D. No reuse
20. Which of these is not a good candidate for recursion?  
 A. Fibonacci series B. GCD  
 C. Tree traversal D. Sorting an already sorted list
21. Recursion is better than iteration in:  
 A. Speed B. Memory C. Readability D. Error handling
22. What happens in the call stack during recursion?  
 A. A variable is created B. A loop starts  
 C. A new frame is added D. The program stops
23. In the RecursionStack example, what is printed first?  
 A. Base Case B. Recursive Method  
 C. main() D. Stack Frame
24. How many times does "Recursive Method" print in recursiveMethod(3)?  
 A. 2 B. 3 C. 4 D. 1
25. Which recursive formula is incorrect?  
 A.  $\text{factorial}(n) = n * \text{factorial}(n - 1)$   
 B.  $\text{GCD}(a, b) = \text{GCD}(b, a \% b)$  C.  $\text{sum}(n) = n + \text{sum}(n - 1)$   
 D.  $\text{power}(x, n) = x + \text{power}(x, n - 1)$
26. Recursion is most commonly used in:  
 A. Networking B. System calls  
 C. Data structure traversal D. Compilers only
27. Which condition leads recursion toward termination?  
 A. Infinite loop B. Recursive case  
 C. Base case D. Return of a loop
28. What will be the output if the recursive function doesn't move toward the base case?

- A. Syntax error
  - B. Stack overflow
  - C. Memory leak
  - D. Runtime optimization
29. Recursive functions can be visualized using \_\_\_\_\_
- A. Queue
  - B. HashMap
  - C. Call Stack
  - D. Switch statement
30. Which of these is the simplest example of recursion?
- A. Printing a list
  - B. Factorial function
  - C. Matrix multiplication
  - D. Variable assignment

### ***Unit Activities***

---

#### **Activity 1: Understanding Base and Recursive Cases**

**Objective:** Understand the difference between base and recursive cases in recursion and why each is necessary.

##### **Instructions:**

Discuss with your classmates the difference between the base case and recursive case in recursion. Use simple examples like factorial or sum of natural numbers to illustrate your points.

##### **Task:**

Write down the base case and recursive case for the factorial function and explain why each is necessary

#### **Activity 2: Trace the Recursion Call Stack**

**Objective:** Visualize how recursive calls are managed using the call stack during execution.

##### **Instructions**

Work in pairs to trace the recursive calls of a factorial function for factorial(4). Use a diagram to show how the call stack expands and then unwinds during execution.

##### **Task:**

Create a step-by-step diagram or list showing each recursive call and its return value. Present your diagram to the class.

### **Activity 3: Implement Recursive Functions**

**Objective: Practice writing and testing recursive methods for common problems.**

#### **Instructions:**

Individually write Java methods for the following recursive problems:

- a. Calculate the factorial of a number
- b. Compute the nth Fibonacci number
- c. Find the sum of digits of a number.

#### **Task:**

Test your methods with different input values and print the results. Submit your code along with sample output.

### **Activity 4: Compare Recursion and Iteration**

**Objective: Analyze the pros and cons of recursion versus iteration to determine when each is appropriate.**

#### **Instructions:**

In small groups, discuss the pros and cons of recursion versus iteration for solving problems like factorial or Fibonacci calculations. Consider readability, performance, and the risk of stack overflow.

#### **Task:**

Prepare a short presentation or a table summarizing your group's findings

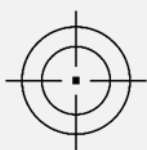
Module Ten

# Exception handling and File I/O

## Unit 1 — Exception Handling and File I/O

### INTRODUCTION

In this unit, you will explore two important topics in Java programming: exception handling and file input/output (I/O). Exception handling allows programs to deal with unexpected events, like dividing by zero or trying to read a missing file, without crashing. You will learn how to use keywords like `try`, `catch`, `finally`, and `throw` to manage these situations effectively. The second part of the unit focuses on file I/O. You will cover file I/O operations, including reading, writing, and appending data, while examining best practices for efficient file handling.



#### LEARNING OUTCOMES

#### At the end of this unit, you should be able to:

- Understand exceptions and their types.
- Use `try`, `catch`, `finally`, `throw` and `throws` to handle exceptions.
- Implement custom exceptions for application-specific errors.
- Perform file operations (read, write, append) using I/O classes.
- Handle checked and unchecked exceptions in file operations.

### 10.1.1 What is an Exception Handling

An exception is an event that disrupts the normal flow of a programme. It is an object representing an error condition. Examples include:

- i. `ArithmeticException` (divide by zero).
- ii. `NullPointerException` (accessing a null object).
- iii. `FileNotFoundException` (file not found)

#### ***Why handles exception?***

The following are the reasons for handling exceptions:

- i. Exception prevents program crashes.
- ii. It provides a user-friendly error message.
- iii. Handle unforeseen situations at runtime.

### 10.1.2 Handling Exceptions

The following are ways of handling exceptions:

#### ***try-catch Block***

The syntax for a try-catch block is:

```
try {
 // Risky mode
} catch (ExceptionType e) {
 // Handle Exception
}
```

#### ***Example 1:***

```
try {
 int result = 10 / 0; // Throws ArithmeticException
} catch (ArithmeticException e) {
 System.out.println("Cannot divide by zero!");
}
```

```
}
```

### ***finally Block***

The finally block of code always executes whether an exception occurs or not. It is used for cleanup (e.g. closing files).

```
try {

 int result = 10 / 0; // Causes ArithmeticException

} catch (ArithmeticException e) {

 System.out.println("Error: Division by zero!");

} finally {

 System.out.println("This block always runs.");

}
```

The output of the code is:

```
Error: Division by zero!

This block always runs.
```

### ***Multiple Crash try-catch Block***

This handles different exceptions separately. Consider the code snippet below:

```
try {

 int[] arr = new int[5];

 arr[10] = 50; // Throws ArrayIndexOutOfBoundsException

} catch (ArithmeticException e) {

 System.out.println("Math error");

} catch (ArrayIndexOutOfBoundsException e) {

 System.out.println("Array index out of bounds!");

}
```

The output of the code is:

Error: Division by zero!

This block always runs.

### ***Multiple Crash try-catch Block***

This handles different exceptions separately. Consider the code snippet below:

```
try {
 int[] arr = new int[5];
 arr[10] = 50; // Throws ArrayIndexOutOfBoundsException
} catch (ArithmeticException e) {
 System.out.println("Math error");
} catch (ArrayIndexOutOfBoundsException e) {
 System.out.println("Array index out of bounds!");
}
```

The output of the code is Array index out of bounds! However, if the try section was 50/0, the output will be a Math Error

### ***Using throw***

throw is used to manually throw an exception.

#### ***Example 1:***

```
if (age < 18) {
 throw new IllegalArgumentException("You must be 18.");
} else {
 System.out.println("Registration successful.");
}
```

#### ***Example 2:***

```
if (age < 18) {
 throw new ArithmeticException("Age must be ≥ 18");
}
```

## Using throws

throws is used to declare exceptions a method might throw.

```
static void checkNumber(int num) throws Exception {

 if (num < 0) {

 throw new Exception("Negative number not allowed");

 }

 System.out.println("Number is " + num);

}
```

## Custom Exceptions

Custom Exception (Figure 10.1) extends Exception or RuntimeException. See the code snippet in Figure xxx.

```
// Custom Exception
class InvalidAgeException extends Exception {
 public InvalidAgeException(String message) {
 super(message);
 }
}

// Usage
public class ExceptionMain {
 public static void main(String[] args) throws InvalidAgeException {
 int age = 16;

 if (age < 18) {
 throw new InvalidAgeException("Age must be >= 18");
 }
 }
}
```

Figure 10.1: Custom Exception

### 10.1.3 File Input and Output (I/O)

Java uses classes in java.io and java.nio.file packages to handle file operations.

#### Writing to a File

Here are some ways to write to a file.

### Example 1: Using FileWriter (Basic Text Writing)

```
import java.io.FileWriter;
import java.io.IOException;

public class WriteToFileExample {
 public static void main(String[] args) {
 try {
 FileWriter wr = new FileWriter("example.txt"); // Open file
 wr.write("Hello, this is a test.\n");
 wr.write("Writing to a file in Java is easy!");
 wr.close(); // Close the file
 System.out.println("File written successfully.");
 } catch (IOException e) {
 System.out.println("An error occurred: " + e.getMessage());
 }
 }
}
```

Figure 10.2: Using FileWriter

FileWriter allows you to write characters directly into a file. In the code example (Figure xxx), a FileWriter object is created and linked to the file named "example.txt". The .write() method is used to add text to the file. It is important to close the writer using .close() to ensure all data is saved properly and system resources are released. FileWriter is ideal for simple and small text writing tasks.

### Example 2: Using BufferedWriter for Efficient Writing

BufferedWriter enhances writing performance by buffering the characters before sending them to the file.

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class BufferedWriterExample {
 public static void main(String[] args) {
 try (BufferedWriter bw = new BufferedWriter(new FileWriter("buffered.txt"))) {
 bw.write("BufferedWriter writes text efficiently.\n");
 bw.newLine(); // Platform-independent newline
 bw.write("It buffers the characters for better performance.");
 System.out.println("File written successfully with BufferedWriter.");
 } catch (IOException e) {
 System.out.println("Error writing file: " + e.getMessage());
 }
 }
}
```

Figure 10.3: Using BuferedWriter

In Figure 10.3, a `FileWriter` is wrapped inside a `BufferedWriter`, which collects data in a buffer and writes it to the file in larger chunks. This reduces the number of direct write operations, making it more efficient for writing multiple lines or larger amounts of text. The code uses `try-with-resources` to automatically close the writer and release resources safely. `BufferedWriter` is ideal when you need better performance during extensive file writing.

**Example 3: Using `PrintWriter` for convenient writing with print methods**

```
import java.io.PrintWriter;
import java.io.IOException;

public class PrintWriterExample {
 public static void main(String[] args) {
 try (PrintWriter pw = new PrintWriter("printwriter.txt")) {
 pw.println("PrintWriter allows easy writing.");
 pw.printf("You can also format numbers like %.2f\n", 123.456);
 System.out.println("File written successfully with PrintWriter.");
 } catch (IOException e) {
 System.out.println("Error writing file: " + e.getMessage());
 }
 }
}
```

Figure 10.4: Using `PrintWriter`

`PrintWriter` offers convenient methods like `.println()` and `.printf()` for writing formatted text to files. It wraps a file stream, allowing you to write lines easily, similar to how you print to the console with `System.out`. Using `try-with-resources` ensures the writer is closed automatically, preventing resource leaks. `PrintWriter` is ideal when you need simple, readable, and formatted output to a file.

***Files.write() (Java NIO - Simplest Modern Way)***

`Files.write()` is a convenient method in the Java NIO (New Input/Output) package used to write data to files easily and efficiently. It accepts a file path and the content (usually as a `byte[]` or a `List<String>`) and writes the data all at once.

Key Features include:

- i. Simple and clean: Requires minimal code to write text or bytes to a file.
- ii. Automatic file creation: If the specified file does not exist, `Files.write()` creates it.
- iii. Overwrites by default: It replaces existing file content unless options to append are specified.
- iv. Works with modern Path objects: Uses the newer `java.nio.file.Path` instead of `java.io.File`.

- v. Exception handling: Throws IOException, so proper error handling is required.

**Use Cases:** Best suited for small to medium files where writing all data in one go is practical. It is ideal for quick file writes without the overhead of streams or writers.

**Example 1: Write a list of strings to a file**

The program in Figure 10.5 demonstrates modern file writing using the java.nio.file package.

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.Arrays;
import java.util.List;

public class ModernFileWrite {
 public static void main(String[] args) {
 List<String> lines = Arrays.asList(
 "First line", "Second line", "Third line", "Last line");

 List<String> lines_2 = List.of("Hello world", "For second file");
 try {
 Files.write(Paths.get("modern.txt"), lines);
 Files.write(Paths.get("modern_2.txt"), lines_2);
 System.out.print("File written successfully to modern.txt");
 System.out.println(" and to modern_2.txt");
 } catch (IOException e) {
 System.err.println("NIO write failed: " + e.getMessage());
 }
 }
}
```

Figure 10.5: Writing a list of strings

The code in Figure 10.5, creates two lists of strings representing lines to be written to files. Using the Files.write() method, it writes these lines to two separate text files: modern.txt and modern\_2.txt. The program handles potential IOExceptions with a try-catch block, ensuring any write errors are caught and reported. Upon successful completion, it prints a confirmation message indicating both files were written successfully. This approach simplifies file writing with concise and efficient code

### ***In-Text Questions (ITQs)***

1. What is the purpose of the finally block?
2. What is exception handling in Java, and why is it important?

3. What are the main keywords used in Java for handling exceptions, and how do they differ?
4. How does a try-catch block work, and how can multiple exceptions be handled?
5. What are custom exceptions, and how are they implemented in Java?
6. What are some common exceptions in Java, and when are they typically thrown?

### ***In-Text Answers (ITAs)***

---

1. To execute cleanup code, such as closing files, regardless of whether an exception occurred
2. Exception handling is a mechanism that allows a program to manage unexpected events or errors (exceptions) such as division by zero or missing files, without crashing. It is important because it prevents program crashes, provides user-friendly error messages, and helps handle unforeseen situations at runtime.
3. The main keywords are:
  - a. try: Defines a block of code that might throw exceptions.
  - b. catch: Catches and handles specific exceptions thrown in the try block.
  - c. finally: Contains code that always executes after try-catch blocks for cleanup purposes.
  - d. throw: Used to manually throw an exception.
  - e. throws: Declares the exceptions that a method might throw.
4. A try-catch block encloses risky code in the try block, and exceptions thrown are caught and handled in the corresponding catch blocks. Multiple exceptions can be handled by writing multiple catch blocks, each targeting a different exception type, allowing separate handling for different error scenarios.
5. Custom exceptions are user-defined exception classes that extend Exception or RuntimeException. They allow developers to create specific exceptions tailored to the needs of their applications, enhancing clarity and error handling.
6. Common exceptions include:
  - a. ArithmeticException: Thrown during illegal arithmetic operations, e.g., division by zero.

- b. `NullPointerException`: Thrown when accessing methods or fields of a null object.
- c. `FileNotFoundException`: Thrown when a program attempts to access a file that does not exist.

#### 10.1.4 Conclusion

Understanding exception handling and file I/O is essential for building robust Java applications that can gracefully manage errors and perform reliable data storage operations. By mastering these concepts, programmers can prevent crashes, provide meaningful error feedback, and manipulate files efficiently, thereby enhancing the overall quality and reliability of software.

#### **Summary**

---

This unit covers two crucial Java programming topics: exception handling and file input/output (I/O). It explains how exceptions can disrupt normal program flow and how to use `try`, `catch`, `finally`, `throw`, and `throws` keywords to handle them effectively. The unit also highlights the creation of custom exceptions for specific application needs. For file I/O, it demonstrates various ways to read, write, and append data to files using classes from `java.io` and `java.nio.file` packages, emphasizing efficient and modern file handling practices.

#### **Self-Assessment Questions**

---

1. What is an exception in Java?
  - A) A method call
  - B) An event that disrupts normal flow
  - C) A variable type
  - D) A file operation
2. Which exception occurs when dividing by zero?
  - A) `NullPointerException`
  - B) `FileNotFoundException`
  - C) `ArithmeticException`
  - D) `IOException`
3. Which keyword begins a block that may throw exceptions?

- A) catch   B) try                      C) throw                      D) finally
4. What keyword is used to handle an exception?
- A) try                      B) throw                      C) catch                      D) finally
5. Which block runs regardless of an exception or not?
- A) catch   B) try                      C) throw                      D) finally
6. How do you manually throw an exception?
- A) throws B) throw                      C) try                      D) catch
7. Which keyword indicates a method might throw exceptions?
- A) throw   B) throws                      C) catch                      D) finally
8. Which exception is thrown for an invalid array index?
- A) ArithmeticException   B) ArrayIndexOutOfBoundsException  
C) NullPointerException   D) FileNotFoundException
9. Why should exceptions be handled?
- A) To crash the program                      B) To prevent program crashes  
C) To make code complex                      D) To ignore errors
10. What exception is thrown when trying to use a null object?
- A) ArithmeticException                      B) NullPointerException  
C) FileNotFoundException                      D) IOException
11. What does the 'finally' block mainly ensure?
- A) Only executes if an exception occurs  
B) Closes resources such as files  
C) Skips exceptions                      D) Creates exceptions
12. Which exception type is checked at compile-time?
- A) RuntimeException                      B) Error  
C) IOException                      D) NullPointerException
13. What is a custom exception?
- A) A Java built-in exception

- B) An exception made by the programmer
- C) An error that cannot be caught      D) None of the above
14. Which class is the base class for all exceptions?
- A) Throwable      B) Error
- C) Exception      D) RuntimeException
15. Which exception is unchecked?
- A) IOException      B) SQLException
- C) NullPointerException      D) ClassNotFoundException
16. How many exceptions can a single try block catch?
- A) Only one      B) Multiple      C) None      D) Exactly two
17. What package contains classes for file I/O in Java?
- A) java.util      B) java.net      C) java.io      D) java.math
18. Which class is used for writing characters directly to a file?
- A) FileReader      B) FileWriter
- C) BufferedReader      D) PrintWriter
19. Which class buffers characters for efficient writing?
- A) FileWriter      B) BufferedWriter
- C) PrintWriter      D) Scanner
20. What method saves changes when using FileWriter?
- A) flush()      B) close()      C) write()      D) open()
21. Which class offers println() and printf() for file writing?
- A) BufferedWriter      B) FileWriter
- C) PrintWriter      D) Scanner
22. What is the benefit of using try-with-resources?
- A) Automatically closes resources
- B) Catches exceptions automatically
- C) Skips file writing      D) Creates new exceptions

23. What does `Files.write()` method do?
- A) Reads bytes from a file
  - B) Writes data to a file simply and efficiently
  - C) Opens file in read mode
  - D) Deletes a file
24. Which package introduced `Files.write()`?
- A) `java.io`
  - B) `java.util`
  - C) `java.nio.file`
  - D) `java.net`
25. If a file does not exist, what does `Files.write()` do?
- A) Returns null
  - B) Creates the file automatically
  - C) Does nothing
  - D) Throws `FileNotFoundException`
26. What type of data does `Files.write()` typically accept?
- A) String only
  - B) `List<String>` or `byte[]`
  - C) Integer
  - D) None of the above
27. What exception must be handled or declared when using `Files.write()`?
- A) `RuntimeException`
  - B) `IOException`
  - C) `NullPointerException`
  - D) `ArrayIndexOutOfBoundsException`
28. Which block catches multiple different exceptions separately?
- A) finally block
  - B) single catch block
  - C) multiple catch blocks
  - D) try block
29. Which exception is thrown when file reading fails because the file is missing?
- A) `IOException`
  - B) `FileNotFoundException`
  - C) `NullPointerException`
  - D) `RuntimeException`
30. What is the difference between `throw` and `throws`?
- A) `throw` declares exception, `throws` throws it
  - B) `throw` is for throwing exceptions manually, `throws` declares exceptions that a method may throw
  - C) Both mean the same
  - D) None of the above
31. What is printed by this code?

```
try {
 int a = 10 / 0;
} catch (ArithmeticException e) {
 System.out.println("Divide by zero");
} finally {
 System.out.println("Finally block");
}
```

- A) Divide by zero                                      B) Finally block  
 C) Divide by zero                                      D) No output

32. Which of the following is a checked exception?

- A) NullPointerException                      B) IOException  
 C) ArithmeticException    D) ArrayIndexOutOfBoundsException

33. Which of the following is used for reading text from a file?

- A) FileWriter                                      B) FileReader  
 C) PrintWriter                                      D) BufferedWriter

34. Which method in BufferedWriter inserts a new line?

- A) writeLine()                      B) newLine()  
 C) nextLine()                      D) println()

35. In a try-catch block, what happens if no exception is thrown?

- A) catch block executes                      B) finally block executes  
 C) Both catch and finally execute                      D) Neither executes

36. Which of the following is NOT an exception type in Java?

- A) ArithmeticException                      B) NullPointerException  
 C) FileNotFoundException                      D) FileExistsException

37. What does the throws keyword do?

- A) Catches exceptions  
 B) Declares exceptions a method might throw

- C) Prevents exceptions    D) Terminates the program
38. When should you use `BufferedWriter` over `FileWriter`?
- A) Writing a single character  
B) Writing large amounts of data efficiently  
C) Reading data                      D) Deleting files
39. Which method in `PrintWriter` formats output?
- A) `println()`                      B) `print()`                      C) `printf()`                      D) `write()`
40. What is the advantage of using `Files.write()` over `FileWriter`?
- A) More code to write files  
B) Simpler, more modern API with Path support  
C) Less efficient                      D) Requires manual closing
1. What is an array in programming?

### ***Unit Activities***

---

#### **Activity 1: Understanding and Handling Exceptions**

**Objective:** Identify and handle common Java exceptions using try-catch-finally blocks.

#### **Instructions:**

1. Review common exceptions such as `ArithmeticException`, `NullPointerException`, and `ArrayIndexOutOfBoundsException`.
2. Learn how to implement try, catch, and finally blocks.

#### **Task:**

1. Write a Java program that demonstrates handling at least two different exceptions using multiple catch blocks.
2. Include a finally block that prints a message regardless of whether an exception occurs.

#### **Activity 2: Advanced Exception Handling with throw, throws, and Custom Exceptions**

**Objective: Practice throwing exceptions, declaring them with throws, and creating custom exception class.**

**Instructions**

1. Understand the difference between throw and throws.
2. Learn how to define and use custom exceptions by extending Exception or RuntimeException

**Tasks:**

1. Create a method that throws a checked exception using throws.
2. Manually throw an exception using throw based on a condition.
3. Define a custom exception class and demonstrate throwing and catching it in your program.

**Activity 3: File Writing Basics with FileWriter and BufferedWriter**

**Objective: Write text files using FileWriter and BufferedWriter to understand basic and efficient file output**

**Instructions:**

1. Learn to write simple text to a file using FileWriter.
2. Improve performance by using BufferedWriter wrapped around FileWriter.

**Tasks:**

1. Write a Java program that writes multiple lines of text to a file using FileWriter.
2. Modify the program to use BufferedWriter and compare the output and performance

**Activity 4: Convenient File Writing Using PrintWriter and Files.write()**

**Objective: Use PrintWriter and the modern Java NIO Files.write() method to write formatted and efficient file output.**

**Instructions:**

1. Practice writing formatted text using PrintWriter.
2. Explore the Files.write() method for writing entire lists of strings to a file in one operation

**Task:**

1. Write a program that uses `PrintWriter` to write formatted lines to a file.
2. Write another program that uses `Files.write()` to write a list of strings to a file

**Activity 5: Integrated File I/O and Exception Handling Project**

**Objective: Combine exception handling and file I/O in a complete Java application.**

**Instructions:**

1. Use `try-with-resources` to manage file resources safely.
2. Handle multiple exceptions that may occur during file reading or writing operations.

**Task:**

1. Develop a Java program that reads data from one file and writes filtered or processed data to another file.
2. Implement exception handling for potential I/O errors and invalid data inputs.
3. Ensure all resources are properly closed using `try-with-resources`

**REFERENCES:**

- Farrell, J. (2019). Java programming (Nineth edition). Cengage.
- Lemay, L., & Perkins, C. (1996). Teach Yourself Java in 21 Days.
- Liang, Y. D. (2015). Introduction to Java programming (Tenth edition). Pearson.
- Schildt, H. (2014). Java: A beginner's guide (6th ed). McGraw-Hill Education.

# Answers to Self-Assessment Questions

## Module One

### 11.1.1 Unit 1: Answers to Self-Assessment Questions

#### *Overview Of Computer Programming*

---

- |       |       |       |       |        |
|-------|-------|-------|-------|--------|
| 1. B. | 2. C. | 3. D. | 4. D. | 5. A   |
| 6. B. | 7. C. | 8. B. | 9. D. | 10. B. |

## Module Two

### 11.2.1 Unit 1: Answers to Self-Assessment Questions

#### *Introduction To Java And Java Development Environment*

---

- |       |       |       |       |        |
|-------|-------|-------|-------|--------|
| 1.A)  | 2. B) | 3. C) | 4. C) | 5. B)  |
| 6. C) | 7. B) | 8. C) | 9. B) | 10. A) |

## Module Three

### 11.3.1 Unit 1: Answers to Self-Assessment Questions

#### *Java basics - data types, variables and operators in java part I*

---

- |        |        |        |        |        |
|--------|--------|--------|--------|--------|
| 1. B)  | 2. B)  | 3. C)  | 4. C)  | 5. B)  |
| 6. C)  | 7. C)  | 8. C)  | 9. C)  | 10. A) |
| 11. D) | 12. C) | 13. C) | 14. B) | 15. C) |
| 16. C) | 17. C) | 18. C) | 19. C) | 20. C) |

21. C)                      22. C)                      23. C)

### 11.3.2 Unit 2: Answers to Self-Assessment Questions

#### ***Q1. Explain Radioactivity***

---

|        |        |        |        |        |
|--------|--------|--------|--------|--------|
| 1. D)  | 2. B)  | 3. B)  | 4. A)  | 5. B)  |
| 6. B)  | 7. B)  | 8. A)  | 9. B)  | 10. B) |
| 11. C) | 12. B) | 13. A) | 14. C) | 15. A) |
| 16. B) | 17. A) | 18. C) | 19. A) | 20. B) |
| 21. A) | 22. B) | 23. B) | 24. A) | 25. C) |
| 26. A) | 27. A) | 28. C) | 29. D) | 30. B) |
| 31. C) |        |        |        |        |

## Module Four

### 11.4.1 Unit 1: Answers to Self-Assessment Questions

#### ***Conditional Statements And Decision Making***

---

|       |       |       |       |       |
|-------|-------|-------|-------|-------|
| 1. B  | 2. B  | 3. A  | 4. C  | 5. A  |
| 6. C  | 7. A  | 8. B  | 9. C  | 10. A |
| 11. A | 12. C | 13. C | 14. C | 15. B |
| 16. B | 17. A | 18. C | 19. B | 20. C |
| 21. B | 22. C | 23. B | 24. A |       |

## 11.4.2 Unit 2: Answers to Self-Assessment Questions

### ***Loops And Branching***

---

|       |       |       |       |       |
|-------|-------|-------|-------|-------|
| 1. B. | 2. C  | 3. C  | 4. C  | 5. C  |
| 6. A  | 7. C  | 8. B  | 9. D  | 10. D |
| 11. D | 12. B | 13. A | 14. B | 15. B |
| 16. C | 17. B | 18. C | 19. B | 20. B |

## Module Five

## 11.5.1 Unit 1: Answers to Self-Assessment Questions

### ***Methods And Parameter Passing***

---

|        |        |        |        |        |
|--------|--------|--------|--------|--------|
| 1. B.  | 2. A.  | 3. C.  | 4. B.  | 5. A.  |
| 6. B.  | 7. B.  | 8. C.  | 9. B.  | 10. B. |
| 11. C. | 12. B. | 13. C. | 14. B. | 15. A. |
| 16. B. | 17. C. | 18. A. | 19. C. | 20. B. |
| 21. B. | 22. B. | 23. B. | 24. B. | 25. B. |
| 26. A. |        |        |        |        |

## Module Six

### 11.6.1 Unit 1: Answers to Self-Assessment Questions

#### ***Object-Oriented Programming (OOP) Basics***

---

|        |        |        |        |        |
|--------|--------|--------|--------|--------|
| 1.C.   | 2. B.  | 3. A.  | 4. B.  | 5. C.  |
| 6.D.   | 7. C.  | 8. B.  | 9. C.  | 10. B. |
| 11. B. | 12. D. | 13. B. | 14. B. | 15. C. |
| 16. C. | 17. B. | 18. B. | 19. B. | 20. C. |

### 11.6.2 Unit 2: Answers to Self-Assessment Questions

#### ***Object-Oriented Programming (OOP) Advanced***

---

|        |        |        |        |        |
|--------|--------|--------|--------|--------|
| 1.C.   | 2. B.  | 3. B.  | 4. B.  | 5. B.  |
| 6. B.  | 7. B.  | 8. B.  | 9. B.  | 10. C. |
| 11. B. | 12. B. | 13. D. | 14. B. | 15. B. |
| 16. B. | 17. C. | 18. A. | 19. C. | 20. A. |
| 21. B. |        |        |        |        |

## Module Seven

### 11.7.1 Unit 1: Answers to Self-Assessment Questions

#### *Data Structures and String Handling*

---

|       |       |        |       |       |
|-------|-------|--------|-------|-------|
| 1. B  | 2. C  | 3. B   | 4. B  | 5. B  |
| 6. B  | 7. C  | 8. B   | 9. A  | 10. B |
| 11. D | 12. B | 13. C  | 14. C | 15. D |
| 16. B | 17. B | 18. A. |       |       |

## Module Eight

### 11.8.1 Unit 1: Answers to Self-Assessment Questions

#### *Arrays And Arrays Manipulation*

---

|       |       |       |       |       |
|-------|-------|-------|-------|-------|
| 1. D  | 2. C  | 3. C  | 4. C  | 5. C  |
| 6. C  | 7. A  | 8. A  | 9. C  | 10. B |
| 11. C | 12. B | 13. B | 14. B | 15. B |
| 16. B | 17. C | 18. C | 19. C | 20. B |
| 21. A | 22. C | 23. B | 24. D | 25. C |
| 26. C | 27. B | 28. D | 29. B | 30. B |
| 31. C | 32. C | 33. B |       |       |

## Module Nine

### 11.9.1 Unit 1: Answers to Self-Assessment Questions

#### *Introduction To Recursion*

---

|       |       |       |       |       |
|-------|-------|-------|-------|-------|
| 1. D  | 2. C  | 3. B  | 4. C  | 5. D  |
| 6. C  | 7. A  | 8. B  | 9. A  | 10. D |
| 11. B | 12. C | 13. C | 14. C | 15. B |
| 16. C | 17. B | 18. C | 19. C | 20. D |
| 21. C | 22. C | 23. B | 24. B | 25. D |
| 26. C | 27. C | 28. B | 29. C | 30. B |

## Module Ten

### 11.10.1 Unit 1: Answers to Self-Assessment Questions

#### *Exception Handling and File I/O*

---

|       |       |       |       |       |
|-------|-------|-------|-------|-------|
| 1. B  | 2. C  | 3. B  | 4. C  | 5. D  |
| 6. B  | 7. B  | 8. B  | 9. B  | 10. B |
| 11. B | 12. C | 13. B | 14. A | 15. C |
| 16. B | 17. C | 18. B | 19. B | 20. B |
| 21. C | 22. A | 23. B | 24. C | 25. B |
| 26. B | 27. B | 28. C | 29. B | 30. B |
| 31. C | 32. B | 33. B | 34. B | 35. B |

36. D

37. B

38. B

39. C

40. B

## GLOSSARY

**Abstract Class:** A class that cannot be instantiated directly and may contain both abstract methods (without implementation) and concrete methods (with implementation). It serves as a base class for other classes.

**Abstraction:** The process of hiding complex implementation details and showing only the essential features or behaviors of an object.

**Accessing (Array):** Retrieving the value stored at a specific index within an array using array indexing.

**Allocation:** The process of assigning memory space for an array using the new keyword in Java.

**append():** A method in StringBuilder and StringBuffer to add content to the end of the existing string.

**Argument:** The actual value that is passed into a method when it is invoked. It is assigned to the method's parameters.

**ArithmeticException:** A runtime error that occurs when illegal arithmetic operations are performed, such as dividing by zero.

**Arithmetic Operators:** Symbols that perform basic mathematical operations such as addition (+), subtraction (-), multiplication (\*), division (/), and modulus (%).

**Array:** A data structure that stores multiple elements of the same data type in contiguous memory locations and allows indexed access.

**Array Length:** The number of elements that an array can hold, accessed using the .length property (e.g., array.length).

**ArrayList:** A resizable array-like data structure in Java found in the java.util package.

**Arrays Class:** A utility class in java.util package that provides methods for manipulating arrays, such as sorting and comparing.

**Assignment Operators:** Operators that assign values to variables, including compound assignments like +=, -=, \*=, and /=.

**Attributes:** Data members or variables that store the state of an object.

**Base Case:** A condition in a recursive function that stops further recursive calls. It prevents infinite recursion and acts as the termination point.

**Bitwise Operators:** Operators that perform operations on the binary representations of integers (e.g., &, |, ^, ~).

**Boolean Expression:** An expression that evaluates to either true or false. Used in conditional and loop statements.

**break Statement:** Terminates a switch case or loop early, preventing fall-through behavior.

**BufferedReader:** A class used to read text from a character-input stream efficiently, often combined with InputStreamReader for console input.

**BufferedWriter:** A wrapper around FileWriter that buffers output for more efficient writing.

**Call Stack:** A memory structure used by the computer to keep track of method calls, including recursive ones. Each recursive call adds a new frame to the stack until the base case is reached.

**case:** A label used inside a switch block. Each case represents a potential match for the evaluated expression.

**charAt():** A method that returns the character at a specified index in a string.

**Class:** A blueprint in OOP defining attributes and behaviors for objects.

**Clone:** A method used to create an exact copy of an array, such as `array.clone()`.

**Command Prompt:** A command-line interpreter in Windows used to run commands, including Java commands like `java --version`.

**compareTo():** Compares two strings lexicographically (dictionary order) based on Unicode values.

**Comparison (Arrays):** Checking whether two arrays have the same content using methods like `Arrays.equals()`.

**Compilation:** The process of converting Java source code (.java files) into bytecode (.class files) using the Java compiler (javac).

**Compile-Time Polymorphism:** Also known as static polymorphism; achieved using method overloading. Method resolution occurs at compile time.

**Computer Programming:** This is the process of writing instructions that enable a computer to perform specific tasks.

**Conditional Statement:** A type of control structure used to make decisions based on a condition (boolean expression). Includes `if`, `if-else`, `else-if`, `switch`, and the ternary operator.

**Constant:** A fixed value that cannot change during program execution, declared using the `final` keyword.

**Constructor:** A special method in a class used to initialize objects. It has the same name as the class and does not have a return type.

**contains():** Checks whether a specific sequence of characters exists within a string.

**continue Statement:** A branching statement that skips the current iteration of a loop and proceeds to the next iteration.

**Contiguous Memory:** Memory blocks allocated side by side, used to store array elements for fast indexed access.

**Control Structure:** A programming construct that determines the order in which code statements are executed. Java control structures include conditional, looping, and branching statements.

**Custom Exception:** A user-defined exception class created by extending `Exception` or `RuntimeException` to handle specific application errors.

**Data Types:** Categories of data that specify what kind of value a variable can hold. Includes primitive types (`int`, `float`, `boolean`, etc.) and non-primitive types (classes, arrays, strings).

**Declaration:** The process of defining a variable with a specific data type and name, e.g., `int x;`

**Declarative Programming:** A paradigm focused on specifying the result rather than the procedure to achieve it.

**default:** An optional block in a `switch` statement that executes if no case matches.

**Default Constructor:** A constructor provided by Java when no constructors are explicitly defined. It takes no parameters and sets default values.

**delete():** Deletes characters between specified indexes.

**Do-While Loop:** A loop that executes its body at least once and then repeats as long as a specified condition is true.

**else-if Ladder:** Allows testing of multiple conditions sequentially. If one condition is true, its block executes; otherwise, the next condition is checked.

**Encapsulation:** A principle of OOP that bundles data (fields) and methods into a single unit (class) and restricts access to some components, typically through private fields and public getter/setter methods.

**endsWith():** Determines whether a string ends with a specific suffix.

**Environment Variable:** A system-wide variable that stores information used by operating systems and applications. JAVA\_HOME is an example that points to the JDK installation directory.

**equals():** Compares two strings for exact equality, including case sensitivity.

**equalsIgnoreCase():** Compares two strings for equality, ignoring case differences.

**Exception:** An event during program execution that disrupts the normal flow, often representing errors like dividing by zero or file not found.

**Exception Handling:** A programming technique to manage runtime errors using constructs like try, catch, finally, throw, and throws to prevent program crashes.

**Factorial:** A mathematical operation (denoted as  $n!$ ) that multiplies a number by every positive integer less than it. Factorials are a classic example of recursive programming.

**Fall-Through:** Behavior in a switch where, if break is omitted, control continues to the next case even if it's not matched.

**Fibonacci Sequence:** A sequence of numbers where each number is the sum of the two preceding ones. It begins with 0 and 1 and is often used to demonstrate recursion.

**File Input/Output (I/O):** Operations that read data from or write data to files on disk.

**Files.write():** A modern method in Java NIO (New Input/Output) package for writing data to files in a simple and efficient way.

**FileWriter:** A class in Java used for writing character streams to files, suitable for basic text writing.

**finally Block:** A block that always executes after try and catch blocks, commonly used for cleanup actions such as closing files.

**for Loop:** A loop structure best used when the number of iterations is known beforehand. It includes initialization, condition, and update expressions.

**Functional Programming:** A paradigm focused on what to compute using pure functions and avoiding mutable data.

**Greatest Common Divisor:** The largest integer that divides two numbers exactly, without a remainder. It can be computed using recursion through the Euclidean algorithm.

**GCD:** See Greatest Common Divisor

**Hierarchical Inheritance:** A type of inheritance where multiple subclasses inherit from the same superclass.

**if Statement:** Executes a block of code only if a given condition evaluates to true.

**if-else Statement:** Chooses between two paths: one block runs if the condition is true, the other if it is false.

**Infinite Recursion:** A programming error that occurs when a recursive function does not reach a base case, causing it to call itself endlessly and often resulting in a stack overflow.

**initialization:** The step in a loop where the loop control variable is given an initial value.

**Imperative Programming:** A paradigm focused on describing how to perform tasks using statements that change a program's state.

**Immutable:** Describes objects (like Strings) whose state cannot be changed once created. Any modification results in a new object.

**Index:** A numeric position of an element within an array, starting at 0 for the first element.

**indexOf():** Returns the index of the first occurrence of a specified character or substring. Returns -1 if not found.

**Inheritance:** A mechanism where a subclass acquires properties and behaviors (fields and methods) from a superclass. Promotes code reuse and establishes a hierarchical relationship.

**insert():** Inserts characters at a specified index in a `StringBuilder` or `StringBuffer` object.

**Initialization:** Assigning an initial value to a variable at the time of declaration.

**Interface:** A reference type in Java that defines a set of abstract methods that implementing classes must provide. It specifies what methods a class should have without defining how they work.

**Iteration:** A process of repeating a set of instructions using loops (e.g., `for`, `while`). It can sometimes be used as an alternative to recursion for solving problems.

**Java:** A versatile, object-oriented programming language used for building applications across multiple platforms, known for its "Write Once, Run Anywhere" (WORA) capability.

**Java Development Kit (JDK):** A software development environment used for developing Java applications; includes tools like the compiler (`javac`) and runtime (`java`).

**Java Virtual Machine (JVM):** An engine that enables Java programs to run on any platform by interpreting compiled Java bytecode.

**Jump Statement:** Statements that alter the flow of control, such as `break`, `continue`, and `return`.

**Lexicographical Order:** A method of ordering strings based on the alphabetical order of their component characters (like dictionary order).

**Logical Operators:** Operators used for logical operations, such as `AND (&&)`, `OR (||)`, and `NOT (!)`.

**Logic-Based Programming:** A programming style based on formal logic rules to derive conclusions.

**Loop Body:** The block of code inside a loop that is executed repeatedly as long as the loop condition is true.

**Loop Condition:** A boolean expression evaluated before (`while` and `for` loops) or after (`do-while` loop) each iteration to determine whether to continue looping.

**Linear Search:** A simple searching algorithm that checks each element in an array sequentially for a target value.

**main Method:** The entry point of a Java program where execution begins, typically declared as `public static void main(String[] args)`.

**Methods:** Functions defined within a class that describe behaviors of objects. A method is a reusable block of code that performs a specific task.

**Method Overloading:** A feature in Java that allows a class to have more than one method with the same name, provided their parameter lists differ.

**Method Overriding:** When a subclass provides a specific implementation of a method already defined in its superclass. It enables runtime polymorphism.

**Method Signature:** The unique identity of a method, defined by its name and the number and type of parameters it accepts.

**Modularization:** The practice of dividing a program into separate methods or modules to make it easier to read, debug, and maintain.

**Multilevel Inheritance:** A scenario where a class inherits from another derived class, forming a chain of inheritance.

**Multi-Dimensional Array:** An array containing more than one dimension, such as a 2D array (array of arrays), used for representing tables or matrices.

**Mutable:** Refers to objects (like `StringBuilder` and `StringBuffer`) whose values can be changed after creation.

**Nested if Statement:** An if or else block containing another if statement. Used to evaluate more than one condition in a hierarchical structure.

**new:** Used to create new objects, including Strings, that are stored in the heap rather than the String pool.

**Non-Primitive Data Types:** Also known as reference types, these include objects such as classes, interfaces, arrays, and strings.

**Non-Void Method:** This type of method performs operations and returns a value after execution, such as an integer or a string.

**Non-Static Method:** A method that is associated with an object of the class and requires object instantiation before it can be used.

**Object:** An instance of a class containing data (attributes) and methods (behaviors).

**Object-Oriented Programming (OOP):** A paradigm organizing code into objects which are instances of classes; key concepts include encapsulation, inheritance, and polymorphism.

**One-Dimensional Array:** A linear array where elements are stored in a single row and accessed using one index.

**Operators:** Symbols that perform operations on variables and values, including arithmetic, relational, logical, unary, ternary, and bitwise operators.

**Operator Precedence:** The order in which operators are evaluated in expressions. Operators with higher precedence are evaluated first.

**Parameter:** A variable listed in a method's definition that is used to accept values when the method is called.

**Parameterized Constructor:** A constructor that accepts arguments to initialize an object with specific values during creation.

**Polymorphism:** The ability of objects of different types to be treated as objects of a common superclass. It allows one interface to be used for different data types.

**Primitive Data Types:** Basic data types that store single values, such as byte, short, int, long, float, double, char, and boolean.

**Programming Paradigm:** A style or approach to programming, such as imperative, functional, declarative, logic-based, and object-oriented programming.

**PrintWriter:** A class that provides convenient methods to write formatted text to files.

**Private:** A keyword used to restrict access to class members so that they can only be accessed within the same class.

**Public:** A keyword that allows class members to be accessible from any other class.

**Recursion:** A programming technique in which a function calls itself to solve smaller instances of a problem. It is used for elegant and concise problem-solving.

**Recursive Case:** The part of a recursive function where the function calls itself with modified arguments, aiming to move closer to the base case.

**replace():** Replaces characters or sequences in a string with specified values.

**return Statement:** A statement used to exit a method immediately and optionally return a value to the caller.

**reverse():** Reverses the characters in the String-Builder or StringBuffer.

**Runtime Polymorphism:** Also known as dynamic polymorphism; achieved using method overriding. Method resolution occurs at runtime.

**Scanner Class:** A utility class in java.util used to read various types of input from the user via the console.

**Selection Control:** Decision-making statements that choose different execution paths based on conditions (e.g., if, if-else, switch).

**Scope:** The region of a program where a variable is accessible.

**Single Inheritance:** A type of inheritance where a class inherits from one superclass only.

**Sorting:** The process of arranging array elements in ascending or descending order using Arrays.sort().

**split():** Divides a string into an array of substrings based on a regular expression.

**Stack Overflow:** An error that occurs when there are too many function calls placed on the call stack, usually caused by improper or infinite recursion.

**startsWith():** Determines whether a string starts with a specific prefix.

**Statements:** Instructions in a Java program that perform actions.

**Static Method:** A method that belongs to a class rather than an instance of the class. It can be called without creating an object.

**String:** A non-primitive data type that represents a sequence of characters.

**StringBuffer:** Similar to StringBuilder but thread-safe due to synchronization. Used in multi-threaded applications.

**StringBuilder:** A mutable class used to manipulate strings efficiently in single-threaded environments. Not thread-safe but faster than StringBuffer.

**String Literal:** A way to create a String by directly assigning characters enclosed in double quotes. Example: String s = "Hello";

**String Pool:** A special memory area in Java where String literals are stored for memory efficiency and reuse.

**Subclass (Child Class):** The class that inherits from another class. It can add or override fields and methods from the superclass.

**substring():** A method used to extract a part of a string, based on a starting and optionally an ending index.

**Sub-array:** A portion or segment of an array derived from the original array.

**super:** A special keyword used in a subclass to refer to its immediate superclass. It can be used to call superclass constructors and methods.

**Superclass (Parent Class):** The class whose features are inherited by another class. Also known as the base class.

**switch Statement:** A control structure that selects and executes a block of code among multiple options based on the value of an expression. More efficient than using many if-else statements.

**Syntax:** The set of rules that defines the combinations of symbols considered to be correctly structured programs in a language.

**System.arraycopy():** A method used to copy elements from one array to another efficiently in Java.

**Ternary Operator:** A shorthand conditional operator with three operands, expressed as condition? value\_if\_true : value\_if\_false.

**Thread-safe:** A property of code or components (like StringBuffer) that can be safely used in multi-threaded programs.

**throw:** A keyword used to manually trigger an exception.

**throws:** A keyword used in method declarations to indicate which exceptions might be thrown by the method.

**toLowerCase():** Methods to convert all characters in a string to lowercase.

**toUpperCase():** Methods to convert all characters in a string to uppercase.

**trim():** Removes whitespace from the beginning and end of a string.

**try-catch Block:** A structure used to enclose code that might throw exceptions, allowing the program to catch and handle errors gracefully.

**Type Promotion:** The automatic conversion of smaller data types to larger compatible types (like int to double) when no exact matching method is found.

**Unary Operators:** Operators that work with a single operand to perform operations like increment (++), decrement (--), negation (-), and bitwise complement (~).

**Value Assignment:** Setting or updating a value at a particular index in the array using array[index] = value;

**Variable:** A named container that stores data in a program. It must be declared with a data type before use.

**Void Method:** A type of method that executes a block of code but does not return any value to the caller.

**while Loop:** A loop that executes its body repeatedly as long as a specified condition remains true, checking the condition before each iteration

